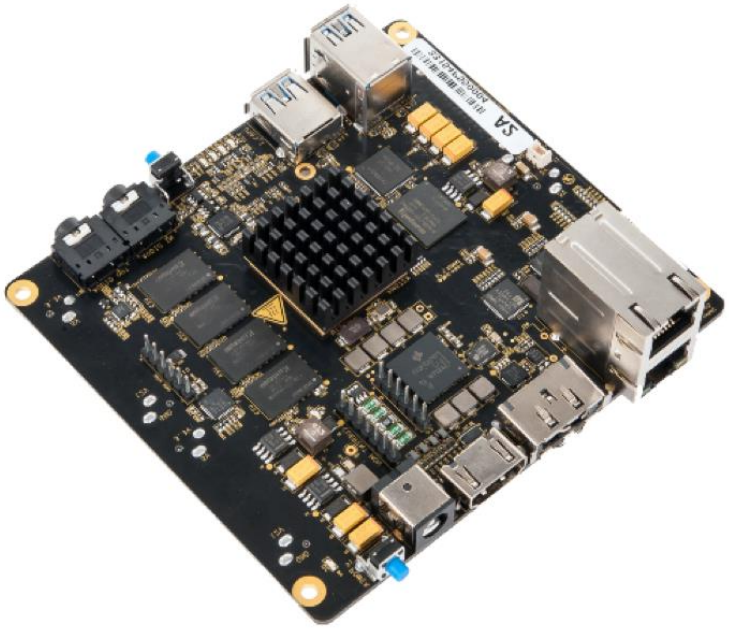


IMPLEMENT AUDIO APPLICATION ON SITARA DSP

-- Final Report

Name: Yixian Lyu
Student number: 00077354
Company: Grass Valley
Version: 2.0
Data: 05-06-2019
HZ University of Applied Sciences

IMPLEMENT AUDIO APPLICATION ON SITARA DSP

-- Final Report

Author	Yixian Lyu
Organization	HZ University of Applied Sciences
TEL	+31(0)655982872
E-mail	lyu0001@hz.nl
Company	Grass Valley
In-company mentor	Jan-Willem Kemna
1st examiner	J.A. Verhage
2nd examiner	J.P.C. Wubben
Place & Date	Breda, 05-06-2019
Version	2.0

Summary

For this thesis project, a new generation of camera, the Titan series is being developed by Grass Valley, and the company would like to have an audio system using AM5728 Sitara processor for the new camera. As for the audio system, there is an audio process system developed for the Sitara processor based on the development board BeagleBoard-X15 in Grass Valley. The audio process system can realize the functions of audio input, changing the volume of the input audio in DSP (Digital Signal Processor), and output processed audio. And the system is using audio data of 24-bit and two channels. For this thesis project, Grass Valley would like to seek the possibility of implementing audio application built in Simulink Audio System Toolbox to the Sitara DSP basing on the audio process system.

So for this project, a software application for the audio implementation system and a hardware system, development system, needs to be designed in this project. The main question for this thesis project is "**What is the design of an Audio Implementation System for the Titan series camera that can realize audio input, run the implemented audio application built in Simulink on the AM5728 Sitara DSP, and output audio based on the Audio Process System?**"

The design method used in this project is the V-model design method. V-model is a requirements-based approach that stresses on verification and validation. The audio implementation system and development system their subsystems were designed and verified using this method.

The Audio Implementation is the software system in this project, and it consists of five subsystems. The Audio Process System is the system that was available in Grass Valley and some modifications were made to make it more suitable for this project. For the Keyboard Capture System, it captures keyboard input and set flags to start or stop audio processing functions in DSP. The Simulink Code System shows the general workflow to implement Simulink audio application to the Sitara DSP. Lastly, the DSP Invoke & Execute System is the configuration for the OpenCL framework to deliver data to the DSP and invoke the DSP to process audio, with the functions chosen by the user using keyboard input.

The Development System is the hardware system, the system is designed to provide a more user-friendly environment for the development basing on BeagleBoard-X15, Linux host machine, and Windows PC. The system consists of four subsystems. The Remote File Transfer System can transfer files between Linux host machine and BeagleBoard in CCS (Code Composer Studio); the Cross-compile System can compile a program in Linux host machine and the compiled application can run on BeagleBoard. The Terminal Connection System can create multiple terminals for the target board in Linux host machine and Windows PC, making it easier for users to send commands to the target board. And the Remote Debug System is a system running in CCS in Linux host machine, with the ability to debug a program running on the target board.

To conclude, the main question for this project has been answered. The deliverables for the entire system is the software application for the audio implementation system and the set up for the development system. The software application has realized the function of implementing audio application built in Simulink to the Sitara DSP.

Foreword

This report is developed from a thesis project done in Grass Valley in Breda, the Netherlands. The title of the project is called “Implement Audio Application to Sitara DSP” and it is suitable for me, as a Mechatronic student from HZ University of Applied Sciences and a Mechanical and Electrical Engineering student from Shanghai Maritime University, to be my thesis project, and to design the audio implementation system and development system in this project.

This thesis project is based on a system called audio process system, which was developed by a student before me in his thesis project and was developed for the AM5728 Sitara processor on the BeagleBoard-X15. The audio process system has already realized the functions of audio input, changing volume in the Sitara DSP and audio output. And this project is to implement audio application built from MATLAB Simulink Audio System Toolbox to the Sitara DSP, to realize more complicated functions to process audio in the Sitara DSP.

This report has been divided into six parts, introducing the company Grass Valley and describing the problem for the project; researching for the knowledge needed in this project; selecting a method for the project; the design results and testing results following in the selected method; discussing the results and answering the sub-questions; and finally, the conclusion for this project and recommendation for future research.

In this thesis project, I have learned a lot of new knowledge, such as OpenCL and audio processing, and learned how to put the knowledge into use. The experience of running into a problem, trying my best to find a solution has been really educating and fun. It was a real pleasure to work in Grass Valley, with all the friendly and experienced colleagues. I have received a great deal of help from my in-company mentor, guiding me through the troubles. The mentor from HZ also provided a lot of effective advice, for my project, for the research proposal and etc. In this foreword, I would like to deliver my sincere gratitude to them.

Yixian Lyu

Breda, the Netherlands

05-06-2019

Abbreviations

ADC	Analog-Digital-Converter
ALSA	Advanced Linux Sound Architecture
ALU	Arithmetic Logic Unit
CCS	Code Composer Studio
CODEC	COder/DECoder
CPU	Central Processing Unit
DAC	Digital-Analog-Converter
DSP	Digital Signal Processor
FFT	Fast Fourier Transformation
FPGA	Field Programmable Gate Arrays
GDB	GNU Debugger
GPU	Graphics Processing Unit
GUI	Graphical User Interface
GV	Grass Valley
I^2S	Inter-IC Sound
IDE	Integrated Development Environment
MPU	Microprocessor Unit
OSS	Open Sound System
PCM	Pulse Code Modulation
RMS	Root-Mean-Square
SDL	Simple DirectMedia Layer
SDLC	Software Development Lifecycle
SoC	System on Chip
SSH	Secure Shell
THD + N	Total Harmonic Distortion plus Noise
TI	Texas Instruments

Table of Contents

1. Introduction 1
1.1 About Grass Valley 1
1.2 Problem Analysis 1
1.2.1 Problem background 1
1.2.2 Problem statement 2
1.2.3 Problem analyses 2
1.3 Research questions 4
1.3.1 Main question 5
1.3.2 Sub-questions 5
1.4 Research Objective 5
2. Theoretical Framework 5
2.1 Basic information for audio processing 5
2.1.1 Audio processing 6
2.1.2 ADC/DAC 6
2.1.3 Inter-IC Sound (I^2S) 7
2.1.4 High-pass filter 8
2.2 Hardware 8
2.2.1 BeagleBoard-X15 & Sitara processor 8
2.2.2 Digital Signal Processor (DSP) 9
2.2.3 Audio analyzer 10
2.3 Software 11
2.3.1 Audio Process System 11
2.3.2 MATLAB Simulink Audio System Toolbox 11
2.3.3 OpenCL 12
2.3.4 Cross compile 13
2.3.5 Code Composer Studio (CCS) 13
2.3.6 Debug tools 14
3. Research method 14
3.1 V-model & Justification 15
3.2 Phases 15
3.2.1 Project preparation phase 16
3.2.2 System design phase 16

3.2.3 Sub-system design phase 16
3.2.4 Component phase 17
3.2.5 Sub-system building & testing phase 17
3.2.6 System building & testing phase 17
3.2.7 System validation phase 18
4. Results 18
4.1 Requirements and wishes 18
4.2 Audio Implementation System 19
4.2.1 System overview 19
4.2.2 System description 20
4.2.3 System integration 21
4.2.4 System test plan 21
4.3 Subsystems for the Audio Implementation System 21
4.3.1 Audio Process System 21
4.3.1.1 Design 21
4.3.1.2 Components 22
4.3.1.3 Subsystem integration (source code) 23
4.3.1.4 Subsystem verification results 24
4.3.2 Keyboard capture system 25
4.3.2.1 Design 25
4.3.2.2 Components 25
4.3.2.3 Subsystem integration (source code) 25
4.3.2.4 Subsystem verification results 26
4.3.3 Simulink Code system 26
4.3.3.1 Design 26
4.3.3.2 Components 26
4.3.3.3 Subsystem integration (source code) 28
4.3.3.4 Subsystem verification results 29
4.3.4 DSP invoke & execute system 32
4.3.4.1 Design 32
4.3.4.2 Components 32
4.3.4.3 Subsystem integration (source code) 33
4.3.4.4 Subsystem verification results 34

4.4 System verification & validation result 34
4.5 Development System 38
4.5.1 System overview 38
4.5.2 System integration 38
4.5.3 System test plan 39
4.6 Subsystems for the Development System 39
4.6.1 Remote File Transfer System 39
4.6.1.1 Design 39
4.6.1.2 Subsystem verification results 39
4.6.2 Cross-compile System 40
4.6.2.1 Design 40
4.6.2.2 Subsystem verification results 40
4.6.3 Remote Debug System 41
4.6.3.1 Design 41
4.6.3.2 Subsystem verification results 41
4.7 System verification & validation result 42
5. Discussion 43
6. Conclusions and recommendations 45
References 46
Appendix I Test plans 49
Appendix II System description 64
Appendix III Detail verification result for Simulink Code System 65
Appendix IV Detail verification result for DSP invoke & execute system 69
Appendix V Components & Integration for subsystems of Development System 71

List of Figures

Figure 1 Audio Process System (Old) vs. Audio Implementation System (New) 3
Figure 2 System description in the previous project (He, 2018) 4
Figure 3 Analog signal vs. digital signal (Krambeck, 2015) 6
Figure 4 Signal change in ADC/DAC (Krambeck, 2015) 6
Figure 5 I^2S Digital Audio Serial Bus Interface Examples (Tomarakos) 7
Figure 6 Example I^2S Timing Diagram for 16-bit Stereo PCM Audio Data (Tomarakos) 7
Figure 7 High-pass filter (Orinj, 2019) 8
Figure 8 BeagleBoard-X15 layout (Kridner, 2015) 8
Figure 9 AM572x block diagram (Texas Instruments, 2018) 9
Figure 10 Typical DSP architecture (Steven W. Smith, 1999) 10
Figure 11 C66x DSP block diagram (Texas Instruments, 2019e) 10
Figure 12 APx 525 audio analyzer (Test equipment Solutions Ltd, 2019) 11
Figure 13 APx500 Measurement Software Interface (Test equipment Solutions Ltd, 2019) 11
Figure 14 Host and compute device for AM572x (Texas Instruments, 2019b) 12
Figure 15 OpenCL execution model (Texas Instruments) 12
Figure 16 Edit view in CCS (Texas Instruments, 2019c) 14
Figure 17 V-model 15
Figure 18 Audio Implementation System overview 19
Figure 19 Decomposition diagram of Audio Implementation System 20
Figure 20 The design of Audio Process System (He, 2018) 22
Figure 21 The flowchart of modified Audio Process System 22
Figure 22 Source code to modified audio data 23
Figure 23 Source code to check the value of processed data 23
Figure 24 Result of modified audio data 24
Figure 25 Decomposition diagram for Keyboard Capture System 25
Figure 26 Flow chart for Keyboard Capture System 25
Figure 27 Source code for Keyboard Capture System 26
Figure 28 Verification results for Keyboard Capture System 26
Figure 29 Decomposition diagram for Simulink Code System 26
Figure 30 High-pass filter audio application in Simulink 27
Figure 31 Flow chart for Simulink code system (Left: Workflow of code from Simulink; Right: Workflow of code running on Sitara DSP) 27

Figure 32 Result value of high-pass filter function 31
Figure 33 Decomposing diagram for DSP invoke & execute system 32
Figure 34 Flow chart for DSP invoke system 32
Figure 35 Flow chart for DSP execute system 32
Figure 36 Source code for DSP invoke system 33
Figure 37 Source code for DSP execute system 33
Figure 38 Result for gain -5dB & signal invert function 34
Figure 39 Initial status (the blue line is BeagleBoard-X15, the red line is a reference) 35
Figure 40 Result of a high-pass filter with cut-off frequency 2kHz 35
Figure 41 Result of a low-pass filter with cut-off frequency at 2kHz 36
Figure 42 Result of gain minus 5 dB 36
Figure 43 Result of signal invert 37
Figure 44 Development System overview 38
Figure 45 Combined workflow of development system & audio implementation system 38
Figure 46 Decomposition diagram for Remote File Transfer System 39
Figure 47 Verification result for Remote file transfer system 40
Figure 48 Decomposition diagram for Cross-compile System 40
Figure 49 Cross-compiled application running on target board 40
Figure 50 Cross-compiled application running on Linux host machine 40
Figure 51 Decomposition diagram for Remote Debug System 41
Figure 52 Verification result for Remote Debug System 41
Figure 53 Development System verification & validation test 42
Figure 54 Audio Implementation System description 64
Figure 55 Development system description 64
Figure 56 Result value of gain minus 5 dB function 65
Figure 57 Result value of signal invert function 66
Figure 58 Result value of low-pass filter function 67
Figure 59 Result value of gain -5dB & signal invert function 69
Figure 60 Result of high-pass & low-pass filter 70
Figure 61 Remote file transfer configuration for BeagleBoard-X15 71
Figure 62 Set up SSH function 71
Figure 63 Remote file transfer system 71
Figure 64 Set up cross-compiler 72

Figure 65 Add libraries for cross-compiler 72
Figure 66 Integration of Cross-compile System and Remote File Transfer System 72
Figure 67 Set up GDB debugger 73
Figure 68 Set up a connection between debugger and target board 73
Figure 69 Start GDB debugger in target board 73

List of Tables

Table 1 Description of different debug tools 14
Table 2 Pros and cons of V-model 15
Table 3 Result for audio process system 24
Table 4 Implement C-code from the high-pass filter audio application on the Sitara DSP 28
Table 5 Result of the high-pass filter, original vs. modified algorithm 29
Table 6 High-pass filter result - Simulink vs. BeagleBoard 30
Table 7 Result verification for the high-pass filter function 31
Table 8 Audio Implementation System verification & validation results and result analysis 37
Table 9 Development system verification & validation result 42
Table 10 Sub-questions and answers 44
Table 11 Result verification for gain minus 5 dB function 66
Table 12 Result verification for signal invert function 67
Table 13 Result verification for low-pass function 68
Table 14 Result verification for gain -5dB & signal invert function 70

1. Introduction

This chapter will introduce and describe the assignment of the thesis project in details, including the company background, complete description of the assignment, research problems and objectives of the thesis project.

1.1 About Grass Valley

Company name	Grass Valley
Street address	Bergschot 69
Postcode	4817 PA
City	Breda
Website	http://www.grassvalley.com/home/

Grass Valley, with headquarter in Montreal, Canada, is the No.1 player in content and media technology. It is a TV production and broadcasting equipment manufacturer with nearly 60 years of history, with main products including cameras, real-time production switchers, editing, routing, conversion equipment, control, and automated playback.

With market-leading solutions, the industry's largest research and development engine, and a growing financial commitment to the future of the industry. With the largest R&D engine in the market, Grass Valley is a driving force for innovation, constantly innovating with game-changing technological advances. Its main products are focusing on live production, news, content delivery and playout, and networking.

The advanced IP solution Grass Valley offers is leading the industry transitioning to software-defined, cloud-based infrastructure. (Grass Valley, 2019).

1.2 Problem Analysis

In this part, the problem for this project will be divided into three parts, problem background, problem statement and problem analyses. The problem background will introduce the background of this project, what already existed in Grass Valley and why is the project needed. The problem statement will describe what needs to be done in this project. And the problem analyses will describe the relationship between the Audio Implementation System in this project, the Audio Process System built by a former student in the previous project, and the Digital Audio Signal Processing Model being developed by a fellow student.

1.2.1 Problem background

A new generation of camera, the Titan series, is being developed by Grass Valley. The company intends to develop an audio system for the Titan series camera using AM5728 Sitara processor and hopes to use the DSP (Digital Signal Processors) inside this Sitara processor to improve the calculating performance.

Additionally, in order to use the processor, the company has decided to use a BeagleBoard-X15, which contains the AM5728 processor, as the platform to develop the audio system.

The old processor used in previous cameras has CPU (Central Processing Unit) and DSP on two separate chips, the dedicated DSP would cost the company at least $10,000 per year. And with the AM5728 Sitara Processor, the CPU and DSP has been integrated on one chip, which can not only reduce the number of chips used but also save at least $10,000 per year (He, 2018).

At this point, an audio process system for the BeagleBoard-X15 has been designed in the previous project. The audio process system can realize the functions of audio in, audio out and use the CPU to invoke the DSP to do some simple audio processing such as increase and decrease volume (He, 2018). Based on this audio process system, Grass Valley would like to go more into the development of using DSP to process audio. However, since manually programming the signal processing algorithm into the DSP is fairly difficult and not standardized (Wong, 2018), GV would like to seek the possibility of generating the audio processing algorithms from audio application, such as a high pass filter, built in Simulink Audio System Toolbox, and implement the audio applications on the DSP so that the audio processing algorithms can be run on the DSP.

For now, MATLAB Simulink does provide support packages to run the audio applications on certain boards, such as BeagleBone Black, BeagleBoard-xM (MathWorks, 2017). However, it does not have any support for the BeagleBoard-X15 or the AM5728 Sitara processor, even in the support package for supported boards, it does not provide support for the audio application to run on the DSP. Therefore, it is necessary for this project to find a way to implement the Simulink audio application on the AM5728 Sitara DSP.

1.2.2 Problem statement

On BeagleBoard-X15, there are two 750-MHz C66x DSP cores and two Arm Cortex-A15 CPU within the AM5728 Sitara processor. And the task for this project is to research and set up the complete workflow to implement audio applications built in Simulink on the C66x DSP. More specifically, is to implement the C-code, which represents the audio processing algorithm, generated from the audio application on the Sitara DSP.

1.2.3 Problem analyses

Figure 1 shows the relation between the Audio Implementation System, Audio Process System and the Digital Audio Signal Processing Model. The Audio Implementation System is the system to be designed in this project, the Audio Process System is the system that is currently available in Grass Valley, and the Digital Audio Signal Processing Model is under development by a fellow student. The design of the Audio Implementation System is based on the Audio Process System and use the audio application model from the Digital Audio Signal Process Model. When comparing the Audio Process System to the Digital Audio Signal Process Model, the Sitara processor is acting the role of MATLAB/Simulink, however, unlike MATLAB, the processor does not know what to do with the audio application when there are input and output. Which is why the Audio Implementation System is needed to manually implement the audio application on the Sitara DSP.

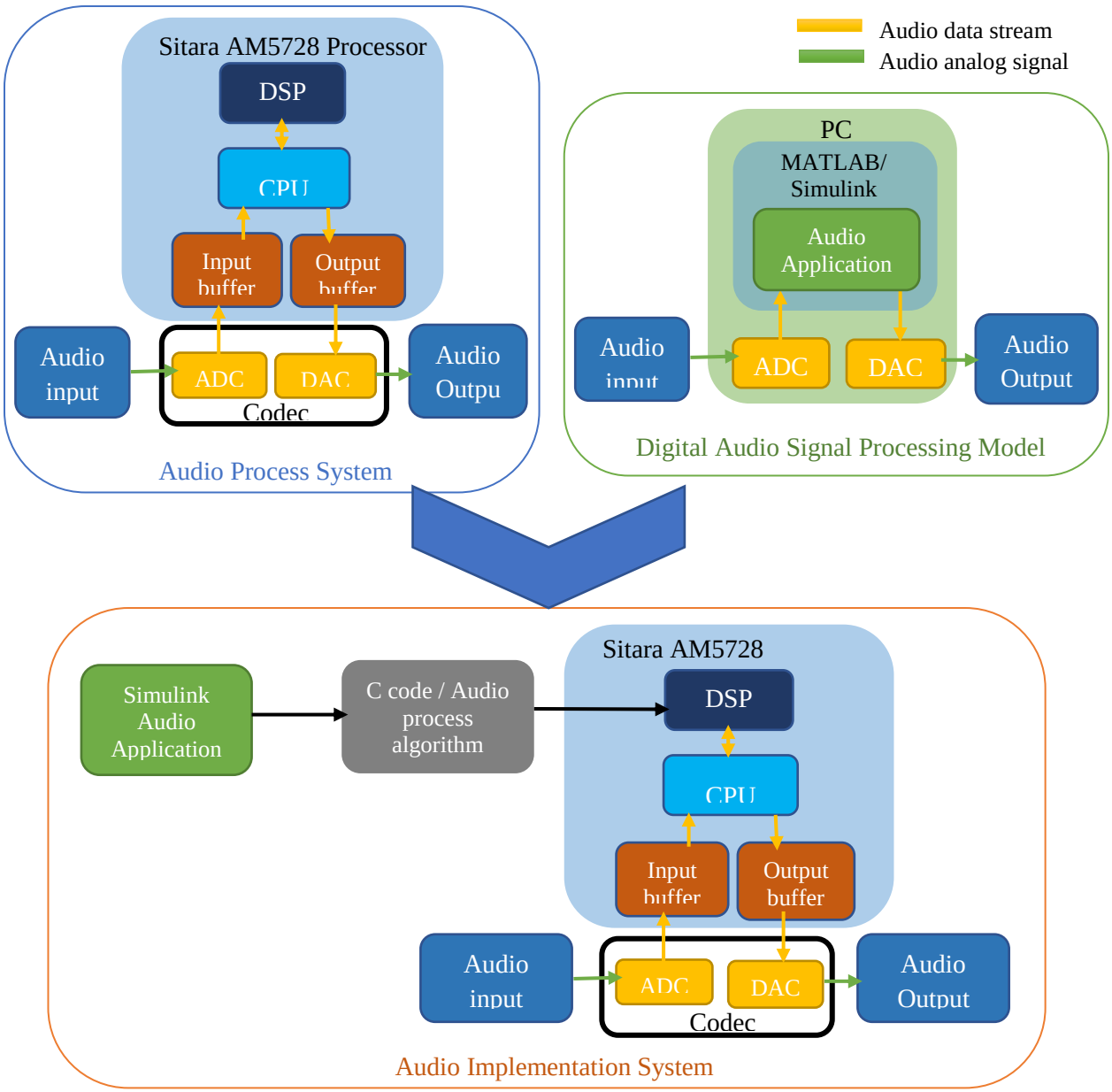


Figure 1 Audio Process System (Old) vs. Audio Implementation System (New)

For the workflow of the Audio Implementation System, first the system needs to convert the analog audio signals input into digital audio data stream using ADC (Analog-Digital-Converter), transfer the data stream to input buffer and send it to the CPU on Sitara processor, then the CPU would invoke the DSP to run the audio process algorithm from the Simulink audio application. After this step, the original audio stream would be processed, and the processed data stream would be sent back to the CPU, convert into analog signals using DAC (Digital-Analog-Converter), and finally output the analog signals to external devices such as headphones or speakers. Furthermore, GV would also like to know if it is possible to transfer the audio data stream between buffer and DSP without using CPU, which is also a part to be researched for this project.

In the previous project, the student also set up a Development System to connect and communicate between the Windows PC, Linux host machine and BeagleBoard-X15. *Figure 2* shows the system description for Audio Process System and Development System in the previous project (He, 2018). In this Development System, the BeagleBoard-X15 is connected to the audio analyzer and Windows PC via cables, the Linux host machine is a virtual machine built in the Windows PC. The Windows PC is in charge of the connecting ports, transferring files and processing user commands. The Linux host machine is to transfer the user commands and files to the target Linux system, which is the system running on BeagleBoard-X15. The network connection between the three systems has also been set up.

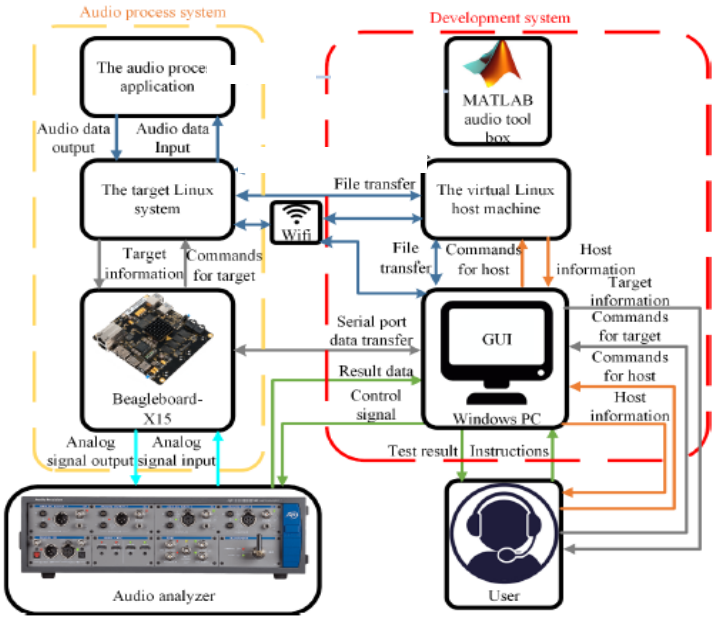


Figure 2 System description in the previous project (He, 2018)

As shown in *Figure* 2, the workflow between the Audio Process System and the Development system is the audio analyzer connects to the BeagleBoard-X15 would input analog audio signal, the user input a command to execute the audio process application, and the signal would be processed in the Audio Process System, then the processed analog audio signal would be output back to the audio analyzer. The changes of the audio signal are shown in the audio analyzer's software on the Windows PC, which can be seen by the user, the user can also modify the input signal with the software. If any error occurs when executing the audio process application, the target would return an error message to the GUI (Graphical User Interface) to inform the user (He, 2018).

In this project, it is also needed to further develop and modify the Development System, which would be the hardware part of this project, to have a deeper and easier connection between the Windows PC, Linux host machine and the BeagleBoard-X15. The cross-compile and remote debug between Linux host machine and target board, which is the BeagleBoard-X15, would be designed, and the file transfer would be modified.

1.3 Research questions

In this part, the main question and sub-questions for this project will be raised and the objective of this research will be defined.

1.3.1 Main question

What is the design of an Audio Implementation System for the Titan series camera that can realize audio input, run the implemented audio application built in Simulink on the AM5728 Sitara DSP, and output audio based on the Audio Process System?

1.3.2 Sub-questions

1. What are the hardware and software needed for the Audio Implementation System?
2. What audio application model to use from Simulink Audio System Toolbox?
3. What modification needs to be made to the C-code generated from the Simulink audio application?
4. What is the way to run the implemented audio application on the Sitara DSP?
5. What is the possible way to bypass CPU to transfer audio data stream between buffer and DSP?
6. What are the descriptions and results of the test plans and validation plan for the systems?
7. What are the descriptions and results of the test plans for the sub-systems?

1.4 Research Objective

The research objective is to find a solution to transfer the input audio data stream to the Sitara DSP, implement the C-code from a model built in Simulink Audio System Toolbox on the Sitara DSP, and finally output the processed data. And preferably not to use CPU in the process. Therefore, the software for transferring data to DSP and compiling on DSP needs to be designed. Additionally, some functions, such as cross-compile, need to be added to the development system, some modifications would also be made to enhance the communication between the Windows PC, Linux host machine and BeagleBoard-X15.

2. Theoretical Framework

In this chapter, the theoretical framework for the new system will be described according to three parts, introducing the basic information for audio processing, hardware, and software related to this project.

For the basic information for audio processing, the different types of audio signals and how to convert the types of signals will be introduced, and the standard for transferring audio data, Inter-IC Sound (I^2S), will be described. In the hardware part, the BeagleBoard-X15 including the AM5728 Sitara processor will be introduced, and the general information about Digital Signal Processor (DSP) and the DSP used in this project will be described. Also, the audio analyzer used in this project as audio input and output will be introduced. For the software in this project, first, the program for the Audio Process System will be introduced, then is the Simulink Audio System Toolbox used to generate algorithms for audio processing. The OpenCL framework used to invoke DSP will be described. Lastly, the tools for building the program, cross-compile, Code Composer Studio (CCS) and debug tools will be introduced.

2.1 Basic information for audio processing

For this part, some general information about audio processing will be introduced, including audio processing, ADC/DAC and I^2S.

2.1.1 Audio processing

There are two types of audio processing, analog processing and digital audio processing. Analog processing converts sound waves into electrical signals with different amplitudes. When it is in the form of electrical signals, the signals can be easily manipulated. The electrical signals used in analog devices are very similar to sound waves, which allows for minimal distortion in the processing of sound.

In digital audio processing, audio signals are converted into digital information, usually binary code, that a computer can interpret, and each bit of data is represented by two different amplitudes. Digital audio processing can process different types of signals to filter, measure, compress and generate analog signals. Digital signals transform the properties of sound from continuous waves to discrete packets. The sounds can be recombined so that the human ear would not be able to recognize the difference between digitally processed sounds and non-digitally processed sounds. Comparing to the analog signal, digital audio processing can control the audio signal better (Fox, 2019).

Another notable difference is that analog signals can be represented as sine waves and digital signals as square waves, as shown in *Figure 3*. Digital signal processing can be applied to almost any field, whether it is oil processing, sound reproduction, medical image processing, or telecommunications -- basically any application of compression and reproduction of signals (Krambeck, 2015).

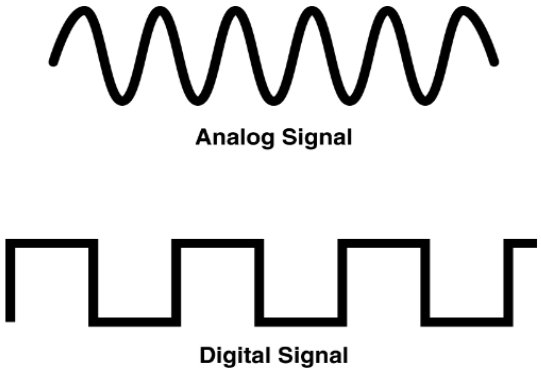


Figure 3 Analog signal vs. digital signal (Krambeck, 2015)

2.1.2 ADC/DAC

In the actual world, signals such as sound, light, pressure or temperature can be detected by analog products. These signals are analog signals, in order to process these signals, as shown in *Figure 4*, an analog-to-digital converter (ADC) is needed to convert the actual signal into a digital format of 1 and 0 (Analog Devices, 2019) and then send to a digital system, such as DSP, for processing. And after the signals have been processed in the digital system, it would be changed back to analog signals using a digital-to-analog converter (DAC) to output the audio to audio output devices such as a loudspeaker. And in an embedded system, ADC and DAC together are called CODEC (COder/DECoder) (Heath, 2017).

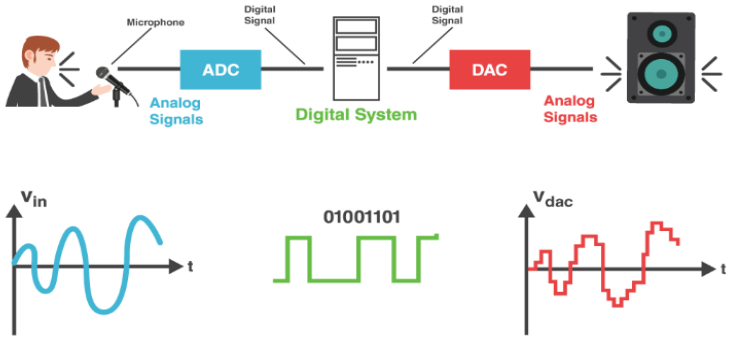


Figure 4 Signal change in ADC/DAC (Krambeck, 2015)

Inside the BeagleBoard, the ADC (Analog-Digital-Converter) and DAC (Digital-Analog-Converter) is integrated into the sound card TLV320AIC3104. The ADC and DAC support rates from 8kHz to 96kHz and the DAC supports data of 16/20/24/32-bit (Texas Instruments, 2007).

2.1.3 Inter-IC Sound (I^2S)

I^2S (Inter-IC Sound), in this project, serves as a hub for audio data communication between DSP and the sound card TLV320AIC3104, which integrated the ADC and DAC.

In recent years, the DSP uses a digital audio serial protocol named I^2S (Tomarakos). The Integrated Inter-IC Sound Bus is the serial bus interface standard to connect digital audio devices together (Cypress Semiconductor Corporation, 2016), it is used in consumer and professional audio products such as CD players, digital TV sound and digital sound processor. I^2S is designed to separate audio data from clock signals, and by separating the data from the clock signal, the time-related errors that cause jitter will not occur, thus eliminating the need for anti-jitter devices (Rouse, 2005).

I^2S signals, shown in *Figure 5* and 6, consist of a serial clock, Left/Right Clock (also known as the Word Select) and serial data (SDATA), regardless of the bits of data valid in the signal of I^2S format, the highest bit of data always appears at the second SCLK pulse after the change of LRCK (Left/Right Clock), which is the beginning of a frame). If the value of LRCK is "1", it means that the data is transmitting from the right channel; if it is "0", it means that the data is transmitting from the left channel. The frequency of LRCK equals the sampling rate. This allows the number of significant digits at the receiving end to be different from the number of significant digits at the sending end. If the receiver can process less significant bits than the sender, the excess bit data in the data frame would be discarded. If the receiver can process more significant bits than the sender, it can make up the remaining bits. This synchronization makes it easier to interconnect digital audio devices without data misalignment and this multiplexing protocol only requires one data path to send and receive two channels of digital audio information (Tomarakos).

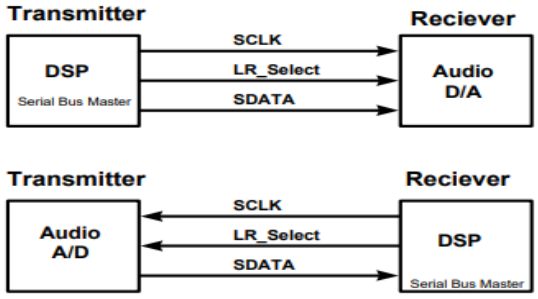


Figure 5 I^2S Digital Audio Serial Bus Interface Examples (Tomarakos)

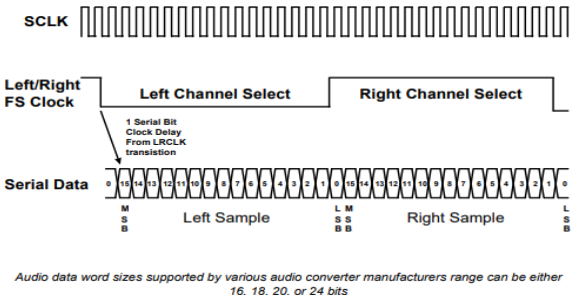


Figure 6 Example I^2S Timing Diagram for 16-bit Stereo PCM Audio Data (Tomarakos)

In this project, the parameters of the audio sample are defined by the company Grass Valley, with a sample rate of 48kHz, 2 channels and the data bit format are 24-bit, little endian.

2.1.4 High-pass filter

The audio application built in Simulink that needs to be implemented on the Sitara DSP in this project is a high pass filter. A high pass filter can attenuate the low frequencies below a certain cut-off frequency and allows frequencies above that cut-off frequency to pass (Sweetwater, 2016). As shown in *Figure 7*, the cut-off frequency is 100 Hz, frequency below 100 Hz has been filtered out, and frequency above 100 Hz can pass through.

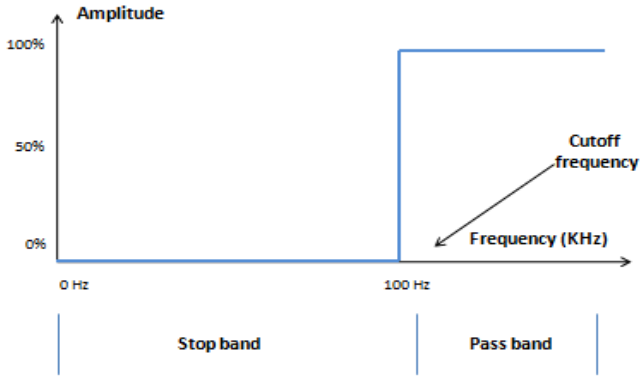


Figure 7 High-pass filter (Orinj, 2019)

2.2 Hardware

For the hardware, the system will be applied on BeagleBoard-X15. The part that concerned this project is the Sitara processor on BeagleBoard, DSP, and audio input.

2.2.1 BeagleBoard-X15 & Sitara processor

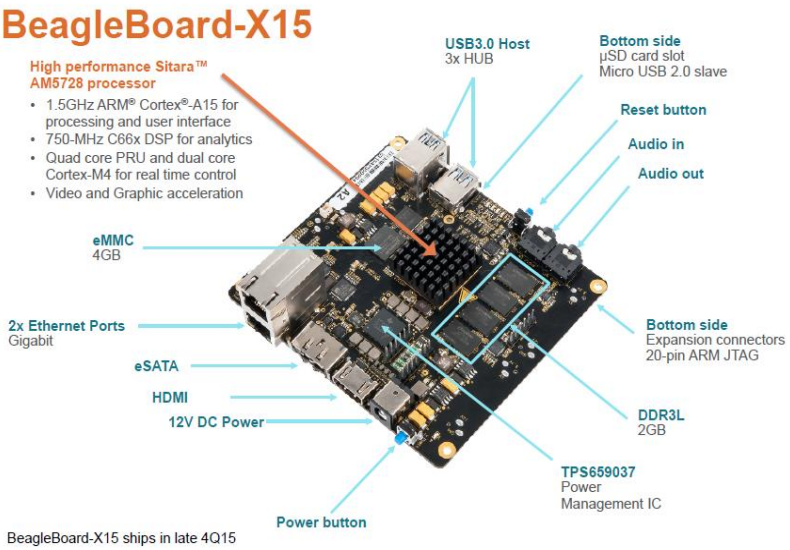


Figure 8 BeagleBoard-X15 layout (Kridner, 2015)

The BeagleBoard-X15 is the best performing, Linux-enabled mainline, the superuser's dream board, custom-made for each computing task and high-speed interface for each connection requirement. Multiple cores have been integrated into the BeagleBoard, including a TI AM5728 processor with 2×1.5-GHz ARM® Cortex-A15 and 2×700-MHz C66 digital signal processors (DSPs) (BeagleBoard, 2019). *Figure 8* shows the primary components on BeagleBoard-X15, with USB ports, Ethernet ports, Audio in/out jack, Sitara AM5728 processor, etc.

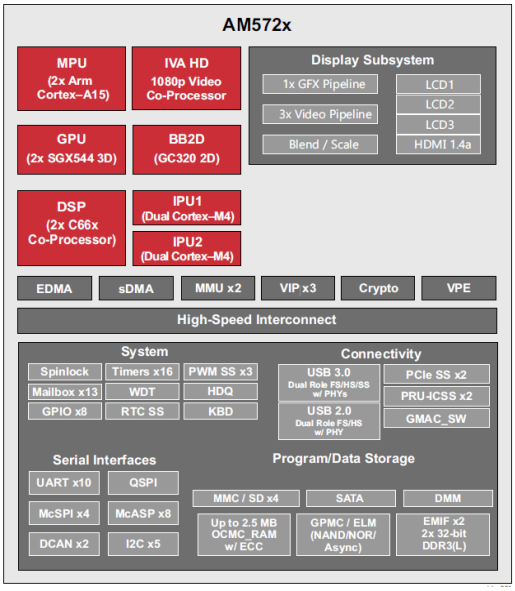


Figure 9 AM572x block diagram (Texas Instruments, 2018)

Figure 9 shows the block diagram of the AM572x Sitara processor. The AM5728 Sitara processor is a multi-core heterogeneous processor that integrated different cores, such as DSPs (Digital Signal Processors), GPUs (Graphics Processing Units), MPUs (Microprocessor Units), etc. (Texas Instruments, 2018). The TI Sitara AM572x is also a System-on-Chip (SoC) that builds around a dual-core ARM Cortex-A15 CPU. An SoC contains both hardware and software, it consumes less power, has higher performance, takes up less space, and is much more reliable than multi-chip systems. However, an SoC requires higher prototyping and architecture cost and adds the complexity in debugging (Techopedia, 2019).

2.2.2 Digital Signal Processor (DSP)

Since the 1960s, with the rapid development of computer and information technology, digital signal processing technology emerged and developed rapidly. Before the DSP chip, digital signal processing can only rely on the microprocessor to complete. But due to the low processing speed of the microprocessor, it cannot satisfy the need for high-speed information processing. Up till now, DSP chips are mostly used in the field of communication, then is used in the computer field and consumer electronics, accounting for 56.1%, 21.16% and 10.69% respectively (Stranneby, 2004).

Digital signal processor (DSP) digitizes real-world signals such as video, sound, audio, temperature, pressure, or position, and then performs mathematical processing on them. DSP is designed to quickly perform mathematical functions such as addition, subtraction, multiplication, and division (Analog Devices, 2019). *Figure 10* shows the architect of the DSP using the super Harvard architecture. The I/O (Input/Output) controller connects to the data memory, this is the path for the digital signals to enter and exit the system. There is an independent address bus and an independent data bus, typically, three steps are required to complete an instruction, including fetch instruction, instruction decoding, and instruction execution. Because of the different storage space and different bus for fetching and accessing data, each instruction can be executed simultaneously. In this way, the bottleneck of data flow transmission can be overcome, and the computing speed is improved (Steven W. Smith, 1999).

The mathematical processing in DSP is divided into three parts, multiplier, arithmetic logic unit (ALU) and barrel shifter as shown in *Figure 10*. The multiplier obtains value from two registers, multiply the value and then put the results into another register. The ALU can perform subtraction, addition, absolute value,

logical operations including AND, OR, XOR, NOT, and can also perform conversions between fixed- and floating-point formats. Basic binary operations are performed by the barrel shifter, such as rotating, shifting, extracting, and depositing segments (Steven W. Smith, 1999).

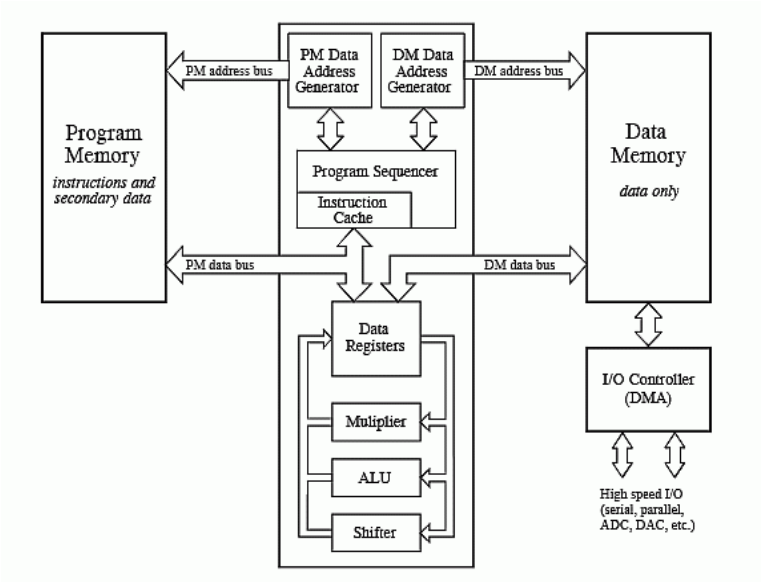


Figure 10 Typical DSP architecture (Steven W. Smith, 1999)

In this project, the TI C66x DSP cores used have integrated both fixed- and floating-point capability, which can eliminate the expensive algorithmic conversions from floating- to fixed-point and allow developers to create high-precision code that runs on high-performance devices. The C66x DSP has a remarkable performance at mathematical operations and data movement. It is suitable for medical imaging, machine vision, testing and measurement and other analysis functions (Kridner, 2015). *Figure 11* shows the block diagram of the C66x DSP, integrated two DSP cores, several I/O, communication, Lanes and system services (Texas Instruments, 2019e).

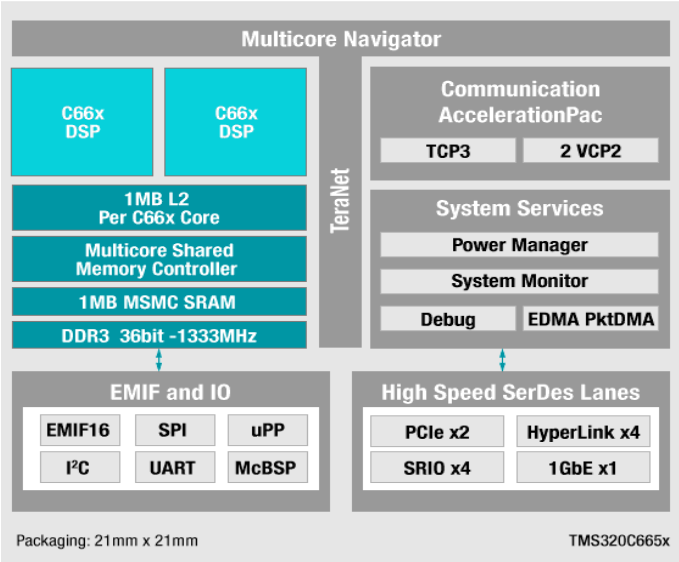


Figure 11 C66x DSP block diagram (Texas Instruments, 2019e)

2.2.3 Audio analyzer

In this project, the APx audio analyzer is used as the audio input and audio output for the BeagleBoard.

The APx audio analyzer provides world-class performance and flexibility (Test equipment Solutions Ltd, 2019). *Figure 12* shows the APx 525 audio analyzer, with two analog input and output channels. Together

with the APx500 measurement software, as shown in *Figure 13*, it can measure and show the RMS of the audio signal, the latency, gain, and frequency between the audio input and output. It is also possible to generate waveform such as a sine wave with the device.



Figure 12 APx 525 audio analyzer (Test equipment Solutions Ltd, 2019)

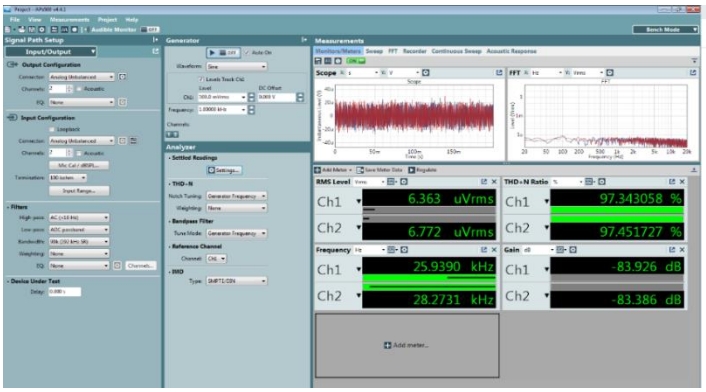


Figure 13 APx500 Measurement Software Interface (Test equipment Solutions Ltd, 2019)

The measurement value in In *Figure 13*, the RMS (Root-Mean-Square) level represents the effective value of the total waveform, the value of THD +N (Total Harmonic Distortion plus Noise) is the sum of all harmonic distortion plus the noise in one measurement. The frequency shows the highest frequency in the audio and the gain represents the loudness of the audio (Winters., 2014).

2.3 Software

For the software in this project, first it is necessary to have an idea of what the old audio process system can do, then is to find the software needed for implementing the C-code on the Sitara DSP.

2.3.1 Audio Process System

The audio process system, as shown in *Figure 1*, built in the previous project has realized the workflow of audio input, convert analog data stream to digital, transferring the audio stream to CPU, invoking the DSP with OpenCL to process data, capture keyboard input to change volume, convert digital data stream to analog and output the processed data. And a Development System, as shown in *Figure 2*, for the BeagleBoard has also been set up, connecting the Windows PC, Linux host machine and the BeagleBoard via the internet, and transferring files between Windows, Linux, and BeagleBoard (He, 2018).

2.3.2 MATLAB Simulink Audio System Toolbox

Audio system toolbox provides algorithm and tools to design, simulate audio processing systems, including audio processing algorithms, sources and measurements, such as filtering, audio oscillators and waveform synthesizers. It also contains an interface to external MIDI controls and low latency audio drivers, for example, ALSA (Advanced Linux Sound Architecture), the audio development tool for this project. Additionally, the audio system toolbox supports C/ C ++ code generation for simulation acceleration or desktop prototyping (MathWorks, 2019a), the generated C code can be used anywhere as the user desires, as long as it contains a C compiler (MathWorks, 2019b).

2.3.3 OpenCL

OpenCL is the framework used in the Audio Process System in the previous project to invoke the DSP, and the OpenCL framework and the associated kernel would be modified in this project.

OpenCL is a framework for representing programs that perform parallel computing across heterogeneous devices, which means it is a way to communicate between ARM and DSP. OpenCL is an open, royalty-free standard that maintained by the technology alliance Khronos group (Texas Instruments, 2019a), it is essentially a subset of C and is a very familiar environment for developers who are familiar with C programming (Littlefield, 2014). There are many advantages of using OpenCL, including the fact that it can facilitate the portability between devices, and does not require knowledge on memory architecture or DSP coding (Texas Instruments). OpenCL code is by nature easier to create, read, understand and maintain than any other DSP assembly language code (Littlefield, 2014).

As for the necessity of using OpenCL on TI Sitara, firstly, the processor ARM Cortex-A15 on AM5728 is a powerful processor, but it is not as efficient as DSP when comes to handling mathematical models. Secondly, the OpenCL kernel can run many times, allowing the ARM to perform other functions at the same time (Texas Instruments). A kernel can be thought of as a C function in OpenCL that contains the calculations waiting to be performed on the target device (Littlefield, 2014). The device for OpenCL can be CPU, GPU, DSP, FPGA (Field Programmable Gate Arrays), etc., the host and compute devices for a platform is defined by the OpenCL specification, the host and compute device for AM572x is as shown in *Figure 14*, with 2 ARM Cortex-A15 CPUs as host and 2 C66x DSP compute units as compute device.

SoC	System	Host	Compute Device
AM572	AM572 EVM	2 ARM Cortex-A15 CPUs, SMP Linux	1 device with 2 C66x DSP compute units

Figure 14 Host and compute device for AM572x (Texas Instruments, 2019b)

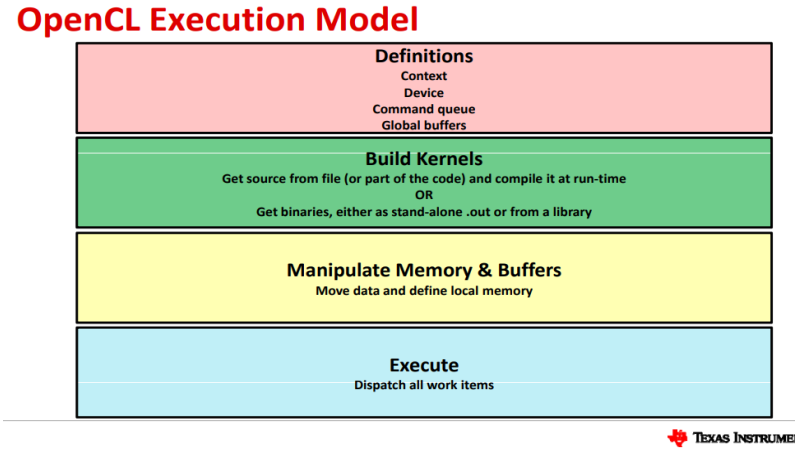


Figure 15 OpenCL execution model (Texas Instruments)

Figure 15 shows the execution model of OpenCL. First is to define the kernel, the definitions will inform OpenCL the architecture of the target device, in this case, the DSP. Then the kernel will build the program for the device, and finally, create buffers for the OpenCL device from memory allocated in the host, which is moving the data to DSP and executing on DSP in this case.

When an OpenCL kernel code is running on DSP side, when using a *printf* function to print out statements, the output of *printf* from DSP side will automatically be prefixed with the DSP core number that the kernel code is running on. For example, when the code in the kernel is "printf(v = <1,5>)", the output of the *printf* will be "[core 0] v = <1,5>", which means the code is running on the core 0 of DSP (Texas Instruments, 2019f). This can also be the easiest way to check if the code is running on DSP.

2.3.4 Cross compile

A compiler is a program that can convert source code into executable code. Like all programs, a compiler runs on a particular type of computer, and the executable it outputs run on a particular type of computer.

The computer on which the compiler runs is called the host, and the computer on which the new program runs is called the target. For this project, the host would be the Linux host machine and the target would be the BeagleBoard-X15. When the host and target machine are the same types of machine, the compiler is the native compiler. When the host and target are different types of machines, the compiler is a cross-compiler. In other terms, cross-compile is building a binary on one platform that will run on another device or platform (Introduction to cross-compiling for Linux, n.d.).

For this project, the BeagleBoard can compile the program on its own since it has a compiler installed in the system, however, compiling can be time-consuming, especially when developing large software. The advantages of using a Linux host machine to compile for the target board BeagleBoard are:

- Speed: The target platform is usually much slower than the host machine, since most embedded hardware are designed for low cost and low power consumption instead of high performance.
- Capacity: Compiling has a high demand for resources and the target platform typically does not have the hundreds of Gigabyte of disk space that the host have.
- Flexibility: Hundreds of packages are included in a fully capable Linux distribution, but a cross-compilation environment can depend on a host's existing distribution. Cross-compilation is focusing on building the target package to deploy, rather than taking the time to obtain prerequisites for building only the target system (Introduction to cross-compiling for Linux, n.d.).

2.3.5 Code Composer Studio (CCS)

In this project, CCS would be the development environment for cross-compiling and debugging the program, the debug tools will be further introduced in *Chapter 2.3.6*.

CCS is an integrated development environment (IDE) for TI microcontrollers and embedded processors. Code Composer Studio includes a set of tools for developing and debugging embedded applications. It provides an optimized C/ C ++ compiler, source editor, project build environment, debugger, and many other features. CCS combines the strengths of the Eclipse software framework with the advanced embedded debugging capabilities provided by TI to provide a compelling and feature-rich development environment for embedded developers. And it also provides support for the Sitara AM5738 processor and the C66x DSP (Texas Instruments, 2019c).

Figure 16 shows an Edit view in CCS, this is the environment for programming and building projects.

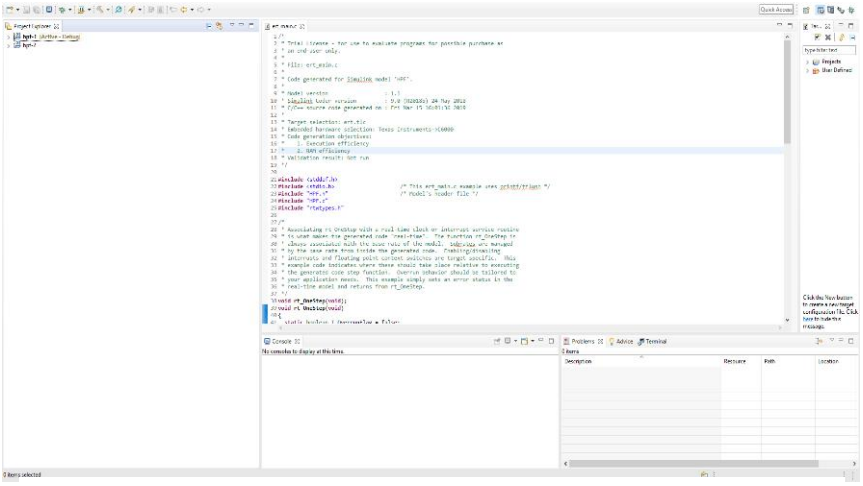


Figure 16 Edit view in CCS (Texas Instruments, 2019c)

2.3.6 Debug tools

To set up a debugger to debug the program runs on BeagleBoard-X15, there are several debug tools available. The descriptions for the different tools are as shown in *Table 1* (Texas Instruments, 2019).

Debug tools	Description
Printf	It is the easiest way to debug the program, simply using the *printf* function. However, the function needs to be manually inserted and if an error occurs, it is hard to tell exactly where went wrong.
GDB (GNU Debugger)	The GDB debug is a run mode debug, which allows the user to debug one or multiple Linux processes. The GDB is also applicable in CCS.
JTAG	The JTAG debug is a sleep mode debug runs in the CCS. It can show more features about the target board, such as the module and registers data than using GDB, however, it requires a JTAG emulator, the price for the JTAG emulator suitable for BeagleBoard-X15 range from $99 to $ 1495 (Texas Instruments, 2019d).
dsptop	The dsptop can debug which DSP cores are executing and can show the kernel activities with timestamps. However, the processor no longer contains the dsptop utilities by default, and it is running on the terminals instead of having a GUI such as the debug view in CCS.

Table 1 Description of different debug tools

3. Research method

In this chapter, the research method, V-model, will be introduced, the phases in V-model and the deliverables for each phase will be described.

3.1 V-model & Justification

The success of any project is based on choosing the right method. The software development lifecycle (SDLC) is critical to the successful implementation of any software project (Sorina, 2016), and V-model is one of the SDLC models and is a requirements-based approach that stresses on verification and validation. The phases of V-model are as shown in *Figure 17*, the left side of the 'V' represents Verification phases while the right side represents Validation phases. The two sides are joint together by the coding phase (Tutorials Point, n.d.).

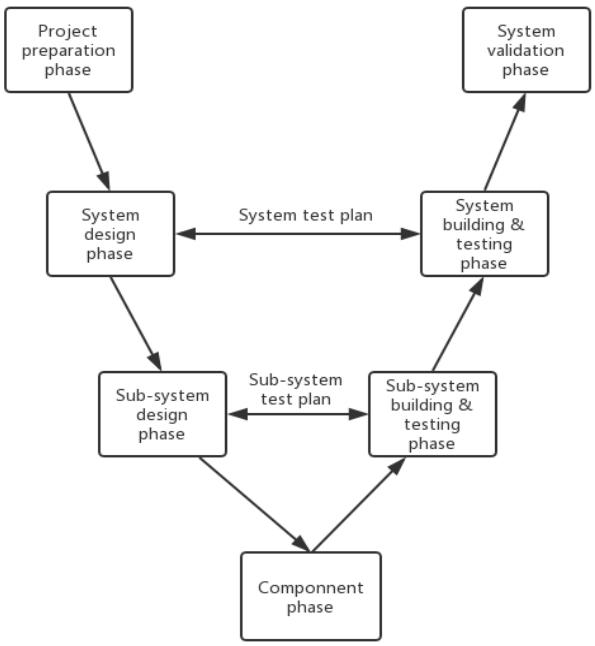


Figure 17 V-model

The reason of using SDLC method for this project instead of the general design method, such as Delft design method, is that the project focuses on the design of software and hardware system, the design can be divided into subsystems, requires coding and testing. And the final product of this project is a program for Audio Implementation System. Since V-model put more emphasis on testing than the other SDLC methods such as Waterfall and spiral, it is the most suitable for this project. The pros and cons of V-model are listed in *Table 2* (Tutorials Point, n.d.). As this project is a small project to design a software system, with deliverables as a program, and a hardware system, with the configurations for the setup and needs to test if the systems are sufficient, V-model is the most suitable choice.

V-model	
Pros	Cons
Each phase has its own specific deliverables.	The V-model lacks flexibility.
The test plan is developed early on in the life cycle, which means it has a bigger chance in succeeding than Waterfall model.	Does not provide a clear path for problems discovered during the test phases.
Ideal for small projects.	It is difficult and expensive to make adjustments to the V-model.
Works well on restricted projects.	

Table 2 Pros and cons of V-model

3.2 Phases

The project is divided into 7 phases using V-model, which is project preparation phase, system design phase, sub-system design phase, component phase, sub-system building & testing phase, system building & testing phase and system validation phase. The description and deliverables are included in each phase in this chapter.

3.2.1 Project preparation phase

In the project preparation phase, first is to rebuild the old audio process system, then to research and have a deeper understanding of the program for the audio process system, how does it execute and to make planning for this project. The sub-question 1 can be answered in this phase.

The deliverables for the preparation phase are:

- Test plan for the audio process system: This test plan should basically be the same as the system test plan from the previous project. The point of this test plan is to check if the rebuild of the old system is successful.
- Test result for the audio process system: The test result is acquired when executing the test plan for the audio process system, to prove that the rebuilding is successful.
- A list of requirements and wishes: Determine the requirements and wishes for the project together with the in-company mentor.
- Doing research for the hardware and software used in the previous audio process system and will be used for this project, such as DSP, OpenCL, MATLAB Simulink Audio System Toolbox.
- A system workflow for the overall system: Create a workflow combining the software and hardware system.
- Validation plan for the overall system: Set up validation plan to test the overall system.

3.2.2 System design phase

In the system design phase, the overall system can be divided into software and hardware system. The hardware system is based on BeagleBoard-X15, the audio input and output. In the case, the audio input and output can be realized by using APx 525 audio analyzer, other forms of audio input and output can also be tried out. In this phase, the sub-question 6, and part of sub-question 7 can be answered.

And the deliverables for the system design phase are:

- System decomposition chart: The division between hardware and software systems in the whole system.
- System description diagram for the software system: Specify the inputs and outputs for the software system and figure out the relationship between sub-systems.
- System description diagram for the hardware system: Specify the inputs and outputs for the hardware system and figure out the relationship between sub-systems.
- System test plans: Setup test plans for both hardware and software systems on the system level.

3.2.3 Sub-system design phase

In the sub-system design phase, several sub-systems will be divided from the software and hardware systems. The parts and connections within the sub-system will be designed and specified. The sub-question 2, 5, and part of sub-question 8 can be answered.

The deliverables for the sub-system design phase are:

- Sub-systems decomposition diagrams for hardware and software systems: Specify the inputs and outputs for the sub-systems.
- Software functions for the software: Determine what functions need to be added and/or adjust based on the program for the audio process system, to compile the C-code from the MATLAB model on DSP.
- Software function flow chart: The flow chart for the functions on the sub-system level.
- Sub-systems test plans: Setup test plans for each sub-system on sub-system level.

3.2.4 Component phase

In the component phase, the focus will be on the coding part. For this phase, sub-question 3, 4 should be answered.

The deliverables for the component phase are:

- Software sub-functions for the software: Design the sub-functions to realize the functions on sub-system level.
- Software sub-function source code: The source code for the sub-function.
- The components for the development system: The components for development system will be the configuration between BeagleBoard-X15 and Linux host machine.

3.2.5 Sub-system building & testing phase

In this phase, the sub-systems for hardware and software will be built up and tested. The sub-systems are integrated with components, and then the tests will be executed according to the test plans for sub-systems. The test results will be recorded and analyzed. If the sub-system failed the test, it would be modified until it passes the test. For this phase, sub-question 8 should be answered.

The deliverables for the sub-system building & testing phase are:

- The built sub-systems: The sub-systems combined with components.
- Test results for sub-systems and the analysis: Test the sub-systems following the test plans and analyze the results.
- The final sub-system: Make changes to the sub-systems if it failed the test and get the final design of the sub-system when it passes the test.

3.2.6 System building & testing phase

In this phase, the system level of hardware and software will be built up and tested. The systems are combined with the sub-systems, and the tests will be executed according to the system test plans. The test

results will be analyzed, and modifications will be made to the system until it passes the test plan. For this phase, sub-question 7 can be answered.

The deliverables for the system building & testing phase are:

- The built systems: The systems combined with sub-systems.
- Test results for systems and the analysis: Test the systems following the test plans for system level and analyzed the results.
- The final systems: Make changes to the systems if necessary, till the systems pass the tests and get the final design of the systems.

3.2.7 System validation phase

In the system validation phase, the design of audio implementation system is finished. To validate the project, the test report and user manual is needed, and the final system will be delivered to Grass Valley in this phase. The answer for sub-question 7, 8 will be documented in this phase.

The deliverables for the system validation phase are:

- Test results for the overall system and the analysis: Test the hardware and software system following the validation plan for the overall system.
- The test report: A test report with all the recorded test results.
- A setup manual: A setup manual with detail instructions to inform the colleagues how to set up the audio implementation system. It can also be a reference for further development based on this project.

4. Results

In this chapter, first, the requirements and wishes for the assignment will be described, then all the results for this project will be explained. The design of the systems, subsystems, and components are according to the requirements and wishes and following the V-model design method. The design, integration, test plan and verified result for each system and subsystem will be explained in this part. Though the design of the software system, Audio Implementation System, and the hardware system, Development System, is separate, they can be combined together to realize the overall system workflow. The Audio Implementation System aims to implement the audio application on Sitara DSP while the Development System aims to make the development environment more user-friendly.

4.1 Requirements and wishes

After analyzing the problem of this project, a list of requirements and wishes is formed.

The requirements for the Audio Implementation System are:

- The system must capture audio samples of 48kHz and process 24 bits, 2 channels of audio data.
- The audio processing algorithm must be generated from the audio application using MATLAB Simulink Audio System Toolbox.

- The audio processing algorithm must be running on AM5728 Sitara DSP.
- The system must be based on the Linux operating system.
- The system must realize the function of a high pass filter.

The requirements for the Development system are:

- The system must be able to cross-compile on the Linux host machine for the BeagleBoard-X15.
- The system must be able to debug the cross-compiled program on Linux host machine.

And the wishes for this project are:

- Import all the audio applications built by the fellow student in the Digital Audio Signal Processing Model.
- Transfer audio data stream between buffer and DSP without using CPU.

4.2 Audio Implementation System

The objective of this project is to implement the audio process algorithm from the audio application built in Simulink on Sitara DSP, the Audio Implementation System is the program that realizes this task.

In this part, the system description, overview, integration, test plan for the Audio Implementation System will be explained.

4.2.1 System overview

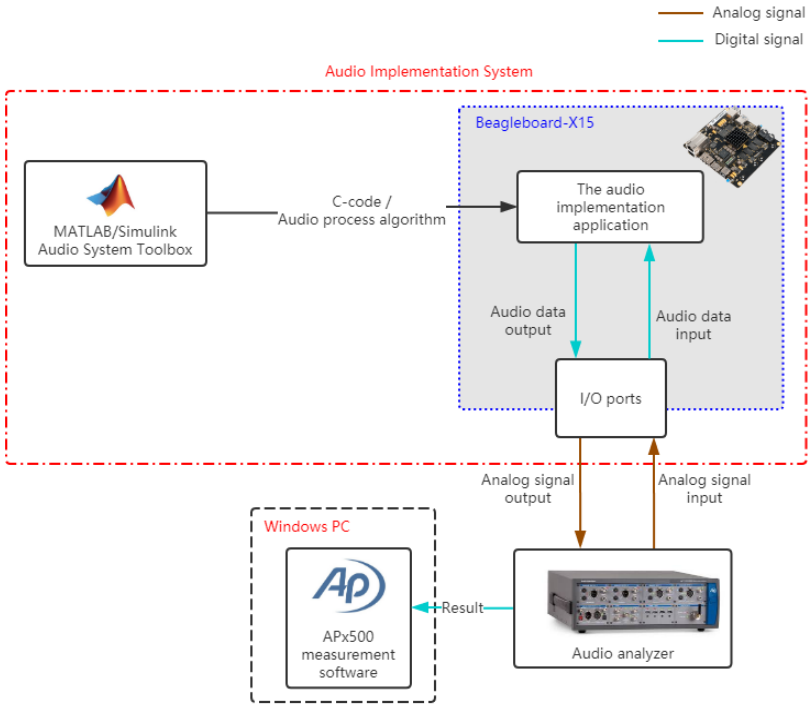


Figure 18 Audio Implementation System overview

The system overview in *Figure 18* shows the workflow in Audio Implementation System. First, the audio process algorithms in the form of C-code is generated from an audio application built in Simulink. Then the algorithms would be placed into the program of audio implementation to run on the Sitara DSP. After that, the audio implementation application would be built from that program. When running the application, the application would capture the analog audio input from the audio analyzer, convert the analog signal to digital, then process the audio data using the algorithm implemented on DSP. After the audio data have been processed, the audio data would be converted back to analog signals, then output the signal back to the audio analyzer. To verify the result of the processed audio, the status of the initial audio and the processed audio can be seen on the measurement software for the audio analyzer.

4.2.2 System description

The system description for Audio Implementation System describing all the inputs and outputs in the system can be found in *Appendix II System description*. This part will mainly describe the decompose diagram for Audio Implementation including the relation with the subsystems.

From the system overview in *Figure 18*, it can be concluded that the most crucial part of the system is the audio implementation application. Therefore, the subsystems for Audio Implementation System all revolve around the audio implementation application. The subsystems are audio process system, keyboard capture system, Simulink code system and DSP invoke & execute system.

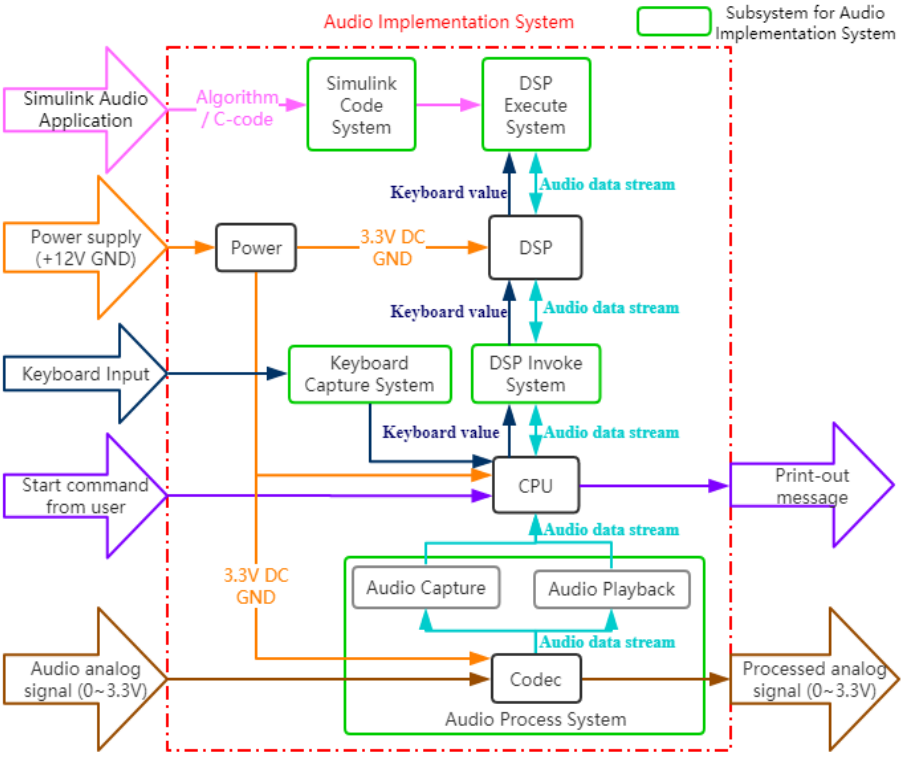


Figure 19 Decomposition diagram of Audio Implementation System

The decomposition diagram in *Figure 19* shows the detailed structure inside the Audio Implementation System, including the connection between the system and subsystems. When the user instructs the audio implementation system to start, the analog audio signals would be inputted to the Codec, the Codec would

convert the analog signal to digital, transfer the digital audio data stream to CPU. Then the CPU would invoke the DSP to process the audio data. Several tasks have already been implemented from the Simulink audio applications in the DSP execute system, and the process of implementing the audio application in the DSP execute system is done by the Simulink code system. Then the DSP would execute the task selected by the user through keyboard capture system to process audio data. After executing the task in DSP, the processed audio data would be sent back to CPU and be converted back to analog signals in Codec, and finally output the processed analog audio signals. In the process of running the application, the related messages will be printed out on the screen.

4.2.3 System integration

The Audio Implementation System is built by integrating all the subsystems. And before integrating the subsystems into the Audio Implementation System, all the subsystems will be tested and verified according to the test plans. Additionally, all the subsystems are built based on the audio process system.

4.2.4 System test plan

A system test plan for the Audio Implementation System has been set up and can be found in *Appendix I Test plans*. The test plan aims to test and verify if the system can realize all the functions and meet all the requirements, and would be executed when the design and verification for all the subsystems are complemented.

4.3 Subsystems for the Audio Implementation System

The subsystems for the Audio Implementation System are audio process system, keyboard capture system, Simulink code system and DSP invoke & execute system. And in this part, the design and component for each subsystem will be described, the integration of the subsystem, which is the source code in this case, will be explained. And to verify the subsystems, the test results of the subsystems following the test plans will be presented. The details of all the test plans are in *Appendix I Test plans*.

4.3.1 Audio Process System

The audio process system is the system developed by the former student in the previous project and is the base of the Audio Implementation System. Additionally, some modifications were made to the original audio process system so that it can be better adapted to the need for this project.

4.3.1.1 Design

Figure 20 shows the design of the audio process system from the previous project. The audio process system has realized the function of capturing audio signal, convert the analog signals to a digital audio data stream, invoke the DSP with CPU and transfer the data stream to DSP. In DSP, it can realize the function of

changing volume. Then the processed audio data will be transferred back to CPU, and output the processed audio after the data stream has been converted back to analog signals.

When comparing the design of audio process system in *Figure 20* to the design of audio implementation system in *Figure 19*, the audio capture and playback system remains unchanged, the keyboard capture system and DSP invoke system will be modified, and the main changes are in the program that runs on the DSP.

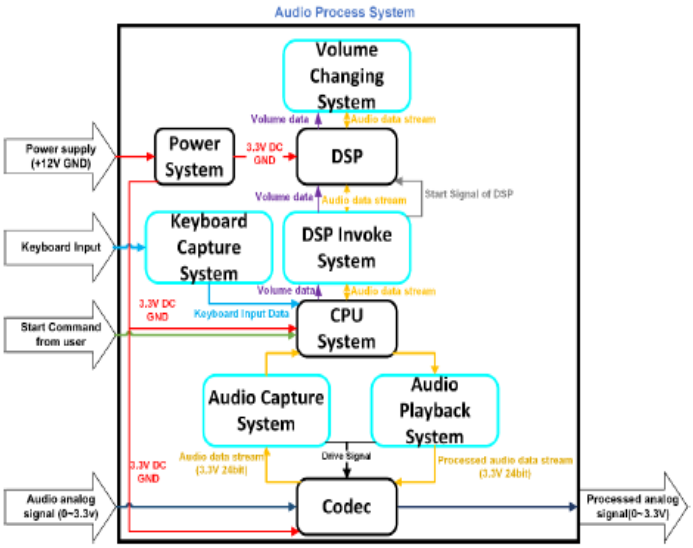


Figure 20 The design of Audio Process System (He, 2018)

4.3.1.2 Components

Figure 21 shows the workflow in the program of the modified audio process system, the parts in red grids are modifications made to the original audio process system.

In *Figure 21*, the program starts with allocating memory spaces to store audio data, then initialize the audio device with the audio configuration from the requirements, then initialize OpenCL and kernel. After the initialization is done, the system will capture keyboard input, audio input and convert the captured audio signals to a digital data stream.

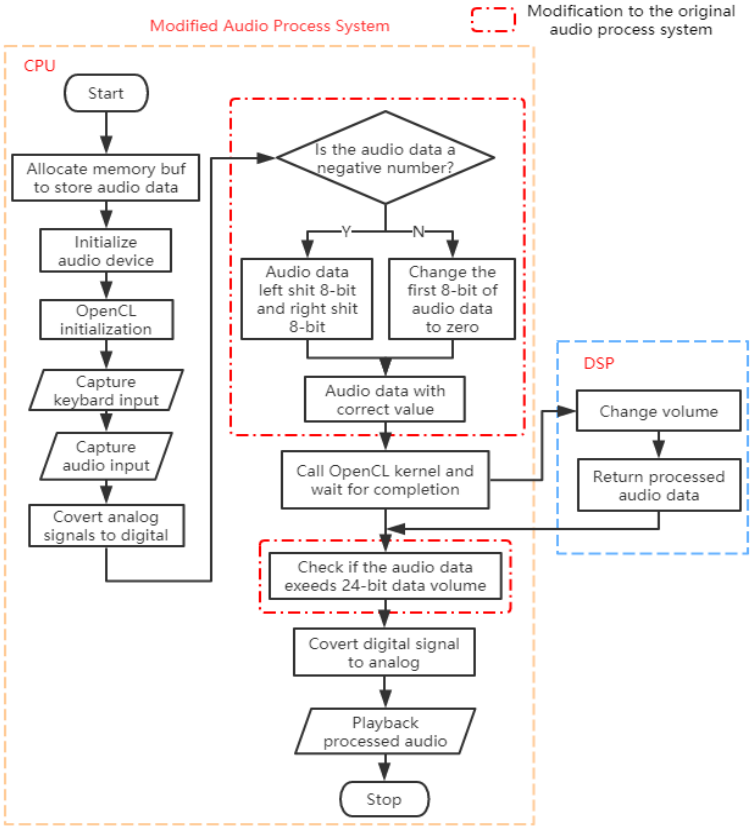


Figure 21 The flowchart of modified Audio Process System

Then is to modify the audio data so that it can represent the real audio data value, this step was originally put into the DSP kernel program, but it is now running on the CPU so that the steps for implementing C-code from Simulink audio application on the DSP can be simplified.

After the audio data have been modified, the audio data will be sent to DSP via OpenCL kernel. In the DSP, it can execute the volume changing function, then return the processed audio data back to CPU. Before

outputting the processed audio data, a step has been added to check if the audio data exceeds the 24-bit data volume. Due to the algorithm from Simulink can be complicated, this step is to make sure the processed data is still within the 24-bit data limit after running the algorithm. Otherwise, if the data value exceeds the limit, the outputted result would be incorrect. And after checking the value of the processed data, the data would be converted back to analog signal and playback to audio devices, such as a loudspeaker.

4.3.1.3 Subsystem integration (source code)

In this part, only the modified source code for the audio process system will be presented. *Figure 22* shows the source code to modify the 32-bit data to represent the 24-bit captured audio data and *Figure 23* shows the source code to check if any processed audio data value exceeds the limit of a signed 24-bit data volume.

The audio data is a 24-bit signed integer, with the signed bit on the 24th bit. However, since CPU only supports data type of 16-bit or 32-bit, the audio data is stored in the 32-bit allocated memory. Therefore, the first 8 bits of the data are random numbers. To remove the random numbers so that the data can represent the real value of the 24-bit audio data, the modification for audio data is needed.

```
for(i=0; i<960; i++)
{
    if(buf[i] & 0x00800000)
    {
       buf[i] = (buf[i] <<8) >>8 ;
    }
    else
    {
       buf[i] = buf[i] & 0x00ffffff;
    }
}
```

Figure 22 Source code to modified audio data

The source code in *Figure 22*, means that if the audio data is a negative value, then it will first be left shifted 8 bits so that the CPU can recognize it as a negative number, then right shift 8 bits, so that it represents the real value of the audio data. And when the audio data is a positive number, simply make the first 8 bits data value as 0, then it will represent the real value of the positive audio data.

As for the reason of placing this part on CPU instead of on DSP like it was originally done in the previous project, the variables sent from CPU to the DSP kernel program are read-only or write-only, thus the audio data would need to be modified in every audio application implemented on the DSP, it would not only complicate the workflow of implementing the code from Simulink audio application but also significantly increase the size of the program running on the DSP and eventually result in longer execution time.

```
for(i=0; i<960; i++)
{
    if(buf[i] > 8388607 || buf[i] < -8388608)
    {
       printf("data overflow\n");
       printf("output buffer[%d]= %d =  ",i, buf[i]);
       binary_print(buf[i]);
       printf("\n");
    }
}
```

Figure 23 Source code to check the value of processed data

And since the limit for a signed 24-bit data is from -8388608 to 8388607, if the processed data exceeds the limits, according to the source code in *Figure 23*, it will print out the value in both decimal and binary form, so that the value of the overflow data can be easily seen and can help the user to identify the problem.

4.3.1.4 Subsystem verification results

Figure 24 shows the result of the modified audio data function in the audio process system, the Input buffer [2] to [3] shows the modification for a negative audio data value, and Input buffer [4] shows the modification for a positive value. The first number in the figure shows the decimal form of the audio data value while the second number represents the binary form of audio data value.

```
Original Input buffer[2] = 32725501 = 0000 0001 1111 0011 0101 1001 1111 1101
Verified input buffer[2] = -828931 = 1111 1111 1111 0011 0101 1001 1111 1101

Original Input buffer[3] = 32656405 = 0000 0001 1111 0010 0100 1100 0001 0101
Verified input buffer[3] = -898027 = 1111 1111 1111 0010 0100 1100 0001 0101

Original Input buffer[4] = 16865317 = 0000 0001 0000 0001 0101 1000 0010 0101
Verified input buffer[4] = 88101 = 0000 0000 0000 0001 0101 1000 0010 0101
```

Figure 24 Result of modified audio data

As for the verification of volume changing function in the audio process system, the system was able to increase volume with pressing “w” on the keyboard and decrease volume with “s”. The result of verifying the audio process system is as shown in *Table 3*.

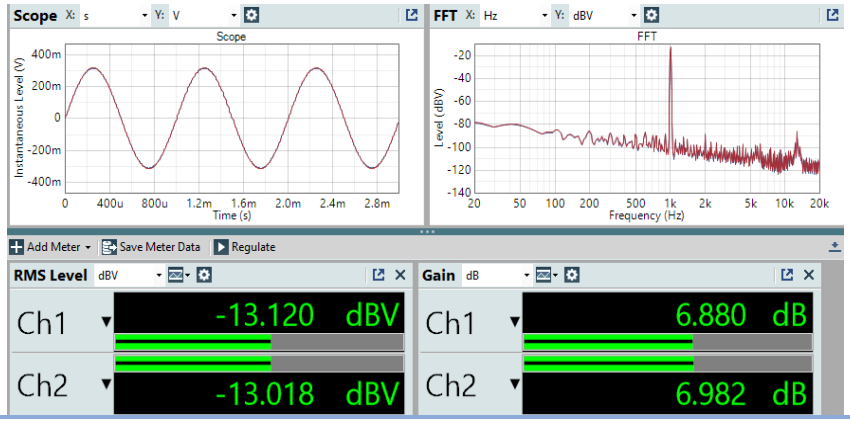
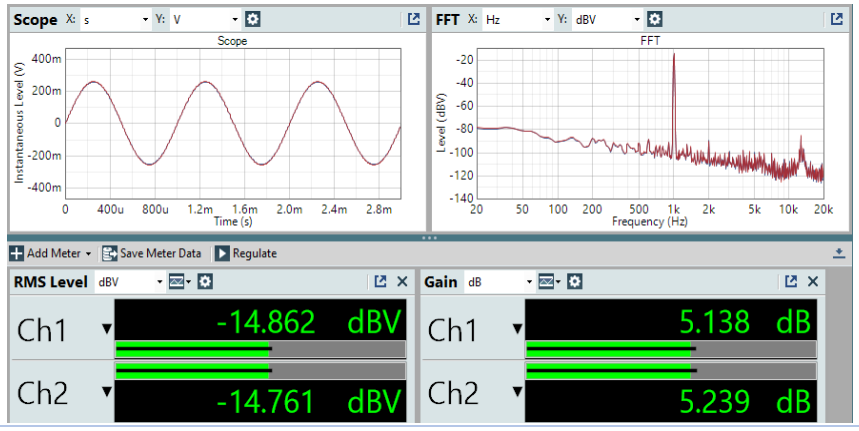
Increase volume with pressing “w” once	
Decrease volume with pressing “s” twice	

Table 3 Result for audio process system

Result analysis: As shown in *Figure 24*, the last 24-bit binary value remains unchanged after modification, the last 24-bit is the captured audio data. After modification, the first 8-bit of data has all become 1 for negative value and has been changed to 0 for a positive number, and the decimal value represents the real

value of the 24-bit data after modification. And after executing the application, there is no overflow data printed, the audio data are all in the range of 24-bit data volume, the output audio data value is correct.

For the result in *Table 3*, the original gain of the system is 6.057 dB, after increasing the volume with pressing "w" once, the gain has been increased to 6.880 dB, and after pressing the "s" twice to decrease volume, the gain has been decreased to 5.138 dB. The gain changes 0.8 dB every time "w" or "s" is pressed. Therefore, the audio processing system has realized the function of volume changing, the rebuilt of the audio processing system is successful.

4.3.2 Keyboard capture system

The keyboard capture system is a system that captures the keyboard input from the user and executes the corresponding function.

4.3.2.1 Design

As shown in *Figure 25*, the keyboard capture system captures keyboard input from the user, changing the value of the variables in the get char function. In this project, the keyboard capture system also acts as a switch to start or stop the execution of functions on DSP, which the corresponding functions will be further explained in *Ch 4.3.4*.

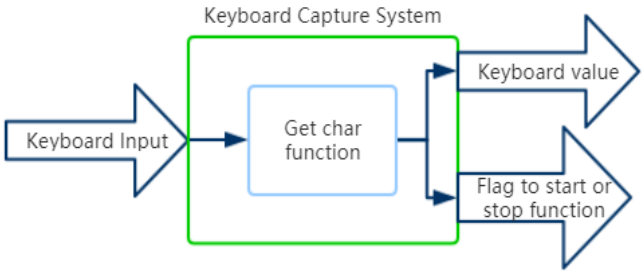


Figure 25 Decomposition diagram for Keyboard Capture System

4.3.2.2 Components

Figure 26 shows the design for the keyboard capture system, first is to initialize all the variables and flags as zero, then read keyboard input and set the corresponding flag in the get char function. After the flag is set, the function related to the flag will be executed in the DSP.

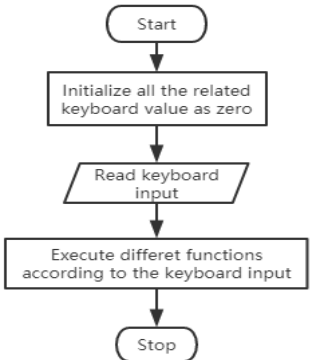


Figure 26 Flow chart for Keyboard Capture System

4.3.2.3 Subsystem integration (source code)

To realize the usage of a switch, flags are set in the get char function. When the value of a flag is 1, the flag is set to true, and the corresponding function in DSP will be executed. And when the flag is 0, the related function will be stopped.

The flags in this system, are "g", "h", "l" and "r", *Figure 27* shows the case of flag "g". Key "g" is the flag to start or stop gain minus 5dB function in the DSP, "h" is the flag for high-pass filter, "l" is for low-pass filter and "r" is for invert signal function.

```
ch=get_char();
if(ch)
{
    switch (ch)
    {
        case 'g':
            keyg++;
            printf("key 'g' pressed.\n");
            if(keyg%2 == 1) {
                g=1;
                printf("Gain -5dB set true.\n\n");
            }
            else {
                g=0;
                printf("Gain -5dB set false.\n\n");
            }
            break;
```

Figure 27 Source code for Keyboard Capture System

4.3.2.4 Subsystem verification results

The result for the keyboard capture function is as shown in *Figure 28*, "g" has been pressed twice to start and stop the gain minus 5dB function.

```
key 'g' pressed.
Gain -5dB set true.

key 'g' pressed.
Gain -5dB set false.
```

Figure 28 Verification results for Keyboard Capture System

Result analysis: When pressing the key, the function can recognize the correct key that has been pressed, and the corresponding flag for the key has been set true or false depending on the times the key was pressed. Therefore, the keyboard capture system has been successfully designed and verified.

4.3.3 Simulink Code system

The Simulink code system is about the workflow in the code generated from MATLAB Simulink audio application and how to implement the code on the Sitara DSP. In this system, several audio applications would be implemented, but this part would only introduce the workflow of implementing a high-pass filter audio application on the Sitara DSP as an example, the details of implementing the other audio applications can be found in *Appendix III Detail verification result for Simulink Code System*. In this chapter, the C-code generated from the high-pass filter audio application will be analyzed, the configurations to implement the audio application on the Sitara DSP will be introduced.

4.3.3.1 Design

As shown in *Figure 29*, the inputs of the Simulink code system are the Simulink audio application, a high-pass filter audio application in this case, and the captured audio data stream. The algorithm from the audio application would be implemented as the high-pass filter function on Sitara DSP. After executing the high-pass filter function, the processed audio data stream would be outputted.

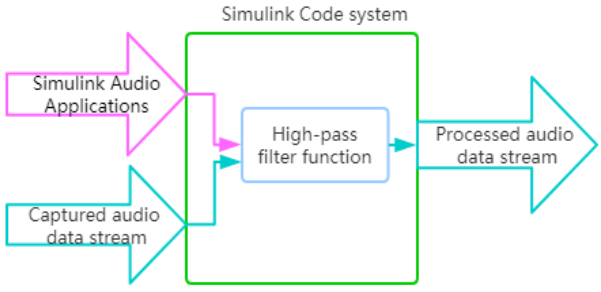


Figure 29 Decomposition diagram for Simulink Code System

4.3.3.2 Components

As shown in *Figure 30*, is the high-pass filter audio application built in Simulink, and the workflow of the generated code from this audio application is as shown in *Figure 31(left)*.

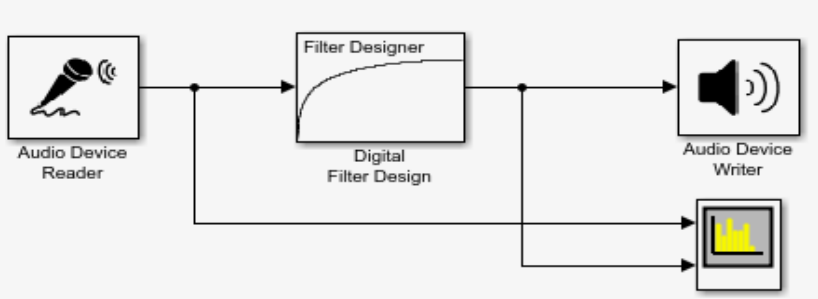


Figure 30 High-pass filter audio application in Simulink

In the C-code generated from the audio application, first is to initialize the audio devices, which is the audio device reader (microphone) and audio device writer (speaker/headphone) in *Figure 30*. Then is to capture audio data using the audio device reader and process the audio data with a high-pass filter, and finally playback the processed data in the audio device writer.

When comparing the flowchart of the code generated from Simulink in *Figure 31(left)* to the flowchart of the modified audio process system in *Figure 21*, the initialization of the audio device, capture audio data and playback processed audio data have already been programmed in the audio process system. Therefore, the most important part in the generated code that needs to be implemented on the Sitara DSP is the processing algorithm from the audio application, which is in the model step function of the generated code.

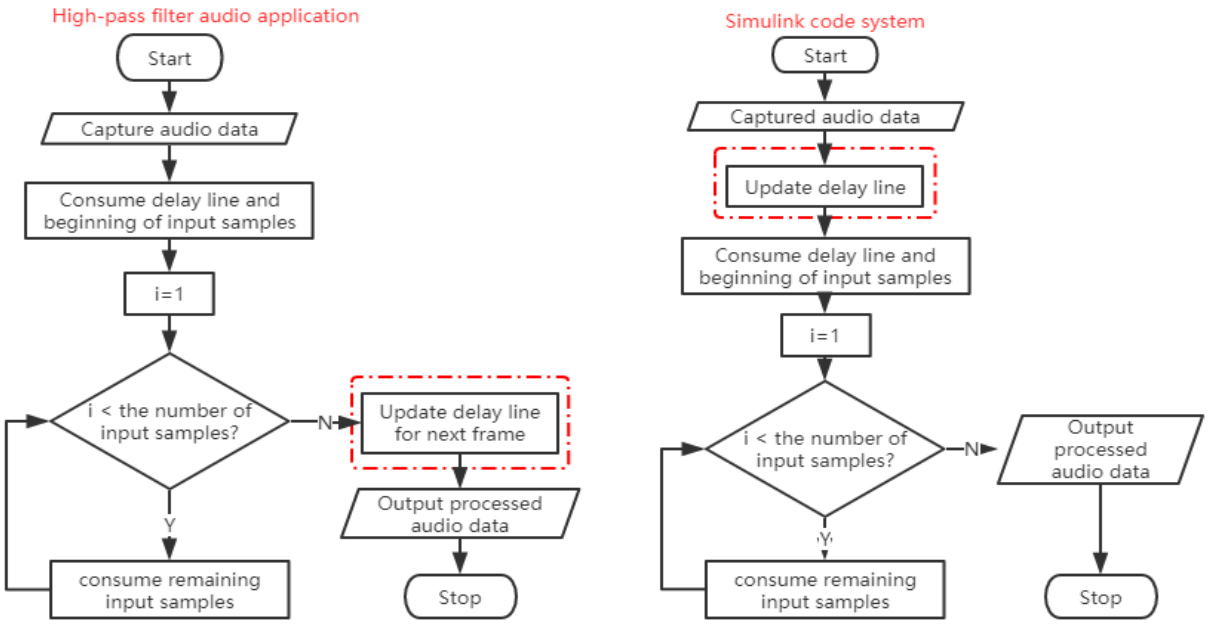


Figure 31 Flow chart for Simulink code system (Left: Workflow of code from Simulink; Right: Workflow of code running on Sitara DSP)

The left flow chart in *Figure 31* shows the workflow of the model step function in the generated code for the high-pass filter in *Figure 30*. After capturing audio data, the algorithm to process audio data samples contains two different steps, the first step is to process the first audio input sample, with a delay line "DigitalFilter_states", then the second step processes the remaining samples. After all the samples from the audio input are processed, the delay line would be updated to process the next frame of samples.

However, when directly implementing the generated code on the Sitara DSP, it would result in distortions in the processed audio. After several attempts to reduce the distortion, it is found that the distortions can be reduced when moving the delay line for the next frame to the initialization of the delay line for that frame of samples. In this case, the flow chart of the modified high-pass filter algorithm is as shown in *Figure 31(right)*, the change has been marked out in a red grid and the comparison of the results for the original algorithm and the modified algorithm can be seen in *Table 5*, *Ch 4.3.3.4 Subsystem verification results*.

4.3.3.3 Subsystem integration (source code)

Table 4 shows the C-code for the high-pass filter in Simulink and the code implemented on the Sitara DSP. The code from Simulink consists of three parts, capture audio data, audio processing algorithm and playback processed audio data. As analyzed in the component part, the most import part to implement is the audio processing algorithm and it has been implemented on the Sitara DSP. As for the modification of the generated code, the modification has been marked out with gridlines.

C-code generated from Simulink	C-code implemented on DSP

sErr = GetErrorBuffer(&rtDW.FromMultimediaFile_HostLib[0U]);
audio = (void *)&rtDW.FromMultimediaFile[0U];
LibOutputs_FromMMFile(&rtDW.FromMultimediaFile_HostLib[0U], GetNullPointer(),
audio, GetNullPointer(), GetNullPointer(),
GetNullPointer());
if (*sErr != 0) {
rtmSetErrorStatus(rtM, sErr);
rtmSetStopRequested(rtM, 1);
}
Capture audio data	

acc2 = rtDW.FromMultimediaFile[0] * rtConstP.DigitalFilter_Coefficients[0];
rtDW.DigitalFilter[0] = acc2 + rtDW.DigitalFilter_states *
rtConstP.DigitalFilter_Coefficients[1];
 	
for (i = 1; i < 1024; i++) {
acc1 = rtDW.FromMultimediaFile[i] * -0.5;
acc2 = rtDW.FromMultimediaFile[i - 1] * 0.5;
acc1 += acc2;
rtDW.DigitalFilter[i] = acc1;
}
Audio processing algorithm
rtDW.DigitalFilter_states = rtDW.FromMultimediaFile[1023];
	DigitalFilter_states = AudioDeviceReader[959];
	acc2 = (int)((float)AudioDeviceReader[0] * DigitalFilter_Coefficients[0]);
	DigitalFilter[0] = (int)((float)acc2 + DigitalFilter_states *
	DigitalFilter_Coefficients[1]);
	
	for (i = 1; i < 960; i++)
	{
	acc1 = (int)((float)AudioDeviceReader[i] * -0.5);
	acc2 = (int)((float)AudioDeviceReader[i - 1] * 0.5);
	acc1 += acc2;
	DigitalFilter[i] = acc1;
	}

sErr = GetErrorBuffer(&rtDW.AudioDeviceWriter_AudioDeviceLi[0U]);
LibUpdate_Audio(&rtDW.AudioDeviceWriter_AudioDeviceLi[0U],
&rtDW.DigitalFilter[0U], 0, 1024, 0U);
if (*sErr != 0) {
rtmSetErrorStatus(rtM, sErr);
rtmSetStopRequested(rtM, 1);
}
Playback processed audio data	

Table 4 Implement C-code from the high-pass filter audio application on the Sitara DSP

By comparing the generated code and the implemented code in *Table 4*, it can be seen that some modifications are required in order to implement the audio processing algorithm on the Sitara DSP. First is to delete the "rtDW." from the generated code, as DSP cannot recognize this type of definition. Then is to cast the calculated result from float to integer, this is due to the audio device can only playback integer value. Otherwise, playback errors would occur when running the program. Additionally, modification may be needed for the generated code to acquire the best performance on Sitara DSP.

4.3.3.4 Subsystem verification results

As shown in *Table 5*, is the FFT (Fast Fourier Transformation) graphs for the status of BeagleBoard-X15 in the initial state, the status of running high-pass filter with the original algorithm from the Simulink audio application and status of running high-pass with the modified processing algorithm.

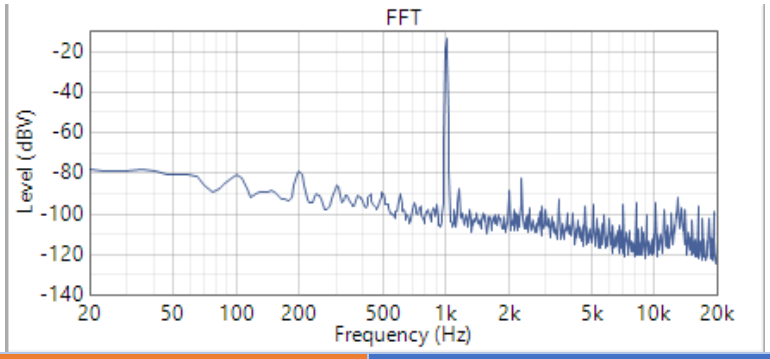
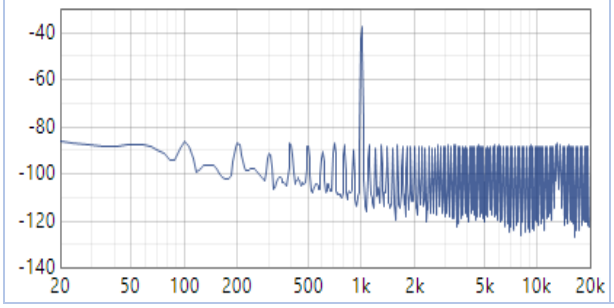
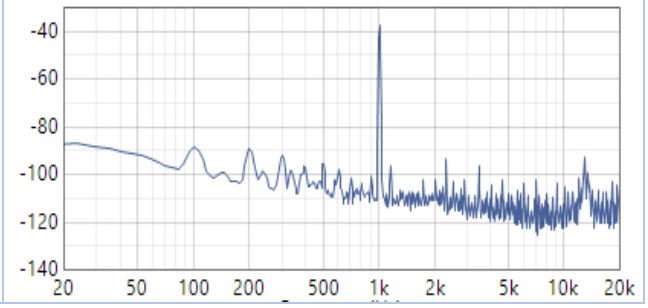
Initial status	
	
Original high-pass filter processing algorithm	**Modified high-pass filter processing algorithm**
	

Table 5 Result of the high-pass filter, original vs. modified algorithm

Result analysis for *Table 5*: By comparing the FFT graphs for the high-pass filter with the original algorithm and modified algorithm, both graphs show the effect of high-pass filtering at the 2kHz frequency. The level has all been decreased for frequencies before 2kHz, from -80 to -110 dBV in initial status to -90 to -115 dBV after filtering, and stays with -120 dBV for frequencies greater than 2kHz. However, the result for the original processing algorithm contains a lot of distortions, as shown in the graph, the peaks starting from 500 Hz has went up comparing to the initial status, while in the result for the modified algorithm, the distortion is, in general, at the same level as the initial status. To conclude, both the high-pass filters show expected results and the result for the high-pass filter with the modified algorithm is more stable and has less distortion.

After checking the result of running the high-pass filter on the BeagleBoard-X15, a comparison between the high-pass filter running on MATLAB Simulink and BeagleBoard is as shown in *Table 6*. In this comparison, the high-pass filters are the same, with a cut-off frequency at 2kHz, and the audio input and audio output are all using the APx 525 audio analyzer. The only difference is running the audio application in Simulink and running the high-pass filter function in BeagleBoard to process audio.

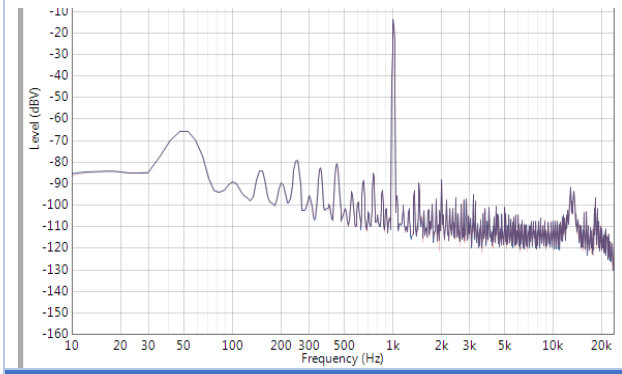
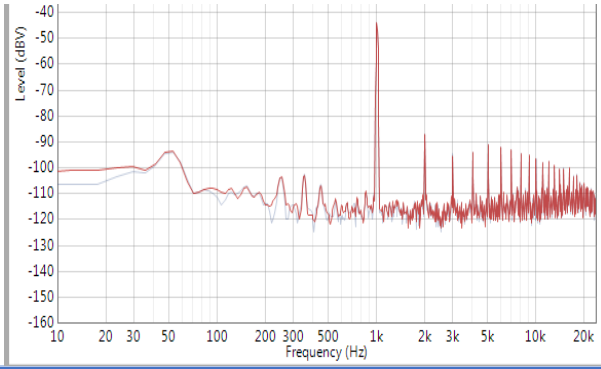
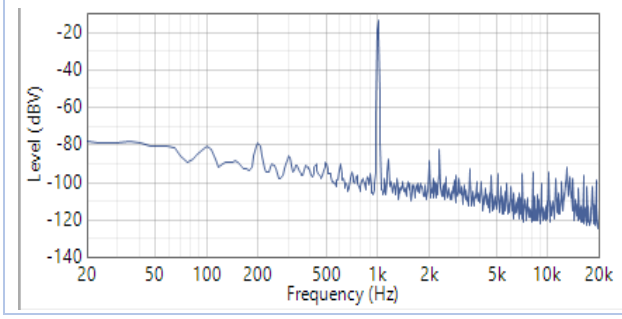
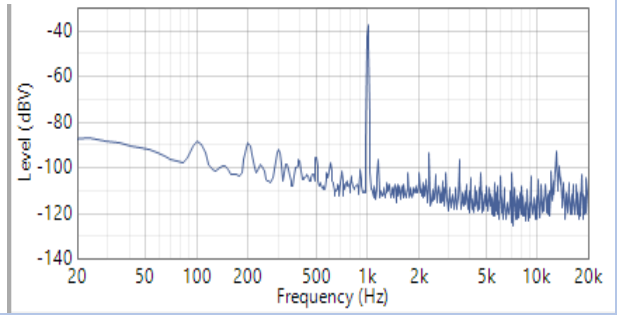
High-pass filter running on MATLAB Simulink	
Initial status	**Filtered status**
	
High-pass filter running on BeagleBoard-X15	
Initial status	**Filtered status**
	

Table 6 High-pass filter result - Simulink vs. BeagleBoard

Result analysis for *Table 6*: Both the high-pass filter running on Simulink and BeagleBoard has shown the result of filtering, the level for frequencies before 2kHz have been decreased, and the level all remained in the range of -110 dBV to -120 dBV for frequencies greater than 2kHz. For high-pass filter running on Simulink, the initial level for frequencies before 2kHz range from -65 dBV to -115 dBV and range from -95 dBV to -120 dBV after running high-pass filter; and for BeagleBoard-X15, the initial level ranges from -80 dBV to -105 dBV and range from -90 dBV to -115 dBV after running the high-pass filter on Sitara DSP.

From the comparison between high-pass filter running on Simulink and BeagleBoard, it can be concluded that both the high-pass filter showed similar results, the implementation of the high-pass filter on Sitara DSP is showing the expected result.

From the comparison in *Table 5* and *6*, the high-pass filter has shown expected result according to the FFT graphs. And the following verification would check if the result from running the high-pass function matches the theoretical result. *Figure 32* shows the result of running the high-pass filter function on Sitara DSP, the input buffer is the input audio data, the output buffer is the output audio data, and the intermediate value in the function are DigitalFilter_states, acc1, and acc2. The verification between the printed value in *Figure 32* and the theoretical value is as shown in *Table 7*, the theoretical value is calculated according to the modified algorithm as shown in *Table 4*. Due to the function contains many variables and the binary value is equal to the decimal value, the result verification only compares the decimal value.

```
[core 0] High-pass FIR filter start
Input buffer[0] = 176701 = 0000 0000 0000 0010 1011 0010 0011 1101

DigitalFilter_states = 101269 = 0000 0000 0000 0001 1000 1011 1001 0101

acc2[0]= -88350 = 1111 1111 1111 1110 1010 0110 1110 0010

Output buffer[0] = -37715 = 1111 1111 1111 1111 0110 1100 1010 1101

Input buffer[1] = 104699 = 0000 0000 0000 0001 1001 1000 1111 1011

acc1[1]= -52349 = 1111 1111 1111 1111 0011 0011 1000 0011

acc2[1]= 88350 = 0000 0000 0000 0001 0101 1001 0001 1110

Output buffer[1] = 36001 = 0000 0000 0000 0000 1000 1100 1010 0001

Input buffer[2] = 162775 = 0000 0000 0000 0010 0111 1011 1101 0111

acc1[2]= -81387 = 1111 1111 1111 1110 1100 0010 0001 0101

acc2[2]= 52349 = 0000 0000 0000 0000 1100 1100 0111 1101

Output buffer[2] = -29038 = 1111 1111 1111 1111 1000 1110 1001 0010
```

Figure 32 Result value of high-pass filter function

	Printed value	Theoretical value	Result matches?
Input Buffer [0] = 176701; DigitalFilter_states = 101269			
acc2	-88350	-88350	✓
Output buffer	-37715	-37715	✓
Input Buffer [1] = 104699			
acc1	-52349	-52349	✓
acc2	88350	88350	✓
Output buffer	36001	36001	✓
Input Buffer [2] = 162775			
acc1	-81387	-81387	✓
acc2	52349	52349	✓
Output buffer	-29038	-29038	✓

Table 7 Result verification for the high-pass filter function

Result analysis: The verification results from *Table 7* showed that all the intermediate value and output audio data value is correct, and with the data overflow checking as described in *Ch 4.3.1.3* *Figure 23*, all the data value is within the signed 24-bit data volume. Therefore, the high-pass filter function contains no error in the calculation.

From the result of the comparison between the original algorithm from Simulink audio application and the modified algorithm, to the comparison between the filter running on Simulink and running on BeagleBoard-X15 Sitara DSP, to the result verification for high-pass filter function running on the DSP, all the results are as expected, and no error has occurred. Therefore, the Simulink code system has been successfully designed and verified. Additionally, from the comparison between the original algorithm from Simulink and the modified algorithm in *Table 5*, it can be concluded that some algorithm may need to be modified for the target board to obtain the most perfect result.

4.3.4 DSP invoke & execute system

The DSP invoke system is a system that runs in the CPU and invokes the DSP, transfers audio data stream and flags from keyboard input to the DSP. The DSP execute system is a system that runs in the DSP kernel program, combining all the functions implemented from the audio applications and controlling how the DSP processes audio data with the functions. This part uses gain -5dB and signal invert functions as an example, more details for executing other functions can be found in *Appendix IV*.

4.3.4.1 Design

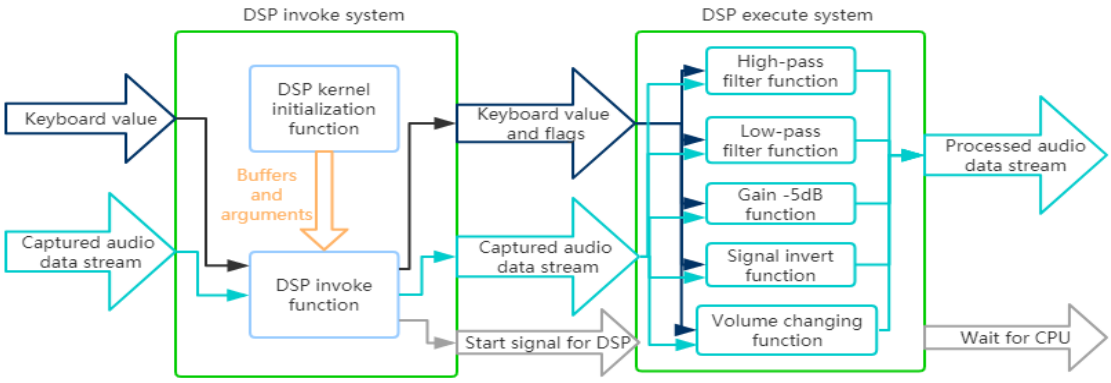


Figure 33 Decomposing diagram for DSP invoke & execute system

The design of the DSP invoke & execute system is as shown in *Figure 33*, the DSP invoke system initializes the OpenCL kernel for DSP, sends keyboard value and flags from keyboard input and the captured audio data stream to DSP, and sends a start signal to start DSP. The DSP execute system receives the data from the DSP invoke system, execute the function(s) that has been set true in the flags. The detail of the flags from keyboard input was explained in *Ch 4.2.2 Keyboard capture system*.

4.3.4.2 Components

Figure 34 and *35* show the flowcharts for DSP invoke system and DSP execute system respectively.

In DSP invoke system, first, the OpenCL configuration would be initialized. The OpenCL device would be set as DSP, the kernel program would be built, buffers would be created in the preparation of sending data to the DSP, and arguments for the kernel would be set up. After the initialization for the DSP kernel is done, the DSP invoke system would send the flags value, which is from the keyboard capture system, along with the captured audio data stream to the DSP for processing and wait for the DSP to complete the execution.

As the audio data, flags value is sent to the DSP and the DSP is invoked, the DSP execute system

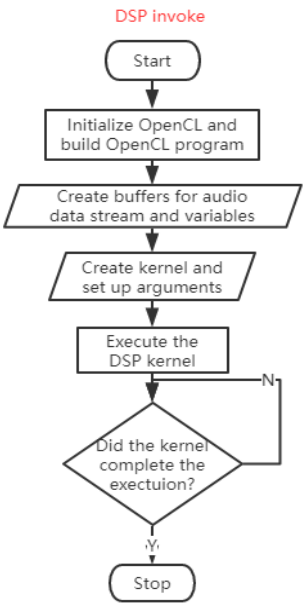


Figure 34 Flow chart for DSP invoke system

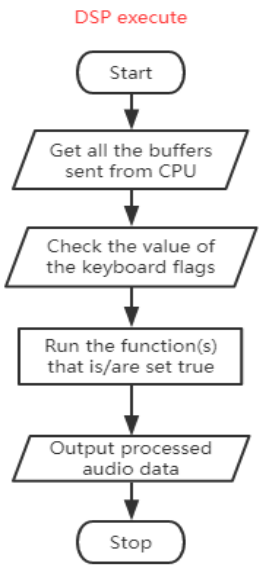


Figure 35 Flow chart for DSP execute system

would execute the functions that are set true according to the flags. If more than one functions are set to true, the final processed audio data would be the sum of all the functions that are set true. For example, if both the gain minus 5 dB function and signal invert function are set to true, the result of the processed data would be: the gain would reduce 5 dB and the signal would invert 180 deg. And after the DSP execute system has complete the execution, the processed data would be sent back to the CPU for playback.

4.3.4.3 Subsystem integration (source code)

As shown in *Figure 36*, is a part of the source code to initialize the OpenCL kernel and to set up buffers and arguments. For the buffers, the "AudioDeviceReader" is read-only, which will be the audio input for all the functions in DSP execute system, the "Gain" and "SignalInvert" are the names for the output audio data in different functions and are set as write-only, which means they can only be written to and cannot be used to assign to other variables.

And after creating the buffers, the arguments for the kernel are set in the same order as the created buffers so that the arguments can be sent to the DSP with the corresponding value.

```
Buffer cl_AudioDeviceReader(context, CL_MEM_READ_ONLY|CL_MEM_USE_HOST_PTR, buffer_size, buf);
Buffer cl_Gain(context, CL_MEM_WRITE_ONLY|CL_MEM_USE_HOST_PTR,  buffer_size, buf);
Buffer cl_SignalInvert(context, CL_MEM_WRITE_ONLY|CL_MEM_USE_HOST_PTR,  buffer_size, buf);

Kernel K (P, "ocl_matmpy");
K.setArg(0, cl_AudioDeviceReader);
K.setArg(1, cl_Gain);
K.setArg(2, cl_SignalInvert);
```

Figure 36 Source code for DSP invoke system

In the DSP, the source code for gain -5dB and signal invert function in the DSP execute system is as shown in *Figure 37*. In the DSP execute system, the flag "g" is the switch to start or stop gain minus 5 dB function, flag "r" is for signal invert function, "h" for high-pass filter and "l" for the low-pass filter. And if none of the flags are set to true, the system will execute the volume changing function by default.

```
const std::string kernelSrc =
R"(
    kernel void ocl_matmpy(const    global int *AudioDeviceReader,
                                    global int *Gain,
                                    global int *SignalInvert,
                                    global int *g,
                                    global int *r)
    {
        int i;
if(*g)
{
    for (i = 0; i < 960; i++)
    {
        Gain[i] = (int)((float)0.56234132519034907 * AudioDeviceReader[i]);
    }
}

if(*r)
{
    for (i = 0; i < 960; i++)
    {
        SignalInvert[i] = -AudioDeviceReader[i];
    }
}
```

Figure 37 Source code for DSP execute system

As for the structure of the DSP execute system, the first intention was to apply a switch structure to execute different audio processing functions. However, the DSP does not support the switch structure, thus the more

basic if-else structure is used to build up this system. And in this way, it can realize the need for running multiple functions simultaneously.

Therefore, in the DSP execute system, the gain minus 5 dB, signal invert, high-pass filter, and low-pass filter functions can run separately or simultaneously, and the processed data would be the sum of the executed function(s).

4.3.4.4 Subsystem verification results

Figure 38 shows the result of executing both gain minus 5 dB and signal invert functions, the initial gain and phase from *Figure 39* is 6.057 dB and +115.453 deg respectively. After executing both functions, the gain and phase have changed to 1.056 dB and -64.844 deg, which shows the result of both gain minus 5dB in *Figure 42* and signal inverting in *Figure 43*. Furthermore, the verification for the theoretical calculation can be found in *Appendix IV*.

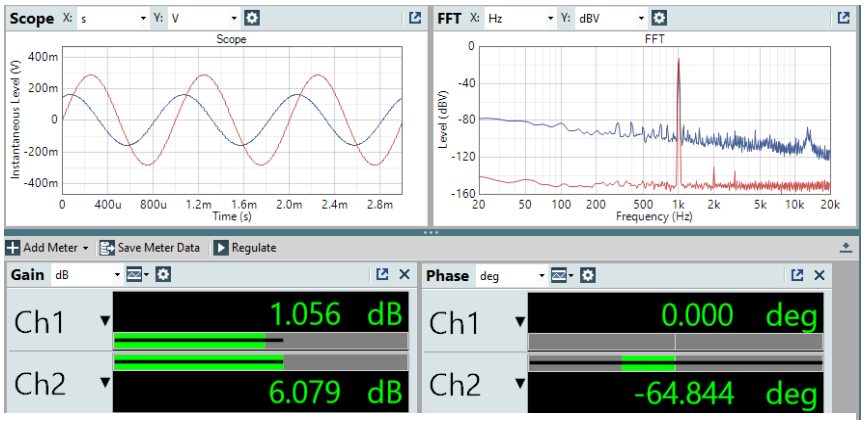


Figure 38 Result for gain -5dB & signal invert function

Result analysis: From the result and result verification for running the gain minus 5 dB & signal invert functions, the results are as expected, and the result value is correct, and no data exceeds the signed 24-bit data volume. Therefore, the DSP invoke & execute system has been successfully designed and verified.

4.4 System verification & validation result

The system verification results were acquired from executing the system test plan for Audio Implementation system, the device used in this test as audio input and audio output is the APx525 audio analyzer, and the results are shown in the APx500 measurement software.

Table 8 shows the verification result and result analysis for the Audio Implementation System. The blue line and channel 1 (Ch1) represent the status of BeagleBoard-X15, and the red line in channel 2 (Ch2) is the reference from the audio analyzer. Additionally, result data only shows the ones that are related to the specific functions.

The initial status

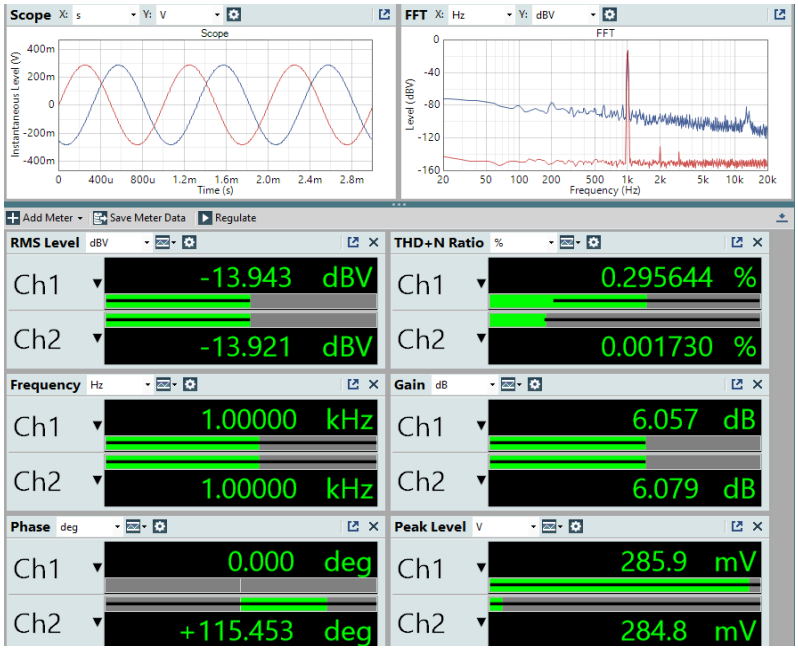


Figure 39 Initial status (the blue line is BeagleBoard-X15, the red line is a reference)

For the initial state of the system, the RMS level is -13.9 dBV, the THD +N value is 0.29%. the peak level is 285.9mV at 1k Hz, the gain is 6dB and the phase angle is +115deg.

High-pass filter (cut-off frequency at 2kHz)

Figure 41 shows the result of executing a FIR high-pass filter with a cut-off frequency at 2kHz, the blue line and Ch1 shows the result from BeagleBoard-X15.

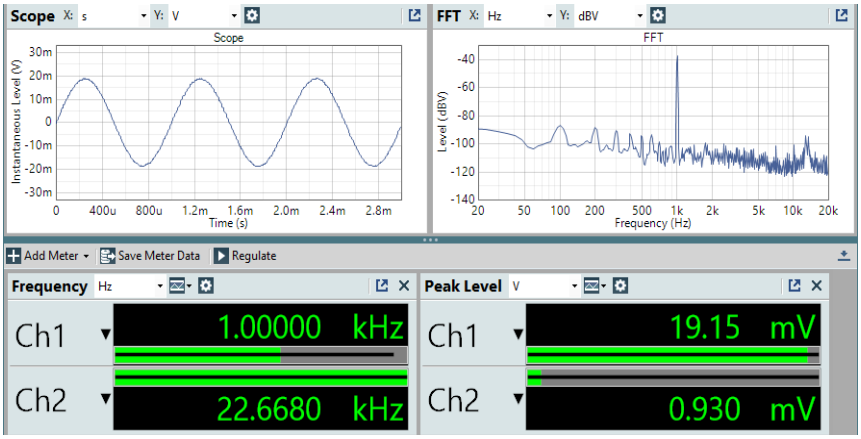


Figure 40 Result of a high-pass filter with cut-off frequency 2kHz

Result analysis: When running the high-pass filter on Sitara DSP, as shown on the FFT graph in *Figure 40*, the level for frequency from 20 Hz to 2kHz has decreased compared to the initial status in *Figure 39*.

The level at 20 Hz is -90 dBV while it is around -70 dBV in the initial status, the peak level value, as shown in the scope graph, has been decreased from 285mV to 19.31mV, the level at 1k Hz has decreased from -15 dBV to -35 dBV, the level between 1k Hz to 2kHz is around -110 dBV when the initial status

is around -100 dBV. And when the frequency is greater than 2kHz, the level value is in the same range as the initial status, between -110 dBV to -120 dBV.

By comparing the FFT graph for the high-pass filter and the initial status, it can be seen that the high-pass filter showed the expected result. The high-pass filter attenuated signals with a frequency below 2kHz and passed signals with a frequency greater than 2kHz.

Low-pass filter (cut-off frequency at 2kHz)

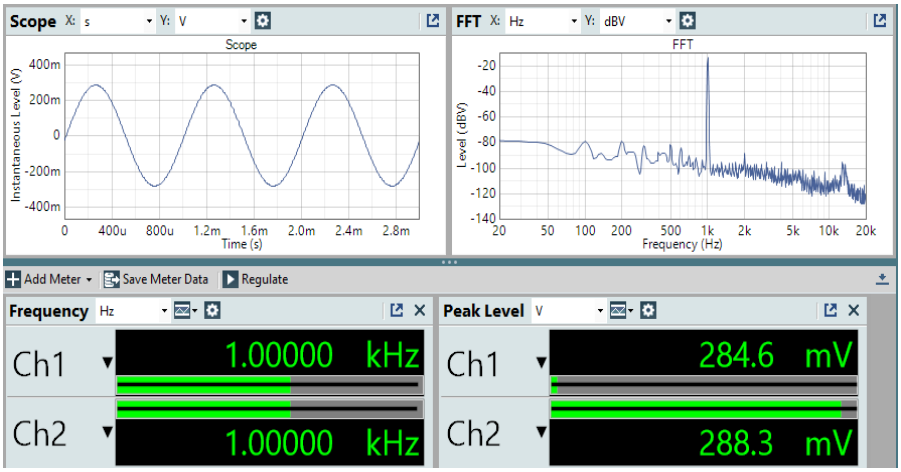


Figure 41 Result of a low-pass filter with cut-off frequency at 2kHz

Figure 41 shows the result of executing a FIR low-pass filter with a cut-off frequency at 2kHz, the blue line shows the result from BeagleBoard-X15.

Result analysis: When running the low-pass filter on Sitara DSP, as shown on the FFT graph in *Figure 41*, the level for frequencies from 2kHz to 20kHz has been decreased compared to the initial status in *Figure 39*. And for frequencies below 2kHz, the value of level stays in the same range as the initial status, between -75 dBV to -100 dBV, the peak level value at 1k Hz also remains the same.

The level at 20k Hz is -130 dBV while it is around -120 dBV in the initial status, the level between 2kHz to 20k Hz is around -115 dBV to -130 dBV when the initial status is around -110 dBV to -120 dBV.

By comparing the FFT graph for the low-pass filter and the initial status, the low-pass filter showed the expected result. The low-pass filter attenuated signals with a frequency greater than 2kHz and passed signals with a frequency below 2kHz.

Gain minus 5dB

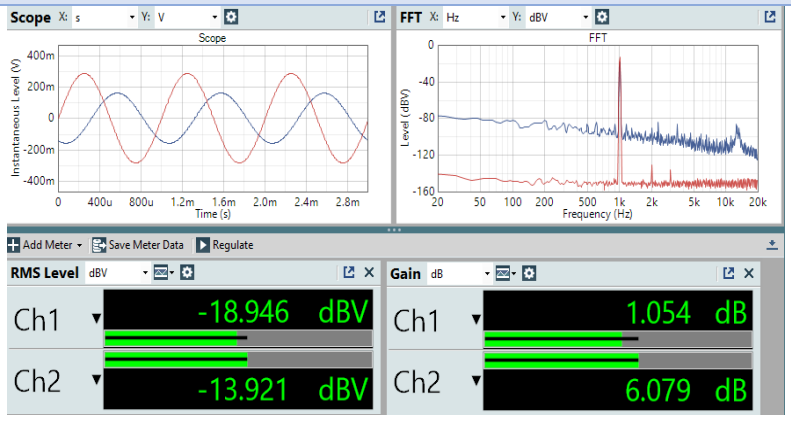


Figure 42 Result of gain minus 5 dB

Figure 42 shows the result of executing the function of decreasing the gain for 5 dB, the blue line shows the result from BeagleBoard-X15.

Result analysis: After executing the gain minus 5 dB function, the RMS level and gain for the signals should reduce 5dBV and 5dB. In *Figure 39*, the original RMS level and gain for the BeagleBoard is -13.943dBV and 6.057dB respectively, therefore, the theoretical results should be *RMS level = -13.943 - 5 = -18.943dBV*, and *Gain = 6.057 -5 = 1.057dB*. The result of executing the gain minus 5 dB function is as shown in *Figure 42*, the RMS level is -18.946dBV and the gain 1.054 dB, which are consistent with the theoretical results.

By comparing the initial status and result of the gain minus 5 dB function, the gain minus 5 dB function showed the expected result. The RMS level and gain have reduced 5 dBV and 5 dB respectively.

Signal invert

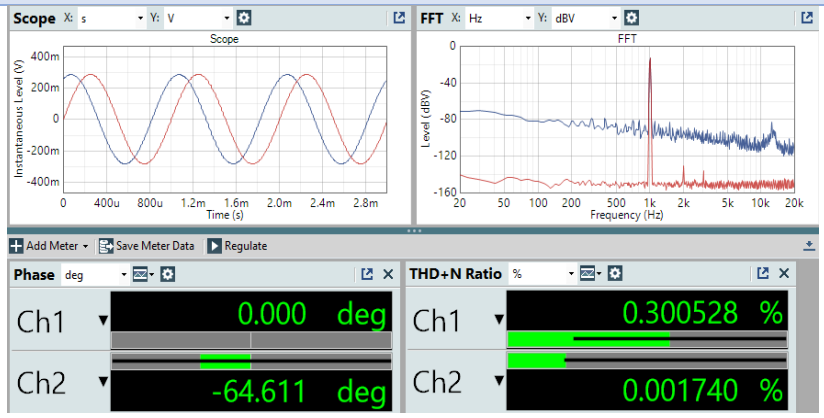


Figure 43 Result of signal invert

Figure 43 shows the result of executing a function of inverting the input signal, the blue line shows the result from BeagleBoard-X15.

Result analysis: After executing the signal invert function, the phase angle should be 180 deg different from the initial status. The initial phase angle as shown in *Figure 39* is +115.453 deg, therefore, the theoretical result should be *Phase = +115.453 – 180 = -64.547 deg*. The result of executing the signal invert function is as shown in *Figure 43*, the angle is -64.611 deg, which fits the theoretical result.

By comparing the initial status and result of the signal invert function, the signal invert function showed the expected result. The phase angle has changed 180 deg.

Table 8 Audio Implementation System verification & validation results and result analysis

From the results in *Table 8*, it can be seen that the audio applications in Simulink have been successfully implemented on the Sitara DSP, and by checking the requirements, the Audio Implementation has fulfilled all the requirements.

- The system must capture audio samples of 48kHz and process 24 bits, 2 channels of audio data.
- The audio processing algorithm must be generated from an audio application using MATLAB Simulink Audio System Toolbox.
- The audio processing algorithm must be running on AM5728 Sitara DSP.
- The system must be based on the Linux operating system.
- The system must realize the function of a high pass filter.

4.5 Development System

In this project, there are three operating systems involved. The PC is running Windows system, a virtual Linux system (Ubuntu), which is the Linux host machine in this project, is created on the same PC. The BeagleBoard-X15 is also running a Linux system, Debian. The Development System is focused on connecting the Linux host machine and the target board system together, realizing the functions of transferring files, cross-compiling and remote debugging.

In this part, the system overview, integration and test plan for the Development System will be explained, the system description for the Development System can be found in *Appendix II System description*.

4.5.1 System overview

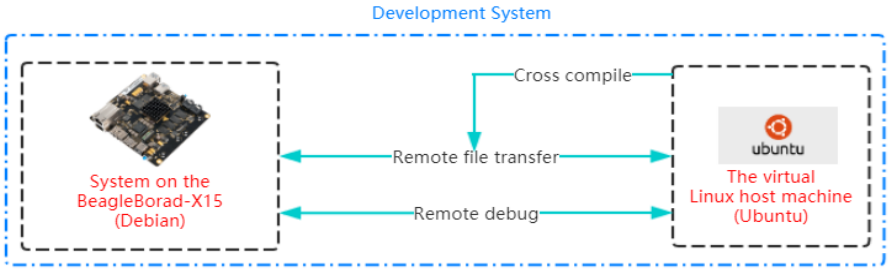


Figure 44 Development System overview

The system overview in *Figure 44* shows the connection between the two systems, which are system on the target board -- BeagleBoard-X15, and the Linux host machine. The functions that will be realized in the development system are remote file transferring between Linux host machine and the target board, cross compiling in Linux host machine for system on the target board, and remote debugging in Linux host machine for a program running on the target board.

To realize all the functions, the development system has been divided into three subsystems: remote file transfer system, cross-compile system and remote debug system.

4.5.2 System integration

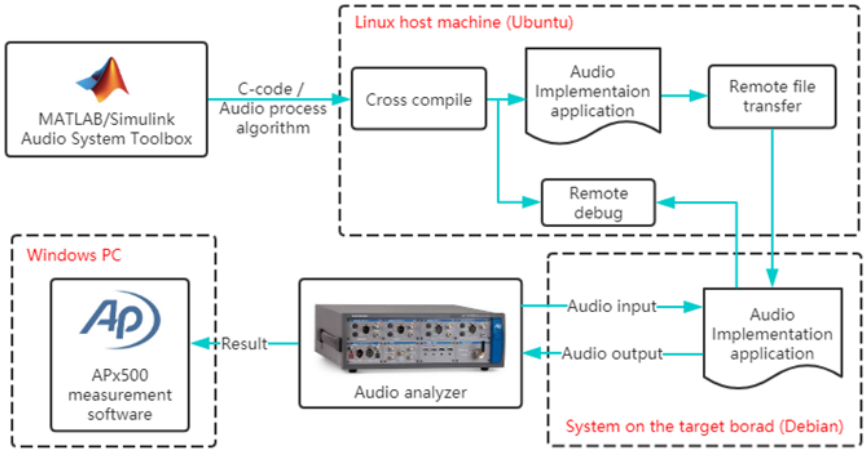


Figure 45 Combined workflow of development system & audio implementation system

The system integration for the Development system is the integration of all the subsystem for the development system. Moreover, the development system can also be integrated with the audio implementation system, creating the complete workflow of this project, as shown in *Figure 45*. The generated C-code from Simulink audio application would be cross-compiled in the Linux host machine, output a compiled audio implementation application, the application can be transferred to the target board system via remote file transfer. When running the application in the target board, the audio would be inputted from the audio analyzer and the processed audio would be outputted back to the audio analyzer and the results can be seen in the audio analyzer measurements software. Additionally, the status of the running program can also be viewed in the Linux host machine with the remote debugger.

4.5.3 System test plan

To verify if the development system has realized all the functions and met all the requirements, a system test plan for the development system has been set up and the test plan can be found in *Appendix I test plans*.

4.6 Subsystems for the Development System

As described in the Development System, the subsystems for Development System are remote file transfer system, cross-compile system and remote debug system. In this part, the design and verification results for all the subsystems will be described. The verification is done following the test plans for the subsystems. For all the subsystems, the components and integration can be found in *Appendix V* and all the detail subsystem test plans can be found in *Appendix I Test plans*.

4.6.1 Remote File Transfer System

The remote file transfer system is a system that transfers files between the Linux host machine and system on the target board, which is the BeagleBoard-X15 in this project.

4.6.1.1 Design

Figure 46 shows the design of the remote file transfer system, the input of the system is a file in the Linux host machine, after applying the SSH function, the file can be transferred to the target board. A file can also be transferred from the target board to the Linux host machine with the SSH function.

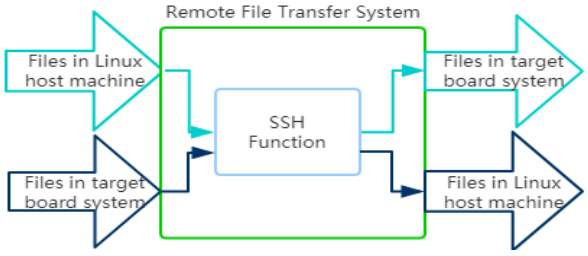


Figure 46 Decomposition diagram for Remote File Transfer System

4.6.1.2 Subsystem verification results

Figure 47 shows the verification result for the remote file transfer system. The file being transferred in this test is “File_transfer_test” from the Linux host machine ‘Local’. After file transfer, the “File_transfer_test” is also in the target board system ‘bb-x15’. And the verification result on the right in *Figure 47* shows the result on the target board, before file transfer, there is no “File_transfer_test” in the target board system, and the file exist after file transfer.

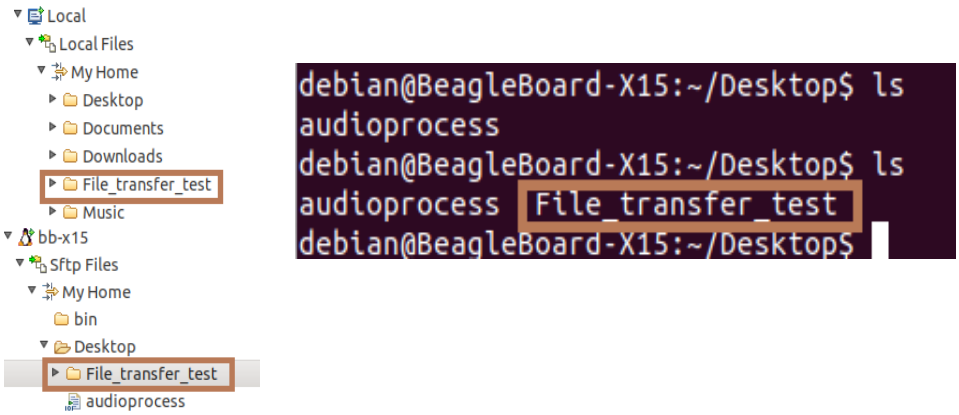


Figure 47 Verification result for Remote file transfer system

Result analysis: The file transfer connection between Linux host machine and the target board has been set up using the SSH function, the remote file transfer system can transfer files between Linux host machine and target board. Therefore, the remote file transfer system has been successfully designed and verified.

4.6.2 Cross-compile System

The cross-compile system is a system that uses a compiler for the target board to compile in the Linux host machine, and the compiled application can run on the target board but not on the Linux host machine.

4.6.2.1 Design

The design for the cross-compile system is as shown in *Figure 48*, with a program written in the Linux host machine and the related libraries as input, then the program is compiled using a cross-compiler for the target board, and an executable application for the target board would be outputted.

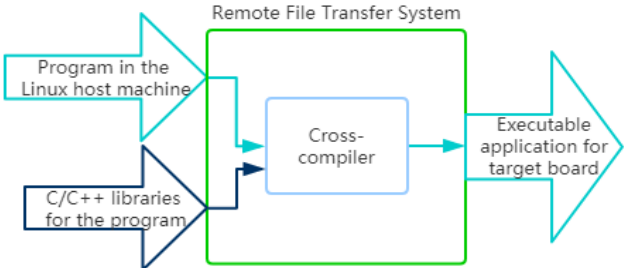


Figure 48 Decomposition diagram for Cross-compile System

4.6.2.2 Subsystem verification results

The program used in this test is the audio-implement program developed in the Audio Implementation System, the executable application is named “audio-implement”. The program is compiled in the Linux host machine and the result of running the “audio-implement” application in the target board and in Linux host machine is as shown in *Figure 49* and *50* respectively.



Figure 49 Cross-compiled application running on target board

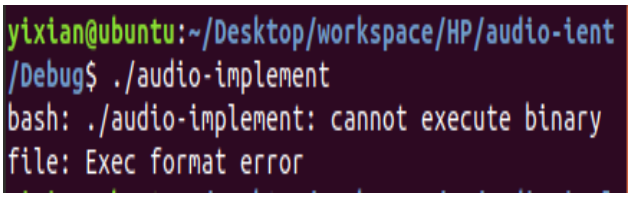


Figure 50 Cross-compiled application running on Linux host machine

Result analysis: The cross-compiled application, which is compiled in the Linux host machine, is able to run on the target board without any error message. And the application cannot run on the Linux host machine, showing an error message when tried to execute the application. Therefore, the cross-compile system has been successfully designed and verified.

4.6.3 Remote Debug System

The remote debug system is a system that debugs an application running on the target board in the Linux host machine, it can help check the changes of the variable's value during the execution of the application and can help the user to determine the error in the program.

4.6.3.1 Design

The design of the remote debug system is as shown in *Figure 51*, the input of the system is the application in the target board, when running the GDB debugger, the messages in the application would be printed out in the terminal, the variables in the program would show in the Linux host machine and if the program contains error, the error message would be printed.

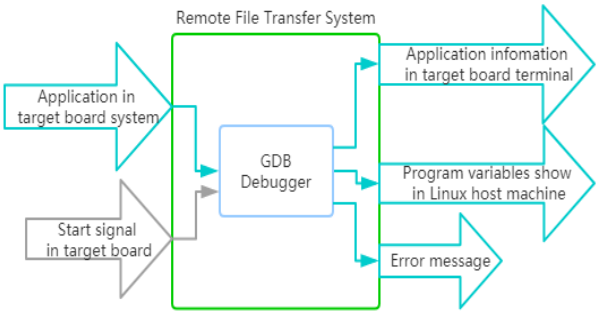


Figure 51 Decomposition diagram for Remote Debug System

4.6.3.2 Subsystem verification results

The verification result for remote debug system is as shown in *Figure 52*, the application is running on the target board, the remote debug system is running in CCS in Linux host machine.

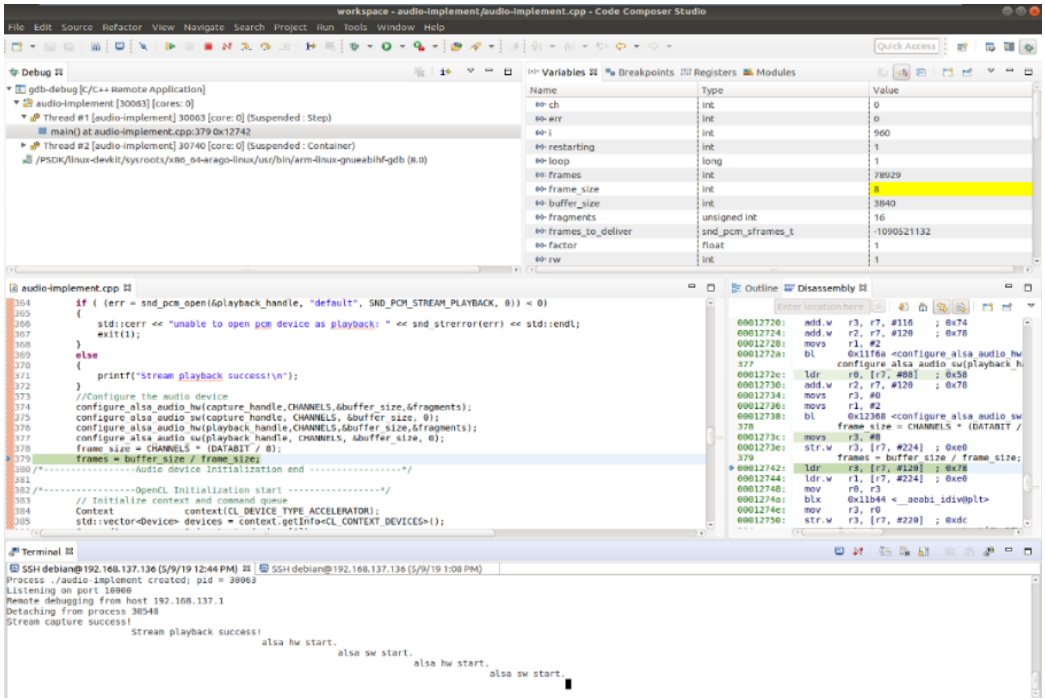


Figure 52 Verification result for Remote Debug System

Result analysis: The application is running on the target board and the program for the application can be seen in the Linux host machine. The changes in a variable when executing a line of code in the program has been highlighted, the memory address for the line of code has been shown, the message within the program has been printed out in the terminal and the debugger can successfully debug the entire program. Therefore, the remote debug system has been successfully designed and verified.

4.7 System verification & validation result

Figure 53 shows the screenshot of the result when executing the test plan for the development system, showing the remote debugging for the audio implementation application. The remote debug has been successfully started and showed information of the application running, in which the application was cross-compiled in Linux host machine and transferred to target board via remote file transfer.

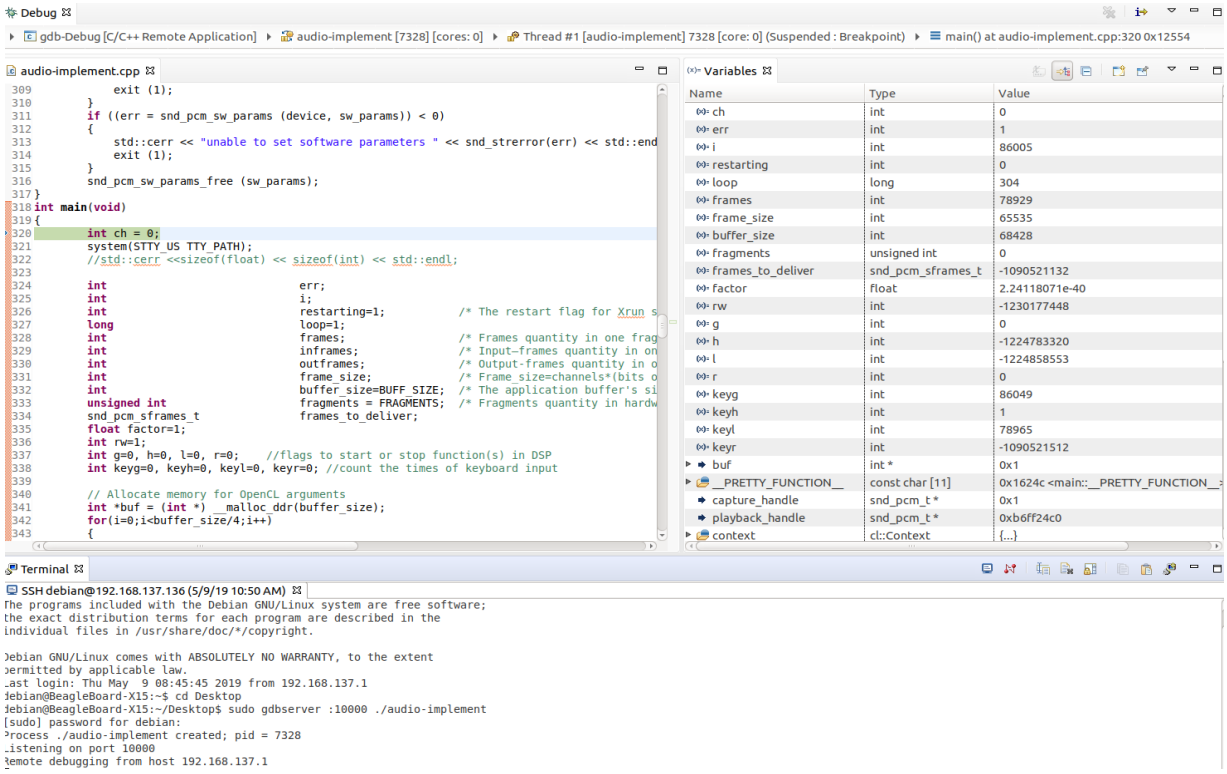


Figure 53 Development System verification & validation test

And the test result is shown in *Table 9*, checking the status before and after the test.

Criteria	Result
Does the target board have an “audio-implement” application before executing the test plan?	No
After the remote file transfer, is there an “audio-implement” application in target board?	Yes
After the cross-compile, can the “audio-implement” application run on the target board?	Yes
Can the “audio-implement” application be debugged in the Linux host machine?	Yes

Table 9 Development system verification & validation result

Result analysis: From the result in *Table 9*, all the functions in the development system has been verified. At first, there is no "audio-implement" application in the target board system, then after cross-compile and remote file transfer, the "audio-implement" application can run on the target board and the status of the application running on the target board can be view by running the remote debugger in Linux host machine.

And by checking the requirements for the development system as shown below:

- The system must be able to cross-compile on the Linux host machine for the BeagleBoard-X15.
- The system must be able to debug the cross-compiled program on Linux host machine.

In conclusion, the development system has been successfully designed and fulfilled all the requirements.

5. Discussion

In this chapter, the discussion will be about the results of audio implementation system, development system and their subsystems, the method used in this project and the answers to the sub-questions.

The Audio Implementation System has realized the function of audio input, processes audio data on Sitara DSP using the implemented algorithm from Simulink audio applications, and audio output. The Development System can realize the functions of cross-compile, file transfer and remote debugging.

As for the result of the subsystems for the Audio Implementation System, the Audio Process System has been successfully rebuilt and modified for this project, realizing the function of audio input, volume changing with keyboard input, and audio output. The Keyboard Capture System can realize the function of using keyboard input as a flag to start or stop functions in the DSP. The Simulink Code System has realized the workflow of implementing a Simulink audio application on the Sitara DSP. And finally, the DSP Invoke & Execute System has realized the functions of transferring data and variables from CPU to DSP, invoke the DSP and execute functions according to keyboard input on the Sitara DSP.

For the subsystems in the Development System, the Remote File Transfer system can transfer the compiled application or other files between the Linux host machine and the target board, the Cross-compile system has realized the function of compiling in the Linux host machine for the target board, and the Remote Debug System can debug the application running on the target board in the Linux host machine.

This thesis project followed the V-model design method, designing and verifying the system and subsystems. The systems were first designed, then the subsystems and components. The test plans were developed in the early stage of the project and were carried out when the corresponding design of the system and subsystems were completed. The tests for subsystems were first carried out, after the subsystems have passed the test, they were integrated into systems and were tested with the system test plans. When the systems have passed the system test plan, the design of the systems has been successfully completed.

And the answers to the sub-question are listed in below table:

Sub-questions and answers
Q1: What are the hardware and software needed for the Audio Implementation System?
A: The hardware involved in this project are AM5728 Sitara processor along with its development platform BeagleBoard-X15 and APx 525 audio analyzer. The software in this project is MATLAB Simulink, Code Composer Studio (CCS), the audio process system's application from the previous project, cross-compiler and remote debugger. The details can be found in *Ch 2 Theoretical Framework*.
Q2: What audio application model to use from Simulink Audio System Toolbox?
A: A high-pass filter audio application model is required to be implemented on the Sitara DSP in this project. And in the Audio Implementation System, the gain minus 5 dB, signal invert and low-pass filter audio application has also used.
Q3: What modification needs to be made to the C-code generated from the Simulink audio application?
A: The most important part of the generated code is the audio processing algorithm in the model step function, the modification needed for that part is to remove the unknown data type for DSP, and cast the data type of the calculated results from float to integer. And sometimes, the algorithm from the generated code might also need some modification to acquire the most stable result, the details are shown in *Ch 4.3.3 Simulink Code System*.
Q4: What is the way to run the implemented audio application on the Sitara DSP?
A: The solution in this project is to implement the audio processing algorithm part in the model step function, which is in the generated code form the audio application, on the DSP kernel. Then uses OpenCL to invoke the DSP to process audio. The details of implementing the code on the DSP can be found in *Ch 4.3.3*, and the details for OpenCL can be found in *Ch 4.3.4*.
Q5: What is the possible way to bypass CPU to transfer audio data stream between buffer and DSP?
A: In this project, the audio data is captured and played back in the CPU, DSP is only used to process audio data. Attempts to directly compile the program for the DSP has failed, and other attempts to capture and playback audio data in the DSP kernel has failed. Therefore, this project has not been able to find a solution to bypass CPU to transfer audio data stream to DSP.
Q6: What are the descriptions and results of the test plans and validation plan for the systems?
A: The system description for the Audio Implementation System can be found in *Ch 4.2*, the result is in *Ch 4.4*. And for the Development system, the description and result are in *Ch 4.5* and *Ch 4.7*. The test plans for the systems are in *Appendix I Test plans*.
Q7: What are the descriptions and results of the test plans for the sub-systems?
A: The descriptions of the subsystem for Audio Implementation System can be found in *Ch 4.3*, and the subsystems for Development System are in *Ch 4.6*. The test plans can be found in *Appendix I Test plans*.

Table 10 Sub-questions and answers

6. Conclusions and recommendations

In this report, the main question for this project has been answered. The audio implementation system, which is a software application designed following the V-model design method, has realized the functions of audio input, executes the implemented Simulink audio application functions on AM5728 Sitara DSP and outputs processed audio. The audio implementation system is developed based on the audio process system from the previous project, the initialization for audio device, capturing audio data and playing back processed audio are all done using ALSA API, which was already developed in the audio process system, and the OpenCL framework is still used to invoke the DSP and deliver data in this project.

And for the development system, the cross-compile, file transfer and remote debug between Linux host machine and target board are done. With this development system, more work can be done in the Linux host machine for the target board, reducing the workload for the target board and strengthening the connection between Linux host machine and the target board.

The design of the systems and subsystems have all passed the corresponding test plans, meeting its expectations. And the test results have been discussed and analyzed in *Ch 4* of the report.

To compare this project to other similar projects, the workflow of having Simulink audio applications run on an embedded system has been done in other projects, however, the embedded system used in those projects have support package from MATLAB Simulink and only need to set up the connection between the development board and Simulink, and the audio applications would run on the CPU directory. As for now, no similar project of manually implementing a Simulink audio application on DSP has been found.

As for the recommendation of this project, the idea of implementing Simulink audio applications on DSP has been proven feasible in the project, and one of the wishes for this project is to implement all the audio applications built by the fellow student in another project. As for now, most of the basic functions have been implemented in the audio implementation system, and the functions can be combined to realize more tasks. However, the DC blocker function has not been implemented yet, this is due to the lack of equipment to test the performance of a DC blocker. Additionally, the mixer function can be realized by using USB audio to add soundcard to the BeagleBoard-X15. In this project, mixing two sounds together, changing the gains of the sounds and switching the sounds on and off for left and right channel on the hardware level has been proven feasible on the BeagleBoard-X15. And due to the limit of time, the possible way to bypass CPU to transfer audio data stream between buffer and DSP has not been found.

For a future project, further development for the USB audio can be done, by finding a way to switch sounds on the left and right channel on the software level and test out multi-channel audio. The research to bypass CPU and transfer audio data between buffer and DSP can also be continued, and as more tasks could be implemented on the DSP, it would also be useful to optimize the OpenCL framework.

References

Analog Devices. (2019). *A Beginner's Guide to Digital Signal Processing (DSP)*. Retrieved Feb. 24, 2019, from https://www.analog.com/en/design-center/landing-pages/001/beginners-guide-to-dsp.html

BeagleBoard. (2019). Retrieved Feb. 21, 2019, from http://beagleboard.org/x15

Cypress Semiconductor Corporation. (2016, July 11). *Inter-IC Sound Bus (I2S).* Retrieved from https://www.cypress.com/file/133906/download

Fox, C. (2019, Feb. 23). *What is Audio Processing?* Retrieved Mar. 23, 2019, from https://www.wisegeek.com/what-is-audio-processing.htm

Grass Valley. (2019). Retrieved Feb. 22, 2019, from https://www.grassvalley.com/about/

He, Z. (2018). *Audio process system within GV broadcast camera.* Breda.

Heath, J. (2017, Feb. 17). *What is a CODEC?* Retrieved Feb. 22, 2019, from https://www.analogictips.com/what-is-a-codec/

Introduction to cross-compiling for Linux. (n.d.). Retrieved Mar. 25, 2019, from https://landley.net/writing/docs/cross-compiling.html

Krambeck, D. (2015, Sept. 13). *An Introduction to Digital Signal Processing*. Retrieved Mar. 23, 2019, from https://www.allaboutcircuits.com/technical-articles/an-introduction-to-digital-signal-processing/

Kridner, J. (2015, Oct. 14). *Introducing BeagleBoard-X15.* Retrieved Feb. 24, 2019, from https://beagleboard.org/static/presentations/BeagleBoard-X15.pdf

Littlefield, M. (2014). *OpenCL and the future of FPGA and DSP development*. Retrieved Feb. 26, 2019, from http://signal-processing.mil-embedded.com/articles/opencl-the-future-fpga-dsp-development/

MathWorks. (2017, Sep. 20). *Simulink Support Package for BeagleBoard Hardware*. Retrieved Apr. 30, 2019, from https://www.mathworks.com/matlabcentral/fileexchange/40310-simulink-support-package-for-beagleboard-hardware

MathWorks. (2019a). Retrieved Feb. 28, 2019, from https://nl.mathworks.com/products/audio-system.html

MathWorks. (2019b). Retrieved Feb. 28, 2019, from https://nl.mathworks.com/products/audio-system/features.html#support-for-c-code-generation

Orinj. (2019). *High pass filter*. Retrieved April 30, 2019, from https://www.recordingblogs.com/wiki/high-pass-filter

Rouse, M. (2005). *I2S (SQUID)*. Retrieved Mar. 25, 2019, from https://whatis.techtarget.com/definition/I2S-SQUID

Sorina. (2016, 10 20). *Designing Software to Aerospace Requirements*. Retrieved 3 1, 2019, from https://www.aversan.com/aerospace-software-requirements/

Steven W. Smith, P. (1999). *The Scientist and Engineer's Guide to*. Retrieved from https://www.dspguide.com/ch28/3.htm

Stranneby, D. (2004). *Digital Signal Processing and Applications*.

Sweetwater. (2016, April 30). *What Is a Highpass Filter? When Should I Use it?* Retrieved Apr. 28, 2019, from https://www.sweetwater.com/insync/what-is-a-highpass-filter-when-should-i-use-it/

Techopedia. (2019). *System on a Chip (SoC)*. Retrieved Mar. 25, 2019, from https://www.techopedia.com/definition/702/system-on-a-chip-soc

Test equipment Solutions Ltd. (2019). *AUDIO PRECISION APX525 Datasheet.* Retrieved Feb. 22, 2019, from http://www.testequipmenthq.com/datasheets/AUDIO%20PRECISION-APX525-Datasheet.pdf

Texas Instruments. (2007). *LOW-POWER STEREO AUDIO CODEC FOR PORTABLE AUDIO/TELEPHONY*. Retrieved Feb. 25, 2019, from file:///C:/Users/ext-yxl02011/Downloads/Texas_Instruments.Datasheet_4.pdf

Texas Instruments. (2018, Oct.). *AM572x Sitara Processors Silicon Revision 2.0 datasheet.* Retrieved Feb. 22, 2019, from http://www.ti.com/lit/ds/symlink/am5728.pdf

Texas Instruments. (2019, Jan. 8). *Debug*. Retrieved Mar. 24, 2019, from http://downloads.ti.com/mctools/esd/docs/opencl/debug/index.html

Texas Instruments. (2019a). *Digital signal processors (DSP) – OpenCL*. Retrieved Feb. 25, 2019, from http://www.ti.com/processors/digital-signal-processors/libraries/OpenCL.html

Texas Instruments. (2019b). *TI OpenCL User's Guide*. Retrieved from http://downloads.ti.com/mctools/esd/docs/opencl/intro.html

Texas Instruments. (2019c). *Code Composer Studio (CCS) Integrated Development Environment (IDE)*. Retrieved Mar. 25, 2019, from http://www.ti.com/tool/CCSTUDIO

Texas Instruments. (2019d). *XDS560v2 System Trace USB & Ethernet Debug Probe*. Retrieved Mar. 24, 2019, from http://dev.ti.com/tirex/#/?link=Development%20Tools%2FDebug%20Probes%2FXDS%2FXDS560v2

Texas Instruments. (2019e). *C6000TM high performance and power efficient DSPs*. Retrieved Mar. 23, 2019, from http://www.ti.com/processors/digital-signal-processors/c6000-floating-point-dsp/overview.html

Texas Instruments. (2019f, Mar. 21). *TI OpenCL User's Guide*. Retrieved April 30, 2019, from http://downloads.ti.com/mctools/esd/docs/opencl/debug/debug_printf.html#dsp-side-opencl-kernel-code

Texas Instruments. (n.d.). *Introduction to OpenCL on TI.* Retrieved Feb. 26, 2019, from https://training.ti.com/sites/default/files/docs/Introduction_to_OpenCL_slides.pdf

Tomarakos, J. (n.d.). *Interfacing I2S-Compatible Audio Devices.* Retrieved from https://www.analog.com/media/en/technical-documentation/application-notes/TN_21065L_I2S.pdf

Tutorials Point. (n.d.). *SDLC - V-Model*. Retrieved 3 1, 2019, from https://www.tutorialspoint.com/sdlc/sdlc_v_model.htm

Winters., L. (2014, Feb 26). Retrieved April 10, 2019, from http://www.adobepress.com/articles/article.asp?p=2171314&seqNum=2

Wong, H. (2018, Sept. 1). *What makes digital signal processing so difficult?* Retrieved Mar. 17, 2019, from https://www.quora.com/What-makes-digital-signal-processing-so-difficult

Appendix I Test plans

This appendix shows the test plans for Audio Implementation System and its subsystems, Development System and its subsystem.

System test plan for Audio Implementation System

1. Aim

This test plan aims to verify whether the Audio Implementation System can meet all the requirements for the Audio Implementation System or not.

2. Functionality to be tested

The Audio Implementation System should realize the audio input function, process the input audio with a high-pass filter, low-pass filter, gain minus 5 dB and signal invert function, then realize the audio output function.

3. Hypotheses

The Audio Implementation system can realize the functions of audio input, high-pass filter, low-pass filter, gain minus 5 dB, signal invert and audio output and can meet all the requirements for it.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is APx 525 audio analyzer, BeagleBoard-X15, Windows PC with virtual Linux host machine and the connecting cables.

5. Actual actions during the test

a. Turn on the Windows PC.
b. Turn on the BeagleBoard-X15 and connect it to the Windows PC via serial cable.
c. Startup VMware Workstation and power on the virtual Linux host machine.
d. Connect the audio analyzer's analog output to the audio input and the analog input to the audio output on BeagleBoard-X15. And also connect the audio analyzer to the Windows PC via USB.
e. Turn on the audio analyzer and start up the APx500 audio analyzer measurement software on the Windows PC.
f. In the audio analyzer software interface, choose the connectors for the input and output configuration as analog balance. Configure the waveform as Sine, 100mVrms, 1kHz and start the waveform generator.
g. In the Linux host machine, open a terminal and enter "sudo su" to perform as a root user, and enter "minicom -w" to connect to the BeagleBoard-X15.
h. Input the user name "debian" and password "temppwd" in the terminal to access the BeagleBoard.
i. In the terminal, enter "cd Desktop" and then "ls" to check if the "audio-implement" application exists.
j. In the terminal, enter "sudo ./audio-implement" to start the application.

k. Make a snapshot to record the initial status of the output waveform, FFT and measured meter value, including RMS level, THD+N, frequency, gain, phase, and peak level in the audio analyzer software.
l. Press “h” once to start up the high-pass filter function and make a screenshot in the audio analyzer software interface to record the waveform, FFT and the value of frequency and peak level. Then press “h” again to stop the high-pass filter function.
m. Press “l” once to start up the low-pass filter function and make a screenshot in the audio analyzer software interface to record the waveform, FFT and the value of frequency and peak level. Then press “l” again to stop the low-pass filter function.
n. Press “g” once to start up the gain minus 5 dB function and make a screenshot in the audio analyzer software interface to record the waveform and the value of RMS level and gain. Then press “g” again to stop the gain minus 5 dB function.
o. Press “r” once to start up the signal invert function and make a screenshot in the audio analyzer software interface to record the waveform and the value of phase angle. Then press “r” again to stop the signal invert function.
p. Press any keys except “h”, “l”, “g”, “r”, “w” and “s” to end the application.

6. Average and bandwidth

The worst situation is the audio implementation system cannot realize all the required functions of audio input, high-pass filter, low-pass filter, gain minus 5 dB, signal invert and audio output. In this case, the audio implementation system fails to meet all the requirements.
The best situation is the audio implementation system can realize all the required functions, able to complete audio input, high-pass filter, low-pass filter, gain minus 5 dB, signal invert and audio output. In this case, the audio implementation system is able to meet all the requirements.

7. Predicted conclusions

When the application starts up, the initial status in the measurement software should be a stable sine wave and the peak in FFT would be at 1k Hz.
When “h” is pressed, the peak value should decrease, the level value for frequencies before 2kHz should decrease and for frequencies over 2kHz should remain the same. And when “h” is pressed again, the measurements should return to the initial status.
When “l” is pressed, the peak value should remain unchanged, the level value for frequencies before 2kHz should remain the same and for frequencies over 2kHz should decrease. And when “l” is pressed again, the measurements should return to the initial status.
When “g” is pressed, the gain should decrease 5 dB, the RMS should also decrease 5 dBV. And when “g” is pressed again, the measurements should return to the initial status.
When “r” is pressed, the phase angle should change 180 deg compared to the initial phase angle. And when “r” is pressed again, the measurements should return to the initial status.

The audio implementation system can realize all the required functions of audio input, high-pass filter, low-pass filter, gain minus 5 dB, signal invert and audio output.

Sub-system test plan for Audio Process System

1. Aim

This test plan aims to check whether the Audio Process System produced in the previous project has been successfully rebuilt in this thesis project, and to check if whether the modification to the original audio process system can have expected result.

2. Functionality to be tested

The functionalities to be tested in the Audio Process System are:

- Audio input function
- Modify audio input data function
- Volume changing function
- Data overflow checking function
- Audio output function

3. Hypotheses

The audio process system can realize the functions of audio input, modify audio input data, volume changing, data overflow checking and audio output.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is APx 525 audio analyzer, BeagleBoard-X15, Windows PC with virtual Linux host machine and connecting cables.

5. Actual actions during the test

a. Turn on the Windows PC.
b. Turn on the BeagleBoard-X15 and connect it to the Windows PC via serial cable.
c. Startup VMware Workstation and power on the virtual Linux host machine.
d. Connect the audio analyzer's analog output to the audio input on BeagleBoard-X15 and the analog input to the audio output on BeagleBoard-X15. And also connect the audio analyzer to the Windows PC via USB.
e. Turn on the audio analyzer and start up the APx500 audio analyzer measurement software on the Windows PC.
f. In the audio analyzer software interface, choose the connectors for the input and output configuration as analog balance. Configure the waveform as Sine, 100mVrms, 1kHz and start the waveform generator.
g. In the Linux host machine, open a terminal and enter "sudo su" to perform as a root user, and enter "minicom -w" to connect to the BeagleBoard-X15.
h. Input the user name "debian" and password "temppwd" in the terminal to access the BeagleBoard.
i. In the terminal, enter "cd Desktop" and then "ls" to check if the "audioprocess" application exists.
j. In the terminal, enter "sudo ./audioprocess" to start the application.

k. Make a snapshot to record the initial status of the output waveform and in the audio analyzer software.
l. The audio input data and modified data would be printed on the screen, make a snapshot of the result.
m. Press “W” once and “S” twice to see the volume increase and decrease and make screenshots of the waveform and gain for every change in the audio analyzer software interface.
n. Press any keys other than “w” and “s” to end the application.

6. Average and bandwidth

The worst condition is the audio process system cannot realize all the functions, which are audio input, modify audio input data, volume changing, data overflow checking and audio output. The audio process system is not successfully rebuilt and modified.
The best condition is the audio process system can realize all the functions, which are audio input, modify audio input data, volume changing, data overflow checking and audio output. The audio process system has been successfully rebuilt and modified.

7. Predicted conclusions

When the application starts up, the initial status in the measurement software should be a stable sine wave. The modified audio data should have the correct value representing the 24-bit audio data value.
When the “W” is pressed, the gain should increase 0.8 dB and the monitor should show a sine wave output waveform. When “S” is pressed once, the gain should return to the initial value, and when “S” is pressed twice, the gain should decrease 0.8 dB compared to the initial value, and the monitor would also output a sine wave.
And there should be no output data value printed in the screen.

Sub-system test plan for Keyboard Capture System

1. Aim

This test plan aims to check whether the Keyboard Capture System can capture the keyboard input correctly and change the flag value to act as a switch.

2. Functionality to be tested

- When keyboard input the value “g”, “h”, “l” or “r”, the corresponding character would be printed in the screen.
- Press a key once, the corresponding flag would set true, press the key again, the flag would set false.

3. Hypotheses

The Keyboard Capture System can capture the keyboard input correctly, print out the corresponding keyboard message and is able to change the flag value.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is APx 525 audio analyzer, BeagleBoard-X15, Windows PC with virtual Linux host machine and connecting cables.

5. Actual actions during the test

a. Turn on Windows PC.
b. Turn on the BeagleBoard-X15 and connect it to the Windows PC via serial cable.
c. Startup VMware Workstation and power on the virtual Linux host machine.
d. In the Linux host machine, open a terminal and enter “sudo su” to perform as a root user, and enter “minicom -w” to connect to the BeagleBoard-X15.
e. Input the user name “debian” and password “temppwd” in the terminal to access the BeagleBoard.
f. In the terminal, enter “cd Desktop” and then “ls” to check if the “keyboard” application exists.
g. In the terminal, enter “sudo ./keyboard” to start the application.
h. Press “g” once, check the printed message, then press “g” again.
i. Repeat step h with pressing “h”, “l” and “r”, make a snapshot of the printed messages.
j. Press any other keys to end the application.

6. Average and bandwidth

The worst condition is the Keyboard Capture System cannot realize all the functions, which are capturing the correct keyboard input, changing flag value with the keyboard input.
The best condition is the Keyboard Capture System can realize all the functions, which are capturing the correct keyboard input, changing flag value with the keyboard input.

7. Predicted conclusions

When “g” is press on the keyboard, the printed message would be “key ‘g’ pressed. Gain -5dB set true”, and when “g” is pressed for the second time, the printed message would be “key ‘g’ pressed. Gain -5 dB set false”. And for “h”, “l” and “r”, the printed message would also be the key the pressed, and the corresponding flag is set to true when the key is pressed for the first time and set false when pressed again. When any other key is pressed, the application would stop running.

Sub-system test plan for Simulink Code System

1. Aim

This test plan aims to check if the Simulink Code System can realize the workflow of implementing Simulink audio applications to the Sitara DSP.

2. Functionality to be tested

- The gain minus 5dB and signal invert function in the audio application can run on the Sitara DSP.
- The calculations of the functions are correct.
- The high-pass filter can run on the Sitara DSP.
- The low-pass filter can run on the Sitara DSP.

- The calculations of the functions are correct.
- The performance of the filters running on Sitara DSP is in consistent with running on Simulink.
- The processed audio data for all the functions do not contain data overflow.

3. Hypotheses

The Simulink Code System can realize the function of running the gain minus 5dB, signal invert, high-pass filter, and low-pass filter functions on the Sitara DSP, the calculations in the DSP are correct, the performance of the filters are in consistent with the performance in Simulink and the value of processed audio data does not contain overflow.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is MATLAB Simulink, APx 525 audio analyzer, BeagleBoard-X15, Windows PC with virtual Linux host machine and connecting cables.

5. Actual actions during the test

a. Turn on Windows PC.
b. Turn on the BeagleBoard-X15 and connect it to the Windows PC via serial cable.
c. Startup VMware Workstation and power on the virtual Linux host machine.
d. Connect the audio analyzer's analog output to the audio input on BeagleBoard-X15 and the analog input to the audio output on BeagleBoard-X15. And also connect the audio analyzer to the Windows PC via USB.
e. Turn on the audio analyzer and start up the APx500 audio analyzer measurement software on the Windows PC.
f. In the audio analyzer software interface, choose the connectors for the input and output configuration as analog balance. Configure the waveform as Sine, 100mVrms, 1kHz and start the waveform generator.
g. Make a snapshot of the initial status of the FFT graph.
h. In the Linux host machine, open a terminal and enter "sudo su" to perform as a root user, and enter "minicom -w" to connect to the BeagleBoard-X15.
i. Input the user name "debian" and password "temppwd" in the terminal to access the BeagleBoard.
j. In the terminal, enter "cd Desktop" and then "ls" to check if the "gain-5db", "signal-invert", "high-pass-filter" and "low-pass-filter" applications exist.
k. In the terminal, enter "sudo ./gain-5db" to start the gain minus 5dB function.
l. In the measurement software, check if the gain has decreased 5dB, then press any key to end the application.
m. After executing the gain-5db application, the audio data result would be printed on the screen, make a screenshot of the result.
n. In the terminal, enter "sudo ./signal-invert" to start the signal invert function.
o. In the measurement software, check if the phase angle has changed 180 deg, then press any key to end the application.
p. After executing the signal invert application, the audio data result would be printed on the screen, make a screenshot of the result.

q. In the terminal, enter “sudo ./high-pass-filter” to start the high-pass filter application.
r. Make a snapshot to record the FFT graph after running the high-pass filter.
s. Press any key to end the application, the audio data result would be printed on the screen, make a screenshot of the result.
t. In the terminal, enter “sudo ./low-pass-filter” to start the low-pass filter application.
u. Make a snapshot to record the FFT graph after running the low-pass filter.
v. Press any key to end the application, the audio data result would be printed on the screen, make a screenshot of the result.
w. Calculate the theoretical values for the gain minus 5dB function, the signal invert function, high-pass filter, and low-pass filter function.
x. Check if the theoretical result matches the printed result.
y. Disconnect the audio analyzer’s analog input and output to the BeagleBoard-X15, and connect the analog output to the Windows PC’s audio input, the analog input to the Windows PC’s audio output.
z. Make a screenshot of the initial FFT graph for MATLAB Simulink.
aa. Run the high-pass filter audio application in Simulink, screenshots the FFT graph in the measurement software.
bb. Stop the high-pass filter audio application in Simulink.
cc. Compare the FFT graph for high-pass filter running in Sitara DSP and Simulink.

6. Average and bandwidth

The worst condition is the Simulink Code System cannot realize all the functions, which are running the gain minus 5dB, signal invert, high-pass filter, and low-pass filter functions on the Sitara DSP, calculation result matches the theoretical result and the processed data value does not have any overflow.
The best condition is the Simulink Code System can realize all the functions, which are running the gain minus 5dB, signal invert, high-pass filter and low-pass filter functions on the Sitara DSP, calculation result matches the theoretical result and the processed data value does not have any overflow.

7. Predicted conclusions

When executing the gain -5dB application, the gain should decrease 5dB in the measurement software, the printed audio data result should match the theoretical result, and the printed message “gain -5 start” should have “[core 0]” in front, representing the function is running on the DSP.
When executing the signal invert application, the phase should change 180 deg in the measurement software, the printed audio data result should match the theoretical result, and the printed message “invert start” should have “[core 0]” in front, representing the function is running on the DSP.
When executing the high-pass filter application, the level for frequencies below 2kHz should decrease and remains the same for frequencies above 2kHz, the printed audio data result should match the theoretical result, the printed message “high-pass filter start” should have “[core 0]” in front, and the high-pass filter running in Simulink should show similar performance to the high-pass filter in Sitara DSP.
When executing the low-pass filter application, the level for frequencies above 2kHz should decrease and remain the same for frequencies below 2kHz. The printed audio data result should match the theoretical result and the printed message “high-pass filter start” should have “[core 0]” in front.
And no data overflow message should be printed on the screen.

Sub-system test plan for DSP Invoke & Execute System

1. Aim

This test plan aims to check if the DSP Invoke & Execute System can transfer audio data and flag value from CPU to DSP, and invoke DSP to process audio with different functions implemented from audio applications.

2. Functionality to be tested

- Audio data and variables on CPU can be transferred to DSP.
- The CPU can invoke the DSP.
- The DSP can execute or stop different function(s) according to the flag value sent from CPU.

3. Hypotheses

The DSP Invoke & Execute System is able to transfer audio data and variables to DSP and DSP can execute or stop audio processing function(s) according to the variables from CPU.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is APx 525 audio analyzer, BeagleBoard-X15, Windows PC with virtual Linux host machine and connecting cables.

5. Actual actions during the test

a. Turn on Windows PC.
b. Turn on the BeagleBoard-X15 and connect it to the Windows PC via serial cable.
c. Startup VMware Workstation and power on the virtual Linux host machine.
d. Connect the audio analyzer's analog output to the audio input on BeagleBoard-X15 and the analog input to the audio output on BeagleBoard-X15. And also connect the audio analyzer to the Windows PC via USB.
e. Turn on the audio analyzer and start up the APx500 audio analyzer measurement software on the Windows PC.
f. In the audio analyzer software interface, choose the connectors for the input and output configuration as analog balance. Configure the waveform as Sine, 100mVrms, 1kHz and start the waveform generator.
g. In the Linux host machine, open a terminal and enter "sudo su" to perform as a root user, and enter "minicom -w" to connect to the BeagleBoard-X15.
h. Input the user name "debian" and password "temppwd" in the terminal to access the BeagleBoard.
i. In the terminal, enter "cd Desktop" and then "ls" to check if the "dsp-invoke-execute" application exists.
j. In the terminal, enter "sudo ./dsp-invoke-execute" to start the application.
k. Make a snapshot to record the initial status of the output waveform, FFT and measured meter value, including RMS level, THD+N, frequency, gain, phase, and peak level in the audio analyzer software.
l. Press "g" and "r" once to start up the gain minus 5 dB and signal invert functions, make a screenshot in the audio analyzer software interface to record the waveform and the value of gain and phase.

m. The audio data result would be printed on the screen, make a snapshot of the data result.
n. Calculate the theoretical value for the gain minus 5 dB and signal invert functions together, check if the result matches the printed result.
o. Then press “g” and “r” again to stop the functions and return to the initial state.
p. Press “h” and “l” once to start up the high-pass and low-pass filter together, make a screenshot in the audio analyzer software to record waveform, FFT, and the value of frequency and peak level.
q. Press “h” and “l” again to stop the high-pass and low-pass filter functions and return to the initial state.
r. Press any other keys to stop the application.

6. Average and bandwidth

The worst condition is the DSP Invoke & Execute System cannot realize all the functions, which are transferring audio data and variables from CPU to DSP, invoking DSP in CPU and execute or stop functions in DSP according to the flag value from CPU.

7. Predicted conclusions

When pressing “g” and “r” once, the gain -5dB and signal invert functions should both be executed, the measurement software should show changes of gain decreasing 5 dB and phase changing 180 deg. The printed message “gain -5 start” and “invert” start should both have “[core 0]” in front, representing both of the functions were executed and both ran on DSP. The printed audio data value should match the theoretical value for the two functions.
When “g” and “r” are pressed again, the graphs and values in the measurement software should return to the initial state.
When “h” and “l” are pressed once, the level for all frequencies should decrease in the FFT graph and the filter effect should be the combination of the high-pass filter and low-pass filter. The frequency with peak level should still be 1k Hz, and the peak value should decrease compare to the initial state.
When “h” and “l” are pressed again, the graphs and values in the software should return to the initial state.

System test plan for Development System

1. Aim

This test plan aims to verify whether the Development System can realize the functions of transferring files between the Linux host machine and the target board, cross-compiling and debugging between the Linux host machine and the target board.

2. Functionality to be tested

The Development System can realize the functions of:

- Transfer files between Linux host machine and target board.
- Cross-compile in Linux host machine for the target board.
- Remote debug between Linux host machine and target board.

3. Hypotheses

The Development System can realize the functions of transferring files between the Linux host machine and target board, cross-compile and remote debug between Linux host machine and the target board.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is CCS (Code Composer Studio), BeagleBoard-X15, Windows PC with virtual Linux host machine and connecting cables.

5. Actual actions during the test

a. Start up the Windows PC.
b. Startup VMware Workstation and power on Linux host machine.
c. Power on BeagleBoard-X15, connect the Ethernet and serial cables to PC.
d. Disconnect the serial port from Windows PC and connect to the Linux host machine.
e. Open a terminal in the Linux host machine, enter “sudo su” and password to run as a root user.
f. Enter “minicom -w” to open a target board terminal. Enter “ifconfig” to get the IP address “192.168.137.xx” for the target board.
g. Startup CCS in the Linux host machine, in the CCS window, go to Window -> Open Perspective -> Other… and choose “Remote System Explorer”, then click ‘OK’.
h. Right-click the blank space in RSE (Remote System Explorer) window, select New -> Connection…, select Linux, click ‘Next’.
i. In ‘Host name’, enter the BeagleBoard’s IP address “192.168.137.xx” and in connection name, enter “bb-x15”, click ‘Next’.
j. Select ‘ssh.files’, click ‘Next’; select ‘processes.shell.linux’, click ‘Next’; select ‘ssh.shells’, click ‘Finish’.
k. In RSE window, right-click ‘bb-x15’ and click ‘Connect’.
l. Enter User ID “debian” and Password “temppwd”, click ‘OK’.
m. Create a new empty C/C++ project named ‘audio-implement’, select ‘Cross GCC’ in ‘Toolchains’.
n. In ‘Cross GCC Command’, Browse the Cross compiler path to /PSDK/linux-devkit/sysroot/x86_64-arago-linux/usr/bin, and in ‘Cross compiler prefix, type in “arm-linux-gnueabihf-“, click ‘Finish”.
o. In the Project Explorer view in CCS, right-click the project ‘audio-implement’ and choose ‘Add Files…” to add the program file ‘audio-implement.cpp’.
p. Right-click the project ‘audio-implement’ again and select ‘Properties’.
q. Navigate to C/C++ Build -> Settings. In Cross G++ Compiler -> includes, add path ‘/PSDK/targetNFS/usr/include’ to include paths; then chose the Cross G++ Linker -> Libraries, in Libarary search path, add ‘/PSDK/targetNFS/usr/lib’, and in Libraries, type in “asound”, “ocl_util” and “OpenCL”, click ‘Apply and Close’.
r. Click the ‘Build’ icon on the toolbar to compile the project.
s. In ‘Project Explorer’, expand the project ‘audio-implement’, expand the directory ‘Debug’, copy the application file ‘audio-implement’, then open the ‘Remote System Explorer’ view, expand ‘bb-x15’ and paste the file in Root -> home -> debian -> Desktop.

t. In the CCS Toolbar, click ‘Open a Terminal’, choose ‘SSH Terminal’; In ‘Host’, enter the target board IP address “192.168.137.xx”; enter ‘User’ as “debian” and ‘Password’ as “temppwd”, click ‘OK’.
u. In the SSH terminal, enter “cd Desktop” and “ls” to check if the file ‘audio-implement’ is there.
v. Enter “sudo chmod +x audio-implement” to give access to execute the file.
w. Enter “gdbserver :10000 audio-implement” in the SSH terminal.
x. In the Project Explorer view, right-click the project ‘audio-implement’ and select Debug As -> Debug Configurations…
y. Create a new ‘C/C++ Remote Application’, for the ‘Name’, enter “audio-implement-Debug”. And select ‘GDB(DSF) Manual Remote Debugging Launcher’.
z. Click the ‘Debugger’ tab, for ‘GDB init file’, browse to ‘/PSDK/.gdbinit’, and for ‘GDB command file’, browse to ‘/PSDK/linux-devkit/sysroot/x86_64-arago-linux/usr/bin/arm-linux-gnueabihf-gdb’.
aa. Under the ‘Connection’ tab, enter “192.168.137.xx” for ‘Host name or IP address’. Then click ‘Debug’.
bb. After debugging is complete, click ‘Resume’ on CCS Toolbar to exit the debug process.

6. Average and bandwidth

The worst situation is the Development System cannot realize all the functions, which are transfer files between Linux host machine and the target board, set up cross-compile and remote debug between Linux host machine and target board.

The best situation is the Development System can realize all the functions for the system, which are transfer files between Linux host machine and the target board, set up cross-compile and remote debug between Linux host machine and target board.

7. Predicted conclusions

The Remote System Explorer connection for the target board can be set up successfully, the compiled application can run on the target board, the remote debugger can be set up between the Linux host machine and the target board.

The Development System can realize all the functions, with transferring files, cross-compiling and remote debugging, and meet all the requirements.

Sub-system test plan for Remote File Transfer System

1. Aim

This test plan aims to verify whether the Remote File Transfer System can realize the functions of transferring files between the Linux host machine and target board using CCS.

2. Functionality to be tested

The function of transferring files between Linux host machine and target board using the CCS in Linux host machine.

3. Hypotheses

The Remote Explorer System can transfer files between Linux host machine and target board.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is CCS, BeagleBoard-X15, Windows PC with virtual Linux host machine and connecting cables.

5. Actual actions during the test

a. Start up the Windows PC.
b. Startup VMware Workstation and power on the Linux host machine.
c. Connect the BeagleBoard to the PC via Ethernet and serial cable, power on BeagleBoard.
d. Open a terminal as root user in Linux host machine, enter "minicom -w" to connect to the BeagleBoard and enter "debian" and "temppwd" as user name and password respectively.
e. Enter "ifconfig" in the terminal, obtain the IP address "192.168.137.xx" for BeagleBoard.
f. In the terminal, enter "cd Desktop" and "ls" to check if there is a 'File_transfer_test', if there is, delete the file.
g. Startup CCS in the Linux host machine.
h. In the CCS window, go to Window -> Open Perspective -> Other… and choose "Remote System Explorer", then click 'OK'.
i. Right-click the blank space in RSE (Remote System Explorer) window, select New -> Connection…, select Linux, click 'Next'.
j. In 'Host name', enter the BeagleBoard's IP address "192.168.137.xx" and in connection name, enter "bb-x15", click 'Next'.
k. Select 'ssh.files', click 'Next'; select 'processes.shell.linux', click 'Next'; select 'ssh.shells', click 'Finish'.
l. In RSE window, right-click 'bb-x15' and click 'Connect'.
m. Enter User ID "debian" and Password "temppwd", click 'OK'.
n. Expand the 'Local' directory in RSE, navigate to My Home -> File_transfer_test, copy the file 'File_transfer_test'.
o. Expand the 'bb-x15', navigate to Root -> home -> debian -> Desktop, and paste the file.
p. In the minicom terminal, enter "cd Desktop", then enter "ls", check if the file 'File_transfer_test' is there.

6. Average and bandwidth

The worst situation is the Remote File Transfer System cannot transfer files between Linux host machine and target board.

The best situation is the Remote File Transfer System can transfer files between Linux host machine and the target board.

7. Predicted conclusions

The 'File_transfer_test' is not in the BeagleBoard's Desktop directory before file transfer, and the file appears in the Desktop directory after file transfer.

The Remote File Transfer System can transfer files between Linux host machine and the target board using the CCS in Linux host machine.

Sub-system test plan for Cross-compile System

1. Aim

This test plan aims to verify whether the Cross-compile System can realize the function of compiling the application on the Linux host machine and run the compiled application on the target board BeagleBoard-X15.

2. Functionality to be tested

The Cross-compile system should realize the functions of compiling the audio implementation C/C++ program on the Linux host machine, and the application obtained from the compilation should be able to run on the target board and fail to run on the Linux host machine.

3. Hypotheses

The Cross-compile System can realize the functions of compile on Linux host machine, the compiled application can run on the target board and fail to run on the Linux host machine.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is CCS, BeagleBoard-X15, Windows PC with virtual Linux host machine and connecting cables.

5. Actual actions during the test

a. Start up the Windows PC.
b. Startup VMware Workstation and power on the Linux host machine.
c. Startup CCS in Linux host machine.
d. Turn on BeagleBoard-X15 and connect the Ethernet and serial cable to PC.
e. Open a terminal as root user in Linux host machine, enter “minicom -w” to connect to the BeagleBoard and enter “debian” and “temppwd” as user name and password respectively.
f. In the CCS window, select File -> New -> Project, and select the C/C++ -> C/C++ Project, then click ‘Next’ to select ‘C++ Managed Build’, click ‘Next’.
g. In ‘Project name’, type in “audio-implement”, for ‘Project type’, select Executable -> Empty Project, and in ‘Toolchains’, choose ‘Cross GCC’, click ‘Next’.
h. In ‘Cross GCC Command’, Browse the Cross compiler path to /PSDK/linux-devkit/sysroot/x86_64-arago-linux/usr/bin, and in ‘Cross compiler prefix, type in “arm-linux-gnueabihf-“, click ‘Finish”.
i. In the Project Explorer view in CCS, right-click the project ‘audio-implement’ and choose ‘Add Files…” to add the program file ‘audio-implement.cpp’.
j. Right-click the project ‘audio-implement’ again and select ‘Properties’.
k. Navigate to C/C++ Build -> Settings. In Cross G++ Compiler -> includes, add path ‘/PSDK/targetNFS/usr/include’ to include paths; then chose the Cross G++ Linker -> Libraries, in

Libarary search path, add '/PSDK/targetNFS/usr/lib', and in Libraries, type in "asound", "ocl_util" and "OpenCL", click 'Apply and Close'.

l. Click the 'Build' icon on the toolbar to compile the project.
m. In 'Project Explorer', expand the project 'audio-implement', expand the directory 'Debug', right-click the application file 'audio-implement' and open the 'Remote System Explorer' view and copy the application file to the desktop of the target board system.
n. In the minicom window, enter "cd ~/Desktop" and "ls" to check if the application file 'audio-implement' is there.
o. Enter "sudo chmod +x audio-implement" and enter "sudo ./audio-implement" to execute the application.
p. Open another terminal in Linux host machine, Enter "cd /CCS/workspace/audio-implement/Debug" and "sudo ./audio-implement" to execute the application.

6. Average and bandwidth

The worst situation is the Cross-compile System cannot realize all the functions, which are successfully compiling the program with cross-compiler on Linux host machine, and the compiled application can run on the target board and fail to run on the Linux host machine.

The best situation is the Cross-compile System can realize all the functions, the program is successfully compiled with cross-compiler on Linux host machine, the compiled application can run on the target board and not on the Linux host machine.

7. Predicted conclusions

The program can be successfully compiled with cross-compiler in CCS on the Linux host machine, the target board can successfully run the compiled application 'audio-implement', and when running the compiled application on Linux host machine, an error message would be printed in the terminal.

The Cross-compile System can compile a program in the Linux host machine, and the compiled application is executable on target board and not executable on Linux host machine.

Sub-system test plan for Remote Debug System

1. Aim

This test plan aims to check if a debugger can be set up in the Linux host machine to debug an application running on the target board.

2. Functionality to be tested

The function to be tested for the Remote Debug System is whether a debugger can be set up in Linux host machine to debug an application running on target board.

3. Hypotheses

The Remote Debug System can set up a debugger in Linux host machine to debug an application running on target board.

4. Target group and devices to test with

The target group is the tester who will perform this test and the colleagues in the company. The device used in this test is APx 525 audio analyzer, BeagleBoard-X15, Windows PC with virtual Linux host machine and connecting cables.

5. Actual actions during the test

a. Startup Windows PC.
b. Startup VMware Workstation and power on Linux host machine.
c. Power on BeagleBoard-X15, connect the Ethernet and serial cables to PC.
d. Open a terminal in the Linux host machine, enter “sudo su” and password to run as a root user.
e. Enter “minicom -w” to open a target board terminal. Enter “ifconfig” to get the IP address “192.168.137.xx” for the target board.
f. Startup CCS and open an SSH Terminal with the IP address “192.168.137.xx”.
g. In the SSH terminal, enter “cd Desktop” and “ls” to check if the file ‘audio-implement’ is there.
h. Enter “sudo chmod +x audio-implement” to give access to execute the file.
i. Enter “gdbserver :10000 audio-implement” in the SSH terminal.
j. In the Project Explorer view, right-click the project ‘audio-implement’ and select Debug As -> Debug Configurations…
k. Create a new ‘C/C++ Remote Application’, for the ‘Name’, enter “audio-implement-Debug”. And select ‘GDB(DSF) Manual Remote Debugging Launcher’.
l. Click the ‘Debugger’ tab, for ‘GDB init file’, browse to ‘/PSDK/.gdbinit’, and for ‘GDB command file’, browse to ‘/PSDK/linux-devkit/sysroot/x86_64-arago-linux/usr/bin/arm-linux-gnueabihf-gdb’.
m. Under the ‘Connection’ tab, enter “192.168.137.xx” for ‘Host name or IP address’. Then click ‘Debug’.
n. After debugging is complete, click ‘Resume’ on CCS Toolbar to exit the debug process.

6. Average and bandwidth

The worst condition is the Remote Debug System cannot set up a debugger in Linux host machine to debug an application running on target board.
The best condition is the Remote Debug System is able to set up a debugger in Linux host machine to debug an application running on target board.

7. Predicted conclusions

When debugger can be started with no error and the entire program for the application can be successfully debugged.

Appendix II System description

Audio Implementation System description

The system description as shown in *Figure 54* shows the inputs and outputs of the Audio Implementation System. For the inputs and outputs of the system, electricity provides power to the BeagleBoard-X15, the audio input into the system, and output the processed audio. Compiling the C/C++ library, C-code from Simulink, the program from Audio Process System and program for DSP kernel, would have the audio implementation application. User can send commands to the system, and the system would respond to the command. If the application runs into an error, an error message would be printed out on the screen. The user can also use the keyboard as input, to control the application.

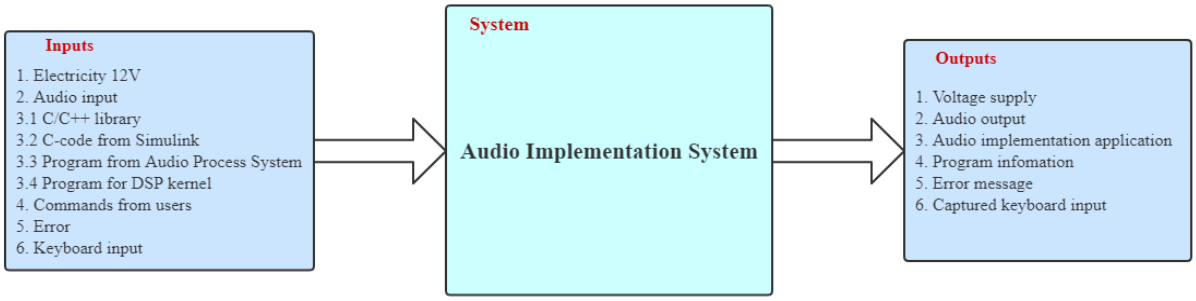


Figure 54 Audio Implementation System description

Development System description

Figure 55 shows the system description for the development system, the 12V electricity supply power to the target board, the SSH (Secure Shell) and serial port connection can transfer the target board data, the cross-compiler can cross-compiled programs and the compiled executable application can run on the target board. The remote debugger can debug the program running on the target board, provide information about the status of the running program and show an error message, the files can be transferred between the three systems, and the network connection provides IP address for each system. And the commands from the user for the three systems would have related changes or response.

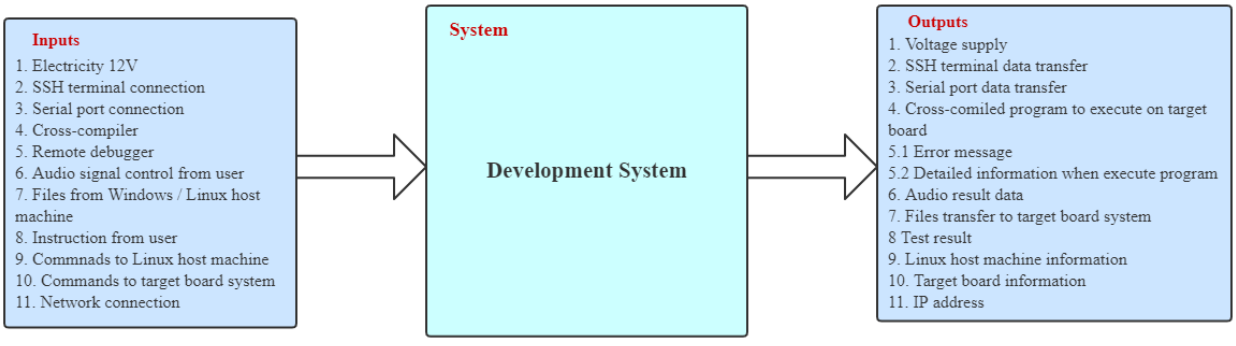


Figure 55 Development system description

Appendix III Detail verification result for Simulink Code System

Verification result for Gain minus 5dB function, Signal invert function

The graphical result of executing the gain minus 5dB and signal invert can be found in the system verification result in *Figure 56* and 57. This part will focus on the verifying whether the results of the functions running on DSP match the theoretical results or not.

As shown in *Figure 56* and *57*, the audio input data is the "Input buffer", and the processed audio output data is the "Output buffer". *Figure 56* shows the result of the gain minus 5dB function running on the Sitara DSP and *Figure 57* shows the result of signal invert function.

```
[core 0] gain -5 start
Input buffer[0] = -4682789 = 1111 1111 1011 1000 1000 1011 1101 1011
Output buffer[0] = -2633325 = 1111 1111 1101 0111 1101 0001 1001 0011

Input buffer[1] = -4765173 = 1111 1111 1011 0111 0100 1010 0000 1011
Output buffer[1] = -2679653 = 1111 1111 1101 0111 0001 1100 1001 1011

Input buffer[2] = -5303788 = 1111 1111 1010 1111 0001 0010 0001 0100
Output buffer[2] = -2982539 = 1111 1111 1101 0010 0111 1101 0111 0101

Input buffer[3] = -5387140 = 1111 1111 1010 1101 1100 1100 0111 1100
Output buffer[3] = -3029411 = 1111 1111 1101 0001 1100 0110 0101 1101

Input buffer[4] = -5833754 = 1111 1111 1010 0110 1111 1011 1110 0110
Output buffer[4] = -3280561 = 1111 1111 1100 1101 1111 0001 0100 1111
```

Figure 56 Result value of gain minus 5 dB function

As shown in *Figure 56* and *57*, the printed statements "gain -5 start" and "invert start" have "[core 0]" before the statements, which is the proof of the function is running on the DSP, as it is described in *Ch 2.3.3 OpenCL.*

And in order to verify the results of the functions, the comparison between the results from the executed functions, as shown in *Figure 56* and *57*, and the theoretical results are as shown in *Table 11* and 12.

(Gain -5dB) Theoretical value = 0.562341325190349O7 * Input buffer			
	Decimal form	Binary form	Result matches?
Buffer [0]			
Input buffer	-4682789	1111 1111 1011 1000 1000 1011 1101 1011	
Output buffer	-2633325	1111 1111 1101 0111 1101 0001 1001 0011	
Theoretical value	-2,633,325	1111 1111 1101 0111 1101 0001 1001 0011	✓
Buffer [1]			
Input buffer	-4765173	1111 1111 1011 0111 0100 1010 0000 1011	
Output buffer	-2679653	1111 1111 1101 0111 0001 1100 1001 1011	
Theoretical value	-2,679,653	1111 1111 1101 0111 0001 1100 1001 1011	✓
Buffer [2]			

Input buffer	-5303788	1111 1111 1010 1111 0001 0010 0001 0100	
Output buffer	-2982539	1111 1111 1101 0010 0111 1101 0111 0101	
Theoretical value	-2,982,539	1111 1111 1101 0010 0111 1101 0111 0101	✓
Buffer [3]			
Input buffer	-5387140	1111 1111 1010 1101 1100 1100 0111 1100	
Output buffer	-3029411	1111 1111 1101 0001 1100 0110 0101 1101	
Theoretical value	-3,029,411	1111 1111 1101 0001 1100 0110 0101 1101	✓
Buffer [4]			
Input buffer	-5833754	1111 1111 1010 0110 1111 1011 1110 0110	
Output buffer	-3280561	1111 1111 1100 1101 1111 0001 0100 1111	
Theoretical value	-3,280,561	1111 1111 1100 1101 1111 0001 0100 1111	✓

Table 11 Result verification for gain minus 5 dB function

```
[core 0] invert start
Input buffer[0] = 2895556 = 0000 0000 0010 1100 0010 1110 1100 0100
Output buffer[0] = -2895556 = 1111 1111 1101 0011 1101 0001 0011 1100

Input buffer[1] = 2826960 = 0000 0000 0010 1011 0010 0010 1101 0000
Output buffer[1] = -2826960 = 1111 1111 1101 0100 1101 1101 0011 0000

Input buffer[2] = 2033846 = 0000 0000 0001 1111 0000 1000 1011 0110
Output buffer[2] = -2033846 = 1111 1111 1110 0000 1111 0111 0100 1010

Input buffer[3] = 1963020 = 0000 0000 0001 1101 1111 0100 0000 1100
Output buffer[3] = -1963020 = 1111 1111 1110 0010 0000 1011 1111 0100

Input buffer[4] = 1133097 = 0000 0000 0001 0001 0100 1010 0010 1001
Output buffer[4] = -1133097 = 1111 1111 1110 1110 1011 0101 1101 0111
```

Figure 57 Result value of signal invert function

(Signal invert) Theoretical value = - Input buffer			
	Decimal form	Binary form	Result matches?
Buffer [0]			
Input buffer	2895556	0000 0000 0010 1100 0010 1110 1100 0100	
Output buffer	-2895556	1111 1111 1101 0011 1101 0001 0011 1100	
Theoretical value	-2,895,556	1111 1111 1101 0011 1101 0001 0011 1100	✓
Buffer [1]			
Input buffer	2826960	0000 0000 0010 1011 0010 0010 1101 0000	
Output buffer	-2826960	1111 1111 1101 0100 1101 1101 0011 0000	
Theoretical value	-2,826,960	1111 1111 1101 0100 1101 1101 0011 0000	✓
Buffer [2]			

Input buffer	2033846	0000 0000 0001 1111 0000 1000 1011 0110	
Output buffer	-2033846	1111 1111 1110 0000 1111 0111 0100 1010	
Theoretical value	-2,033,846	1111 1111 1110 0000 1111 0111 0100 1010	✓
Buffer [3]			
Input buffer	1963020	0000 0000 0001 1101 1111 0100 0000 1100	
Output buffer	-1963020	1111 1111 1110 0010 0000 1011 1111 0100	
Theoretical value	-1,963,020	1111 1111 1110 0010 0000 1011 1111 0100	✓
Buffer [4]			
Input buffer	1133097	0000 0000 0001 0001 0100 1010 0010 1001	
Output buffer	-1133097	1111 1111 1110 1110 1011 0101 1101 0111	
Theoretical value	-1,133,097	1111 1111 1110 1110 1011 0101 1101 0111	✓

Table 12 Result verification for signal invert function

Result analysis: As shown in *Table 11* and *12*, the results from executing the function are the same with the theoretical value, therefore, the gain minus 5 dB and signal invert functions are successfully implemented. And because the printed statements automatically printed "[core 0]" in the front of the statements, the functions are running on the Sitara DSP. And since there was no data overflow message printed, all processed data contain the correct calculated value. Therefore, the Simulink code system is successfully designed and verified.

Verification result for Low-pass filter function

```
[core 0] Low-pass FIR filter start
 Input buffer[0] = 171301 = 0000 0000 0000 0010 1001 1101 0010 0101
 DigitalFilter_states = 104395 = 0000 0000 0000 0001 1001 0111 1100 1011
 acc2[0]= 85650 = 0000 0000 0000 0001 0100 1110 1001 0010
 Output buffer[0] = 137847 = 0000 0000 0000 0010 0001 1010 0111 0111

 Input buffer[1] = 99965 = 0000 0000 0000 0001 1000 0110 0111 1101
 acc1[1]= 49982 = 0000 0000 0000 0000 1100 0011 0011 1110
 acc2[1]= 85650 = 0000 0000 0000 0001 0100 1110 1001 0010
 Output buffer[1] = 135632 = 0000 0000 0000 0010 0001 0001 1101 0000

 Input buffer[2] = 168251 = 0000 0000 0000 0010 1001 0001 0011 1011
 acc1[2]= 84125 = 0000 0000 0000 0001 0100 1000 1001 1101
 acc2[2]= 49982 = 0000 0000 0000 0000 1100 0011 0011 1110
 Output buffer[2] = 134107 = 0000 0000 0000 0010 0000 1011 1101 1011

 Input buffer[3] = 97603 = 0000 0000 0000 0001 0111 1101 0100 0011
 acc1[3]= 48801 = 0000 0000 0000 0000 1011 1110 1010 0001
 acc2[3]= 84125 = 0000 0000 0000 0001 0100 1000 1001 1101
 Output buffer[3] = 132926 = 0000 0000 0000 0010 0000 0111 0011 1110

 Input buffer[4] = 170201 = 0000 0000 0000 0010 1001 1000 1101 1001
 acc1[4]= 85100 = 0000 0000 0000 0001 0100 1100 0110 1100
 acc2[4]= 48801 = 0000 0000 0000 0000 1011 1110 1010 0001
 Output buffer[4] = 133901 = 0000 0000 0000 0010 0000 1011 0000 1101
```

Figure 58 Result value of low-pass filter function

	Printed value	Theoretical value	Result matches?
Input Buffer [0] = 171301; DigitalFilter_states = 104395			
acc2	85650	85650	✓
Output buffer	137847	137848	✓
Input Buffer [1] = 99965			
acc1	49982	49982	✓
acc2	85650	85650	✓
Output buffer	135632	135632	✓
Input Buffer [2] = 168251			
acc1	84125	84125	✓
acc2	49982	49982	✓
Output buffer	134107	134107	✓
Input Buffer [3] = 97603			
acc1	48801	48801	✓
acc2	84125	84125	✓
Output buffer	132926	132926	✓
Input Buffer [4] = 170201			
acc1	85100	85100	✓
acc2	48801	48801	✓
Output buffer	133901	133901	✓

Table 13 Result verification for low-pass function

Result analysis: The verification results from *Table 13* showed that all the intermediate value and output audio data value is correct, and with the data overflow checking, all the data value is within the signed 24-bit data volume. Therefore, the low-pass filter function contains no error in the calculation.

Appendix IV Detail verification result for DSP invoke & execute system

Gain -5dB & Signal invert functions

To verify the value of the executed functions, the value is as shown in *Figure 59* and the verification result is shown in *Table 14*.

```
[core 0] gain -5 start
                        [core 0] invert start
Input buffer[0] = 178373 = 0000 0000 0000 0010 1011 1000 1100 0101
Output buffer[0] = -100306 = 1111 1111 1111 1110 0111 1000 0010 1110

Input buffer[1] = 108469 = 0000 0000 0000 0001 1010 0111 1011 0101
Output buffer[1] = -60996 = 1111 1111 1111 1111 0001 0001 1011 1100

Input buffer[2] = 169287 = 0000 0000 0000 0010 1001 0101 0100 0111
Output buffer[2] = -95197 = 1111 1111 1111 1110 1000 1100 0010 0011

Input buffer[3] = 99329 = 0000 0000 0000 0001 1000 0100 0000 0001
Output buffer[3] = -55856 = 1111 1111 1111 1111 0010 0101 1101 0000

Input buffer[4] = 162593 = 0000 0000 0000 0010 0111 1011 0010 0001
Output buffer[4] = -91432 = 1111 1111 1111 1110 1001 1010 1101 1000
```

Figure 59 Result value of gain -5dB & signal invert function

Theoretical value = -0.56234132519034907 * Input buffer			
	Decimal form	Binary form	Result matches?
Buffer [0]			
Input buffer	178373	0000 0000 0000 0010 1011 1000 1100 0101	
Output buffer	-100306	1111 1111 1111 1110 0111 1000 0010 1110	
Theoretical value	-100306	1111 1111 1111 1110 0111 1000 0010 1110	✓
Buffer [1]			
Input buffer	108469	0000 0000 0000 0001 1010 0111 1011 0101	
Output buffer	-60996	1111 1111 1111 1111 0001 0001 1011 1100	
Theoretical value	-60996	1111 1111 1111 1111 0001 0001 1011 1100	✓
Buffer [2]			
Input buffer	169287	0000 0000 0000 0010 1001 0101 0100 0111	
Output buffer	-95197	1111 1111 1111 1110 1000 1100 0010 0011	
Theoretical value	-95197	1111 1111 1111 1110 1000 1100 0010 0011	✓
Buffer [3]			
Input buffer	99329	0000 0000 0000 0001 1000 0100 0000 0001	
Output buffer	-55856	1111 1111 1111 1111 0010 0101 1101 0000	
Theoretical value	-55856	1111 1111 1111 1111 0010 0101 1101 0000	✓

Buffer [4]			
Input buffer	162593	0000 0000 0000 0010 0111 1011 0010 0001	
Output buffer	-91432	1111 1111 1111 1110 1001 1010 1101 1000	
Theoretical value	-91432	1111 1111 1111 1110 1001 1010 1101 1000	✓

Table 14 Result verification for gain -5dB & signal invert function

Result analysis: The value from the gain -5dB and signal invert function running on the DSP is consistent with the theoretical value, the calculation joining two functions together has no error.

High-pass & Low-pass filter functions

There can also be other combination of functions, *Figure 60* shows the result of combining the high-pass and low-pass filter together. As shown in *Figure 60*, in the FFT graph, the level for all the frequency has decreased, the filtering effect is the combination of high-pass and low-pass filter in *Table 8*.

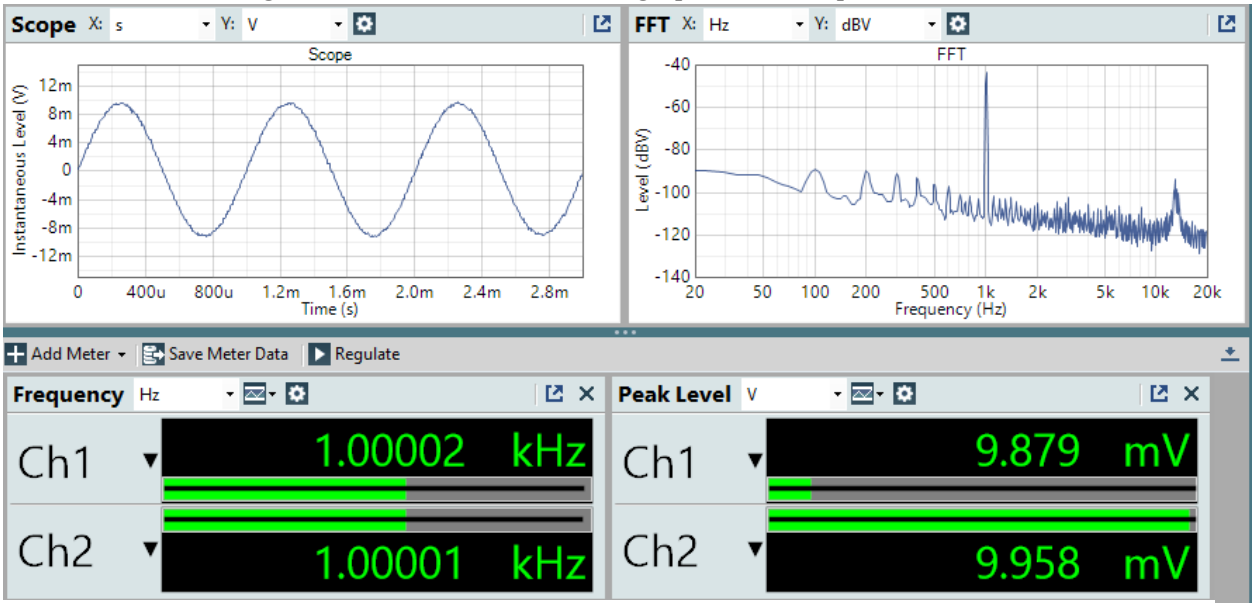


Figure 60 Result of high-pass & low-pass filter

Result analysis: By comparing the FFT graph in *Figure 60* to the initial status in *Figure 39*, the level for all the frequencies has decreased, ranging from -90 to -130 dBV, showing a combination of the high-pass and the low-pass filter in *Table 8*.

Remote file transfer

Components

The remote file transfer system is set up in the software CCS (Code Composer Studio), *Figure 61* shows the configuration to set up a file system for the target board, using the IP address of the target board and naming the connection “bb-x15”.

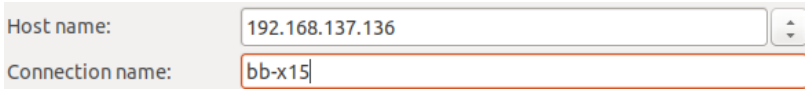


Figure 61 Remote file transfer configuration for BeagleBoard-X15

The file transfer protocol using in this system is the SSH file transfer protocol, and the setup of this protocol in CCS is as shown in *Figure 62* by selecting the ssh.files.

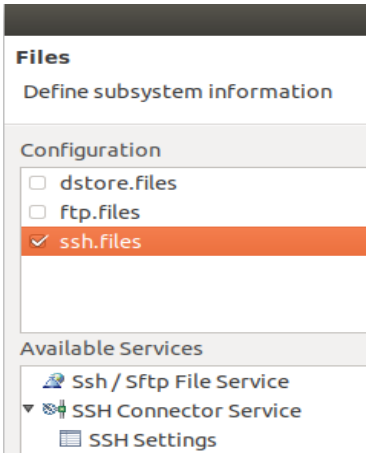


Figure 62 Set up SSH function

Subsystem integration

After setting up the SSH function for the target board, the “Remote Systems” view in CCS, as shown in *Figure 63*, shows the files both in the Linux host machine and in the target board. The “Local” represents the file system in Linux host machine and “bb-x15” is the file system in the target board that has been set up with the SSH function.

In the “Remote System” view, the files can easily copy and paste between Linux host machine and target board.

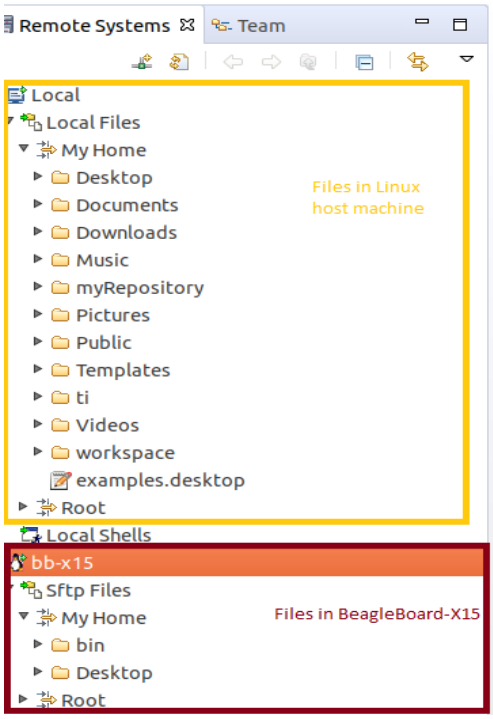


Figure 63 Remote file transfer system

Components

The cross-compile system is also set up in the software CCS, as shown in *Figure 64*, is the cross-compiler configuration for the target board, and the procedure of adding libraries for the program is as shown in *Figure 65*.

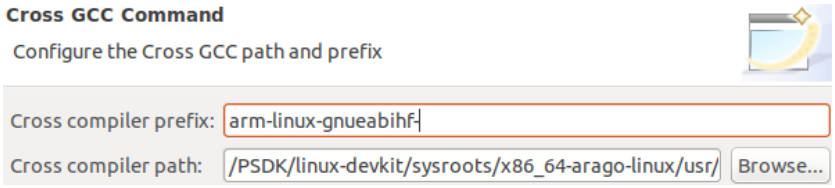


Figure 64 Set up cross-compiler

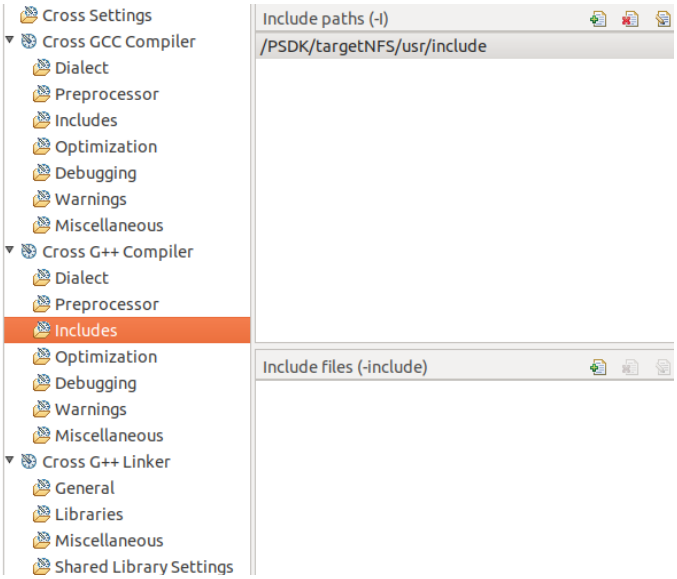


Figure 65 Add libraries for cross-compiler

Subsystem integration

Figure 66 shows the integration of cross-compile system and remote file transfer system. After the program has been cross-compiled in the Linux host machine, the executable application can be transfer to the target board in the remote file transfer system. The integration of these two systems allows the user to develop the software program in the Linux host machine and run the compiled application in the target board.

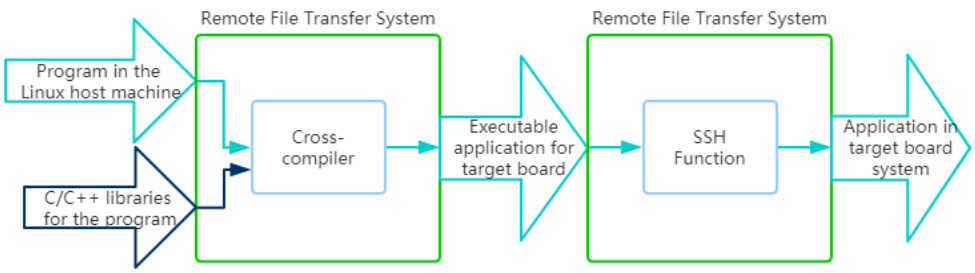


Figure 66 Integration of Cross-compile System and Remote File Transfer System

Components

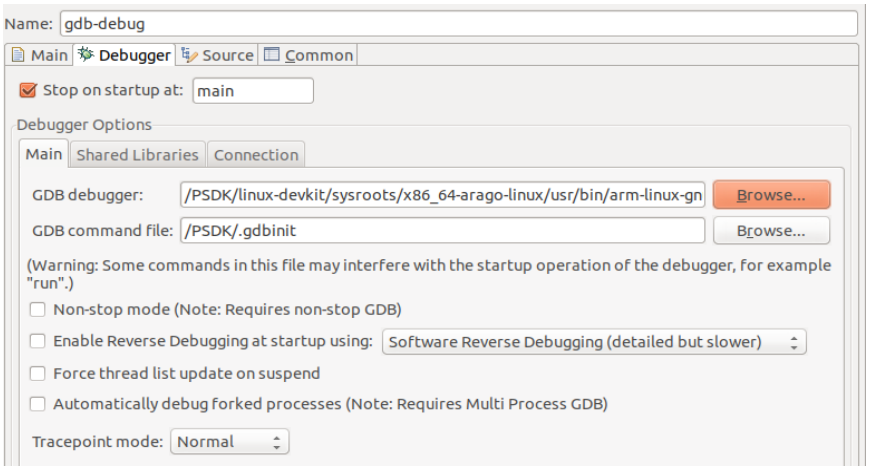


Figure 67 Set up GDB debugger



Figure 68 Set up a connection between debugger and target board

The GDB debugger is set up in CCS, the configuration for the debugger is shown in *Figure 67*, and the debugger needs to be connected to the target board system, which is shown in *Figure 68*, the connection is done using the IP address for the target board.

After the configuration of the debugger in Linux host machine is done, the debugger also needs to be started in the target board. The configuration in the target board is shown in *Figure 69*, invoking the gdbserver to debug the application.

```
debian@BeagleBoard-X15:~/Desktop$ sudo gdbserver :10000 ./audio-implement
[sudo] password for debian:
Process ./audio-implement created; pid = 17324
Listening on port 10000
Remote debugging from host 192.168.137.1
```

Figure 69 Start GDB debugger in target board

Subsystem integration

The integration for Remote debug system is to start up the debug in CCS in the Linux host machine, and invoke the gdbserver on the target board system.